

Aula 21

DataPrev (Perfil 3: Desenvolvimento de Software) Desenvolvimento de Software - 2026 (Pós-Edital)

Autor:
Vinicius Borges

09 de Julho de 2026

Índice

1) Android - Teoria	3
2) Android - Questões Comentadas	40
3) Android - Lista de Questões	57
4) Kotlin - Teoria	78
5) Kotlin - Questões Comentadas	113
6) Kotlin - Lista de Questões	124
7) Swift - Teoria	130
8) Swift - Questões Comentadas	172
9) Swift - Lista de Questões	183

Sumário

Apresentação da Aula.....	2
Android	4
Conceitos Básicos	4
Como navegar no smartphone Android	5
Controle de Processos	7
Fundamentos de aplicativos	7
Gerenciamento de Memória.....	9
Android Emulator.....	12
AndroidManifest	14
Limites da execução em segundo plano	15
Manifest.permission	18
Manifest.permission_group.....	20
Classes	22
Classes do Anadroid App	24
Android Enterprise.....	28
Casos de Uso Android Enterprise	28
WebKit.....	30
Extras!.....	33
Referências	36

APRESENTAÇÃO DA AULA

Olá, pessoal! Hoje vamos falar sobre **desenvolvimento mobile** e as diferentes opções disponíveis para criar aplicativos para dispositivos móveis. Existem várias plataformas e tecnologias que podemos usar, como iOS, Android, Kotlin e desenvolvimento híbrido utilizando frameworks como **React Native, Ionic e Flutter**.

O desenvolvimento para iOS é realizado utilizando a linguagem de programação Swift. O Swift é uma linguagem moderna e poderosa que permite criar aplicativos para iPhone, iPad e outros dispositivos da Apple. Com o iOS, temos acesso a recursos exclusivos da plataforma e podemos aproveitar toda a integração com o ecossistema da Apple.



Por outro lado, o Android é a plataforma mais popular em termos de market share. Para desenvolver aplicativos Android, utilizamos a linguagem Java ou, mais recentemente, a linguagem Kotlin. O Kotlin é uma linguagem moderna e concisa que traz várias melhorias em relação ao Java, tornando o desenvolvimento Android mais eficiente e agradável.

Além do desenvolvimento nativo para iOS e Android, existe também a opção de desenvolvimento híbrido, que permite **criar aplicativos utilizando frameworks multiplataforma**. Entre as opções mais populares estão o **React Native, o Ionic e o Flutter**.

O React Native é baseado no React, uma biblioteca JavaScript amplamente utilizada para a construção de interfaces de usuário. Com o React Native, podemos escrever código em JavaScript que é executado nativamente nos dispositivos iOS e Android, oferecendo uma experiência de usuário similar à de um aplicativo nativo.

O Ionic é outro framework híbrido que utiliza tecnologias web, como HTML, CSS e JavaScript, para criar aplicativos para iOS e Android. Ele fornece um conjunto de componentes e ferramentas para o desenvolvimento rápido e fácil de aplicativos móveis.

Já o Flutter é um framework desenvolvido pelo Google que permite criar aplicativos nativos para iOS e Android a partir de um único código-base em Dart. Ele utiliza widgets personalizados e oferece um alto desempenho, resultando em aplicativos fluidos e responsivos.

Quando estamos projetando soluções mobile, também precisamos considerar a arquitetura do aplicativo. Uma arquitetura de solução mobile eficiente é crucial para garantir a escalabilidade, manutenibilidade e testabilidade do aplicativo.

Algumas das arquiteturas populares no desenvolvimento mobile incluem o MVC (Model-View-Controller), MVP (Model-View-Presenter) e MVVM (Model-View-ViewModel). Essas arquiteturas ajudam a separar as responsabilidades do aplicativo e promovem uma estrutura organizada para o código.

No desenvolvimento mobile, temos uma infinidade de opções de tecnologias, plataformas e arquiteturas para escolher. Cada uma delas tem suas vantagens e desvantagens, e a escolha depende dos requisitos do projeto, da equipe de desenvolvimento e do público-alvo. Agora é hora de debruçar nos estudos e aprender sobre dispositivos móveis! 😊

ANDROID

Conceitos Básicos

O **Android** é um sistema operacional desenvolvido para dispositivos móveis, como smartphones e tablets. Ele utiliza o kernel Linux como sua base fundamental, que é responsável por gerenciar recursos de hardware, como memória, processamento e comunicação com dispositivos externos. O kernel Linux foi escolhido para o Android devido às suas vantagens, como estabilidade, segurança e suporte a uma ampla variedade de hardware. Além disso, o kernel Linux possui uma grande comunidade de desenvolvedores que continuamente atualizam e aprimoram o sistema.

(CEBRASPE (CESPE) - 2022 - POLITEC RO) Assinale a opção em que é apresentado o kernel utilizado para Android.

- a) Kernel MS-DOS
- b) Kernel Solaris
- c) Kernel Windows
- d) Kernel Linux
- e) Kernel Symbian

Comentários: A opção correta é a letra D: Kernel Linux. O Android utiliza o kernel Linux como base para seu sistema operacional. O kernel Linux foi escolhido devido às suas vantagens de estabilidade, segurança e suporte a uma ampla variedade de hardware. É importante ressaltar que o Android é um sistema operacional de código aberto, desenvolvido principalmente para dispositivos móveis, como smartphones e tablets. (Gabarito: Letra D)



Além do kernel, o Android possui uma série de bibliotecas adicionais que facilitam a adição de funcionalidades e recursos extras aos aplicativos. Algumas dessas bibliotecas incluem:

- **Jetpack:** Jetpack é um conjunto de bibliotecas que oferece ferramentas e componentes para acelerar o desenvolvimento de interfaces de usuário (UI) no Android. Ele inclui o Jetpack Compose, que é uma moderna ferramenta para a criação de interfaces de usuário nativas do Android, simplificando o desenvolvimento com menos código e APIs intuitivas em Kotlin.
- **AndroidX:** O AndroidX é uma versão refatorada das APIs do Android que não são incluídas no sistema operacional. Ele oferece suporte a bibliotecas como o Constraint Layout,

que fornece APIs para a construção de layouts baseados em restrições, e o Data Binding Library, que ajuda a escrever layouts declarativos e minimiza o código necessário para vincular a lógica do aplicativo aos layouts.



- **Bibliotecas do dispositivo:** Existem bibliotecas específicas para dispositivos, como a Android Automotive Library, que fornece APIs para a construção de aplicativos para veículos automotivos, e a Android Game Development Kit libraries, que facilitam a criação e otimização de jogos.

- **Outras bibliotecas:** O Android também possui bibliotecas como o Material Components, que ajuda os desenvolvedores a implementar o design do Material Design em seus aplicativos, e o Android NDK, que permite implementar partes do aplicativo em código nativo, como C e C++.

Além dessas bibliotecas, o Android conta com o **Android Gradle Plugin**, que é o sistema de compilação oficial para aplicativos Android, e o Google Play Services, que oferece

bibliotecas para integração com recursos do Google Play, como faturamento em aplicativos, atualizações e integração com o Google Assistant.

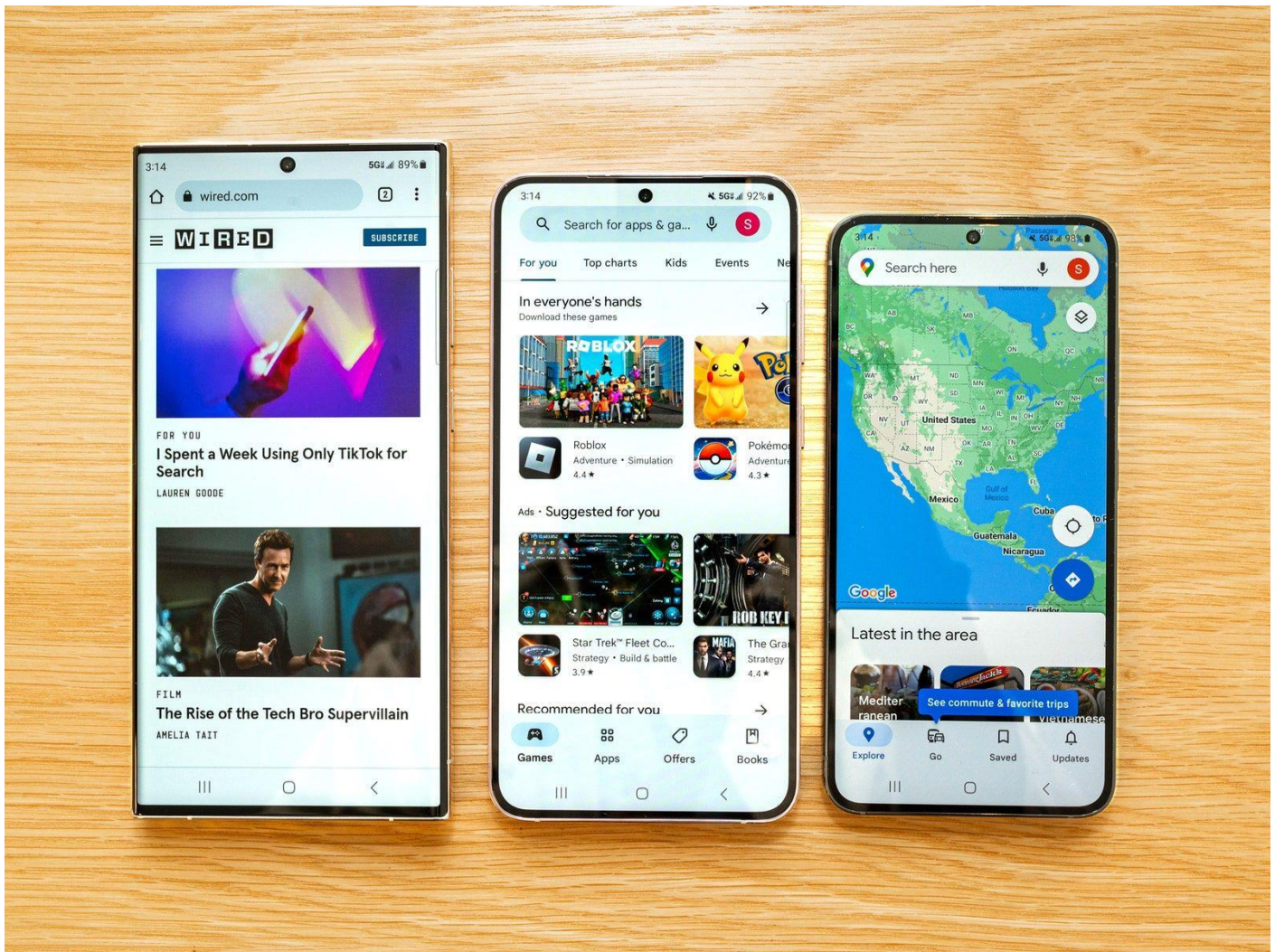
Algumas bibliotecas mais antigas, como a Android Support Library e a Android Test Support Library, foram substituídas pelas bibliotecas AndroidX e não estão mais sendo mantidas, mas ainda são fornecidas para suporte a aplicativos legados.

Como navegar no smartphone Android

É possível navegar pelo seu smartphone Android de várias maneiras, dependendo das configurações do sistema e das preferências do usuário. Ao escolher uma opção de navegação, você pode alternar entre aplicativos, páginas da web e telas com facilidade. Aqui estão algumas dicas para ajudá-lo a navegar pelo seu dispositivo Android.

Para começar, abra o aplicativo Configurações no seu smartphone. Em seguida, vá para a seção "Sistema" e procure a opção "Gestos e Navegação". Dentro das configurações de navegação, você terá diferentes opções disponíveis. Uma delas é a "Navegação por gestos", onde você pode usar gestos na tela para navegar, sem a necessidade de botões físicos. Essa opção oferece uma experiência mais fluida e maximiza o espaço da tela. Outra opção é a "Navegação com dois botões", que inclui botões para a tela inicial e para voltar. Essa é uma opção mais tradicional, que pode ser preferida por quem está acostumado com a navegação clássica do Android.

Há também a "Navegação com três botões", que adiciona um botão adicional para acessar a visão geral dos aplicativos recentes. Com esse botão, você pode alternar rapidamente entre os aplicativos abertos e acessar funções de multitarefa.



Paolla, por que você está me dizendo isso? Eu já sabia! Perfeito, então está apto a acertar a questão que já caiu em concurso!

(UNESC – Pref Criciúma/ 2019) As primeiras versões do sistema operacional android apresentavam a barra de navegação de forma física. Com o surgimento de novas versões do sistema a barra de navegação ficou virtual. Os botões que compõem a barra de navegação do android são?

- a) Desligar, aumentar e diminuir volume.
- b) Menu, voltar e aplicativos abertos.
- c) Voltar, iniciar e aplicativos abertos.
- d) Menu, volta, Iniciar.

Comentários:

Os botões que compõem a barra de navegação do Android são: b) Menu, voltar e aplicativos abertos. Nas versões mais recentes do sistema operacional Android, a barra de navegação é composta pelos seguintes botões: Botão Voltar: O botão voltar permite que você retorne à tela anterior, seja dentro de um aplicativo ou na navegação geral do sistema. Botão Iniciar: O botão Iniciar, também conhecido como Home, leva você de volta à tela inicial do dispositivo. Pressioná-lo enquanto estiver em um aplicativo ou em qualquer outra

tela leva você à tela inicial. Botão de Aplicativos Abertos: Esse botão, representado por um ícone de quadrado ou listra, permite visualizar e alternar rapidamente entre os aplicativos recentemente usados. Ao pressioná-lo, uma lista dos aplicativos abertos ou em segundo plano é exibida, permitindo que você escolha o aplicativo desejado para acessá-lo novamente. Esses botões facilitam a interação com os aplicativos, a navegação entre telas e o acesso rápido aos aplicativos recentes. É importante notar que as versões mais recentes do Android podem ter variações na interface e na aparência dos botões de navegação, mas a funcionalidade básica geralmente permanece a mesma. (Gabarito: Letra B)

Controle de Processos

No contexto do controle de processos do sistema operacional Android, é importante compreender alguns aspectos relacionados às características desse sistema. O Android é um sistema operacional utilizado em dispositivos móveis, como celulares e tablets. Desenvolvido por um consórcio liderado pelo Google, ele oferece diversas funcionalidades que permitem realizar uma ampla variedade de atividades.

Uma das principais características do Android é o seu caráter aberto, o que significa que múltiplos desenvolvedores podem contribuir para aprimorar o sistema conforme surgem novas tecnologias e necessidades. Essa abertura proporciona flexibilidade e inovação contínua ao ecossistema Android.

No contexto do sistema operacional Android, há diferentes componentes que desempenham funções específicas. Por exemplo, os broadcast receivers são responsáveis por **receber e tratar eventos** provenientes do sistema ou de outras aplicações. Eles não possuem interface direta com o usuário, mas podem gerar notificações de alerta.

O broadcast receiver é um componente que faz o sistema entregar eventos ao aplicativo fora de fluxo de usuários comum. Isso permite que o aplicativo responda a anúncios de transmissão por todo o sistema. Como os broadcast receivers são mais uma entrada bem definida no aplicativo, o sistema consegue entregar transmissões até a aplicativos que não estejam em execução no momento. Por exemplo, um aplicativo pode programar um alarme para postar uma notificação que avise o usuário sobre um evento futuro. Ao entregar esse alarme a um receptor de transmissão, o aplicativo não precisa permanecer em execução até o alarme ser desativado. Muitas transmissões têm origem no sistema — por exemplo, uma transmissão que informa o desligamento da tela, bateria com pouca carga ou captura de imagem.

Aplicativos também podem iniciar transmissões — por exemplo, para permitir que outros aplicativos saibam que alguns dados foram salvos no dispositivo e que estão prontos para o uso. Embora os broadcast receivers não exibam nenhuma interface do usuário, eles podem criar uma notificação na barra de status para alertar ao usuário quando ocorre um evento de transmissão. Mais comumente, no entanto, um broadcast receiver é somente um gateway para outros componentes e realiza uma quantidade mínima de trabalho. Por exemplo, ele pode programar um JobService para executar um trabalho com base no evento com JobScheduler. Os broadcast receivers são implementados como subclasses de BroadcastReceiver e cada transmissão é entregue como um objeto Intent.

Fundamentos de aplicativos

Os apps Android podem ser escritos usando-se Kotlin, Java e linguagens C++. Entretanto, Kotlin e Java são **oficialmente suportadas pelo Google para o desenvolvimento de aplicativos Android no ambiente de desenvolvimento Android Studio**. As ferramentas do Android SDK compilam o código em conjunto com

todos os arquivos de dados e recursos em um APK, um pacote Android, que é um arquivo de sufixo .apk. Os arquivos de APK contêm todo o conteúdo de um app Android e são os arquivos que os dispositivos desenvolvidos para Android usam para instalar o aplicativo.

Cada app Android é ativado na própria sandbox de segurança, protegido pelos seguintes recursos de segurança do Android:

- **O sistema operacional Android é um sistema Linux multiusuário em que cada aplicativo é um usuário diferente.**
- Por padrão, o sistema **atribui a cada aplicativo um código de usuário do Linux exclusivo** (o código é usado somente pelo sistema e é desconhecido para o aplicativo). O sistema define permissões para todos os arquivos em um aplicativo, de modo que somente o código de usuário atribuído àquele aplicativo pode acessá-los.
- **Cada processo tem a própria máquina virtual (VM)**, portanto, o código de um aplicativo é executado isoladamente de outros aplicativos.
- Por padrão, **cada aplicativo é executado no próprio processo do Linux**. O Android inicia o processo quando é preciso executar algum componente do aplicativo. Em seguida, encerra-o quando não mais é necessário ou quando o sistema precisa recuperar memória para outros aplicativos.

O sistema operacional Android adota o princípio do **privilegio mínimo** para garantir a segurança e a privacidade dos usuários. Cada aplicativo é executado em sua própria sandbox de segurança, que é um ambiente isolado do sistema operacional e dos demais aplicativos. Isso significa que cada aplicativo tem acesso apenas aos componentes essenciais para o seu funcionamento, sem permissão para acessar outras partes do sistema ou dados de outros aplicativos sem autorização explícita.

Essa abordagem de sandboxing é implementada por meio de mecanismos de controle de acesso, como as permissões do sistema. As permissões são declaradas em arquivos XML durante o desenvolvimento do aplicativo, e elas definem quais recursos e funcionalidades um aplicativo pode acessar. Essas permissões são estáticas e definidas pelo desenvolvedor, sendo verificadas pelo sistema durante a instalação do aplicativo. Assim, cabe ao desenvolvedor garantir que as permissões solicitadas sejam apropriadas e estejam em conformidade com as necessidades do aplicativo.

A sandbox de segurança também impede que um aplicativo interfira nos outros aplicativos ou acesse dados privados sem permissão. Cada aplicativo possui sua própria área de armazenamento privada, onde pode armazenar seus dados de forma isolada. Isso garante que os dados de um aplicativo não sejam acessíveis diretamente por outros aplicativos, a menos que seja concedida permissão explícita.

Além disso, o Android oferece recursos adicionais para compartilhar dados entre aplicativos e acessar serviços do sistema. Por exemplo, é possível utilizar os Content Providers, que são componentes do sistema que permitem o compartilhamento estruturado de dados entre aplicativos. Esses componentes facilitam a troca de informações de maneira controlada e segura, garantindo que apenas os dados autorizados sejam compartilhados entre os aplicativos.

O controle de permissões é um aspecto fundamental no Android. As permissões são estaticamente declaradas em arquivos XML durante o desenvolvimento do aplicativo. Essas permissões definem quais recursos e funcionalidades um aplicativo pode acessar. É importante ressaltar que a responsabilidade de definir as permissões recai sobre o desenvolvedor, não sobre o usuário. Assim, cabe ao desenvolvedor

garantir que as permissões solicitadas pelo aplicativo sejam adequadas e estejam em conformidade com as necessidades do mesmo.

Uma possibilidade é dois aplicativos compartilharem o mesmo código de usuário do Linux, o que permite que eles acessem os arquivos um do outro. Esses aplicativos também podem ser combinados para serem executados no mesmo processo Linux e compartilharem a mesma máquina virtual (VM), visando otimizar o uso dos recursos do sistema. Para isso, é necessário que esses aplicativos tenham o mesmo certificado.

Além disso, um aplicativo pode solicitar permissões específicas para acessar dados do dispositivo, como contatos, mensagens SMS, armazenamento externo, câmera, Bluetooth, entre outros. É importante ressaltar que essas permissões devem ser concedidas explicitamente pelo usuário.

Os componentes fundamentais de uma biblioteca, juntamente com o arquivo de manifesto que declara os componentes e os recursos necessários para o aplicativo, são elementos essenciais para a definição e funcionamento adequado de um aplicativo Android. Além disso, os recursos separados do código do aplicativo permitem otimizar o comportamento do aplicativo para diferentes configurações de dispositivo.

Dessa forma, o Android adota medidas de segurança e controle de acesso que garantem um ambiente confiável e protegido para os aplicativos, ao mesmo tempo em que oferece flexibilidade para compartilhar dados e acessar serviços do sistema, desde que devidamente autorizados e dentro das limitações estabelecidas.

Portanto, essa estrutura de segurança do Android, com sandbox de segurança, atribuição de códigos de usuário exclusivos e execução isolada em processos separados, é fundamental para manter a integridade e a proteção dos dados entre os aplicativos. Isso garante que cada app tenha sua identidade de usuário exclusiva e que a interação entre os aplicativos seja controlada e segura.

(CEBRASPE – TCE-PA / 2016) O Android, sistema operacional Linux multiusuário em que cada aplicativo é visto como um usuário diferente, atribui a cada aplicativo uma identidade de usuário exclusiva.

Comentários:

Perfeita questão! Vejam que há um alinhamento bem intenso em relação à documentação. O Android, como sistema operacional Linux multiusuário, possui uma abordagem de segurança em que cada aplicativo é tratado como um usuário individual. Essa abordagem é essencial para garantir a proteção e o isolamento entre os aplicativos. Ao iniciar um app Android, ele é ativado em sua própria sandbox de segurança, que é um ambiente isolado. O sistema operacional atribui a cada aplicativo um código de usuário exclusivo, desconhecido para o aplicativo em si. Isso significa que somente o código de usuário atribuído ao aplicativo pode acessar seus arquivos e recursos. Além disso, cada aplicativo é executado em seu próprio processo do Linux, com sua própria máquina virtual (VM). Isso proporciona um nível adicional de isolamento, garantindo que o código de um aplicativo seja executado separadamente dos demais. O Android inicia o processo de um aplicativo quando necessário e encerra-o quando não está mais em uso, liberando memória para outros aplicativos. (Gabarito: Correto)

Gerenciamento de Memória

O Android Runtime (ART) e a máquina virtual Dalvik usam paginação e mapeamento em memória (mmapping) para gerenciar a memória. Isso significa que qualquer memória modificada por um app, seja por alocação de objetos ou manipulação de páginas mapeadas, permanece armazenada na RAM e não pode ser despaginada. A única forma de liberar memória de um app é remover as referências aos objetos, permitindo que o coletor de lixo os recupere. No entanto, há uma exceção: arquivos mapeados em memória que não foram modificados, como códigos, podem ser despaginados da RAM se o sistema precisar usar essa memória em outro lugar.

A coleta de lixo é o mecanismo utilizado pelo ART e Dalvik para liberar memória não utilizada. Esse processo monitora cada alocação de memória e libera recursos usados por objetos que não serão mais acessados no programa. O heap de memória do Android é geracional, ou seja, existem diferentes grupos de alocação baseados na vida útil e no tamanho dos objetos. Por exemplo, objetos alocados recentemente pertencem à geração jovem e, conforme sua permanência aumenta, podem ser promovidos a gerações mais antigas.

Cada geração do heap possui um limite máximo dedicado à quantidade de memória que os objetos podem ocupar. Quando uma geração começa a ficar cheia, o sistema realiza uma coleta de lixo para tentar liberar memória. A duração desse processo depende da geração de objetos coletada e da quantidade de objetos ativos em cada geração.

Embora a coleta de lixo seja rápida, ela pode impactar o desempenho do app. Geralmente, não é possível controlar quando a coleta de lixo ocorre, pois o sistema possui critérios próprios para decidir quando executá-la. Se uma coleta de lixo ocorrer durante uma tarefa intensiva de processamento, como uma animação ou reprodução de música, o tempo de processamento pode aumentar, podendo exceder o limite recomendado de 16 ms para uma renderização suave de frames.

Além disso, certos fluxos de código podem aumentar a frequência e a duração dos eventos de coleta de lixo. Por exemplo, alocações excessivas de objetos durante uma repetição intensiva podem poluir o heap de memória, resultando em múltiplas coletas de lixo e impactando o desempenho do app.

O Android busca compartilhar páginas da RAM entre processos para otimizar o uso de memória. Isso é feito por meio de bifurcação de processos, mapeamento em memória de dados estáticos e compartilhamento explícito de regiões de memória dinâmica. Essas técnicas permitem que a memória seja compartilhada e paginada quando necessário, reduzindo o consumo geral de recursos.

Devido ao uso extensivo de memória compartilhada, é importante monitorar cuidadosamente o uso de memória do app. É possível determinar o uso correto de memória por meio de técnicas de investigação detalhadas, conforme explicado em Investigar o uso de RAM.

O heap Dalvik do app é restrito a um único intervalo de memória virtual, que define o tamanho lógico do heap. Esse tamanho pode aumentar dinamicamente, mas apenas até um limite definido pelo sistema para cada app, levando em consideração a quantidade de RAM disponível no dispositivo. É importante lembrar que o tamanho lógico do heap não corresponde diretamente à quantidade de memória física utilizada. O Android calcula o tamanho do conjunto proporcional (PSS), considerando as páginas compartilhadas com outros processos. Esse valor é o que o sistema considera como memória física.

O heap Dalvik não compacta seu tamanho lógico, o que significa que o Android não realiza desfragmentação para liberar espaço contíguo. No entanto, a memória física utilizada pelo heap pode ser reduzida por meio

da coleta de lixo, que identifica páginas não utilizadas e as libera para o kernel por meio do `madvise`. A liberação de memória pode ser menos eficiente para pequenas alocações, já que as páginas usadas por essas alocações podem estar compartilhadas com outros recursos.

O Android estabelece um limite rígido para o tamanho do heap de cada app, visando manter a funcionalidade multitarefa do sistema. Esse limite varia de acordo com a quantidade de RAM disponível em cada dispositivo. Quando um app atinge o limite de heap e tenta alocar mais memória, pode ocorrer um `OutOfMemoryError`.

Para determinar exatamente a quantidade de espaço disponível no heap do dispositivo atual, é possível consultar o sistema usando o método `getMemoryClass()`. Essa chamada retorna o número de megabytes disponíveis para o heap do app.

Quando os usuários alternam entre apps, o Android mantém em cache os apps que não estão em primeiro plano, como uma forma de acelerar a alternância entre eles. Ao abrir um app pela primeira vez, um processo é criado e mantido em cache mesmo quando o usuário sai do app. Isso permite que, ao retornar ao app, o sistema reutilize o processo, tornando a alternância mais rápida.

No entanto, se um app reter recursos desnecessários, mesmo quando não está sendo usado, isso pode afetar o desempenho geral do sistema. Quando o sistema fica com recursos escassos, como memória, ele encerra processos em cache. A decisão de encerramento leva em consideração a quantidade de memória utilizada por cada processo, e o sistema pode encerrar apps de menor prioridade para liberar RAM. Por exemplo, caso o conjunto de aplicativos utilizados exceda o total de memória disponível, o sistema operacional Android irá terminar automaticamente o aplicativo de **menor prioridade**.

A memória de acesso aleatório (RAM) é um recurso valioso em qualquer ambiente de desenvolvimento de software, especialmente em sistemas operacionais móveis com restrições de memória física. Portanto, é crucial gerenciar cuidadosamente a alocação e liberação de memória em aplicativos Android.

Tanto o Android Runtime (ART) quanto a máquina virtual Dalvik realizam a coleta de lixo automaticamente, porém isso não significa que a alocação e liberação de memória podem ser negligenciadas. É importante evitar vazamentos de memória, que ocorrem quando referências de objetos são armazenadas em variáveis estáticas ou quando objetos `Reference` não são liberados adequadamente de acordo com os callbacks do ciclo de vida do aplicativo.

Nesse sentido, é necessário monitorar a disponibilidade e o uso da memória em seu aplicativo. O Android Studio oferece a ferramenta `Memory Profiler`, que permite acompanhar o uso de memória ao longo do tempo, visualizando gráficos em tempo real, número de objetos Java alocados e momentos da coleta de lixo. Além disso, é possível iniciar eventos de coleta de lixo e gravar um snapshot do heap Java durante a execução do aplicativo, permitindo inspecionar alocações de memória, verificar o stack trace para cada alocação e acessar o código correspondente no editor do Android Studio.

Para liberar memória em resposta a eventos e otimizar o equilíbrio do sistema, é recomendado implementar a interface `ComponentCallbacks2` nas classes de `Activity`. O método de callback `onTrimMemory()` permite que o aplicativo detecte eventos relacionados à memória, mesmo quando está em primeiro ou segundo plano, e libere objetos de acordo com o ciclo de vida do aplicativo ou eventos indicativos de que o sistema precisa liberar memória.

Dessa forma, ao adotar boas práticas de gerenciamento de memória, é possível reduzir proativamente o uso da memória em aplicativos Android, garantindo um melhor desempenho e evitando problemas relacionados à escassez de recursos.

Android Emulator

O Android Emulator **simula dispositivos Android no seu computador**, permitindo que você teste seu aplicativo em uma variedade de dispositivos e níveis da API do Android, sem precisar possuir todos esses dispositivos fisicamente. O emulador oferece várias vantagens adicionais.

Em primeiro lugar, ele proporciona flexibilidade ao simular diversos dispositivos, incluindo smartphones, tablets, dispositivos Wear OS e Android TV. Além disso, o emulador vem com configurações predefinidas para esses dispositivos, facilitando o teste em diferentes ambientes.

Em segundo lugar, o emulador oferece alta fidelidade, replicando quase todos os recursos de um dispositivo Android real. Isso significa que você pode simular chamadas telefônicas, mensagens de texto, executar operações como excluir e(ou) recuperar arquivos do emulador, definir a localização do dispositivo, simular diferentes velocidades de rede, girar o dispositivo e acessar a Google Play Store, entre outros recursos.

Por fim, o teste no emulador tende a ser mais rápido e fácil em comparação com um dispositivo físico. Por exemplo, a transferência de dados para o emulador costuma ser mais rápida do que em um dispositivo conectado via USB.

Quadrix – CFO-DF / 2017) O Android possui um emulador que permite simular o sistema operacional real. Contudo, não é possível executar operações como excluir e(ou) recuperar arquivos do emulador.

Comentários:

O Android possui o Android Emulator, um emulador oficial que permite simular um dispositivo Android no computador. É amplamente usado por desenvolvedores para testar aplicativos sem a necessidade de um dispositivo físico. Contrariando a afirmação anterior, é possível realizar operações como excluir e recuperar arquivos no emulador. Ele oferece recursos avançados, incluindo acesso ao sistema de arquivos, permitindo criar, excluir, modificar e recuperar arquivos e pastas dentro do emulador, reproduzindo as ações que seriam feitas em um dispositivo real. Essa funcionalidade é de extrema importância no desenvolvimento e teste de aplicativos, permitindo aos desenvolvedores simular diferentes cenários e comportamentos do sistema operacional Android. (Gabarito: Errado)

O Android é composto por uma variedade de componentes técnicos que desempenham papéis cruciais para o seu funcionamento eficiente. Entre eles, destacam-se o Dalvik e o ART, que são máquinas virtuais responsáveis por executar aplicativos Android. O Dalvik, projetado por Dan Bornstein em conjunto com outros engenheiros do Google, foi substituído pelo ART a partir do Android 5.0. Enquanto o Dalvik era uma máquina virtual baseada em registradores, o ART trouxe melhorias significativas ao fornecer um ambiente de execução mais eficiente e otimizado. Essas máquinas virtuais são responsáveis por **interpretar e executar o código bytecode** dos aplicativos, garantindo um desempenho adequado e uma experiência fluida aos usuários.

Além disso, a plataforma Android também conta com o **Runtime**, que é fornecido pelo **Android Runtime** (ART). Essa máquina virtual está localizada na **camada de aplicação** do sistema operacional Android e desempenha um papel crucial no momento da **execução das aplicações Java**. O Runtime permite que os aplicativos interajam com as diferentes camadas do sistema, garantindo um ambiente de execução seguro e eficiente. Por meio do Runtime, são disponibilizados recursos como gerenciamento de memória, carregamento de classes e otimização do código, proporcionando uma experiência de uso aprimorada e uma resposta rápida aos comandos dos usuários.

Outro componente importante presente no ecossistema Android são os **Content Providers**. Esses componentes são responsáveis por facilitar o compartilhamento estruturado de dados entre os aplicativos. Ao contrário dos aspectos visuais e gráficos, que são tratados pela camada de interface do usuário, os **Content Providers** focam no **gerenciamento seguro e controlado dos dados**. Eles permitem que os aplicativos acessem e compartilhem informações de forma estruturada, mantendo a integridade dos dados e garantindo a privacidade dos usuários. Com os Content Providers, é possível estabelecer uma comunicação eficiente entre os aplicativos, facilitando a troca de informações e enriquecendo a experiência do usuário no universo Android.

AndroidManifest



O arquivo **AndroidManifest.xml** é um componente essencial para o desenvolvimento de aplicativos Android. Ele descreve a estrutura e as configurações do aplicativo, fornecendo informações importantes para o sistema operacional Android. **AndroidManifest.xml** está localizado na raiz do diretório do projeto do aplicativo e é **lido pelo sistema antes de iniciar qualquer componente do aplicativo**, ele declara os componentes do aplicativo.

Em primeiro lugar, o manifesto identifica todas as permissões que o aplicativo precisa solicitar ao usuário. Essas permissões podem incluir acesso à internet, permissão para ler os contatos do usuário ou outros recursos sensíveis. Ao declarar essas permissões no manifesto, o aplicativo informa ao sistema quais recursos ele requer para funcionar corretamente.

Outro aspecto importante do manifesto é a declaração do nível mínimo de API exigido pelo aplicativo. Isso significa que o aplicativo especifica a versão mínima do sistema operacional Android que ele pode ser executado. Isso é essencial para garantir que o aplicativo funcione corretamente em dispositivos com diferentes versões do sistema operacional.

Além disso, o manifesto é usado para declarar os recursos de hardware e software que o aplicativo utiliza ou requer. Isso pode incluir recursos como câmera, Bluetooth ou tela multitoque. Ao declarar esses requisitos no manifesto, o sistema pode verificar se o dispositivo possui os recursos necessários antes de instalar ou executar o aplicativo.

O **AndroidManifest.xml** é escrito em XML (Extensible Markup Language) e contém várias seções e tags que definem as características e comportamentos do aplicativo. Alguns elementos comuns encontrados no manifesto são:

- **<manifest>**: A tag raiz que envolve todo o conteúdo do arquivo manifest.
- **<uses-permission>**: Permite solicitar permissões ao usuário para acessar recursos ou funcionalidades do dispositivo, como câmera, localização ou acesso à internet.
- **<application>**: Define as configurações e componentes principais do aplicativo, como atividades, serviços e provedores de conteúdo.
- **<activity>**: Descreve as atividades do aplicativo, que representam as diferentes telas e interações com o usuário.
- **<service>**: Especifica os serviços em segundo plano que podem ser executados pelo aplicativo.
- **<receiver>**: Define os receptores de transmissão (broadcast receivers), que são componentes capazes de responder a eventos ou ações específicas do sistema ou de outros aplicativos.
- **<provider>**: Descreve os provedores de conteúdo, que permitem compartilhar dados entre aplicativos ou fornecer acesso a dados específicos do aplicativo para outros componentes.
- **<intent-filter>**: Define filtros de intenção (intent filters) para os componentes, especificando quais tipos de ações, categorias ou esquemas de URI eles podem responder.
- **<meta-data>**: Permite adicionar metadados adicionais ao aplicativo.

O `AndroidManifest.xml` é gerado automaticamente pelo Android Studio durante o processo de compilação do aplicativo, com base nas configurações definidas no projeto. É importante manter o manifesto atualizado e revisar cuidadosamente as permissões e configurações necessárias para garantir o bom funcionamento do aplicativo. O arquivo serve como um guia que detalha quais recursos e permissões são necessários para que o aplicativo funcione corretamente.

Dentro do arquivo `Manifest`, existem duas classes que destacamos: `Manifest.permission` e `Manifest.permission_group`. A classe `Manifest.permission` trata das permissões específicas que o aplicativo precisa solicitar ao usuário para acessar determinados recursos protegidos. Por exemplo, a permissão `android.permission.ACCESS_NETWORK_STATE` é utilizada para verificar o estado da conexão de rede do dispositivo.

Essa permissão permite que o aplicativo identifique se o aparelho está conectado a uma rede e obtenha informações sobre o tipo de conexão, como Wi-Fi, dados móveis ou nenhuma conexão. Ao declarar a permissão `android.permission.ACCESS_NETWORK_STATE` no arquivo `Manifest`, o aplicativo informa ao sistema operacional que precisa dessa autorização para executar suas funcionalidades relacionadas à rede.

Já a classe `Manifest.permission_group` agrupa as permissões relacionadas a uma funcionalidade específica. Ela fornece uma organização mais estruturada das permissões, facilitando a compreensão e o gerenciamento das autorizações necessárias para o aplicativo. Por exemplo, a permissão `android.permission_group.LOCATION` agrupa as permissões relacionadas à localização do dispositivo.

Ao utilizar essas classes e suas permissões correspondentes, o desenvolvedor pode garantir que o aplicativo seja capaz de acessar os recursos protegidos necessários para o seu funcionamento adequado, ao mesmo tempo em que solicita ao usuário as autorizações apropriadas para utilizar esses recursos.

Limites da execução em segundo plano

Sempre que um aplicativo é executado em segundo plano, ele consome parte dos recursos limitados do dispositivo, como a RAM. Isso pode resultar em uma experiência reduzida para o usuário, especialmente se ele estiver usando um aplicativo que consuma muitos recursos, como um jogo ou um vídeo. Para aprimorar a experiência do usuário, o Android 8.0 (API de nível 26) restringe o que os aplicativos podem fazer durante a execução em segundo plano. Este documento descreve as mudanças no sistema operacional e como você pode atualizar seu aplicativo para que ele funcione adequadamente com as novas restrições.

Muitos aplicativos e serviços do Android podem ser executados simultaneamente. Por exemplo, um usuário pode jogar em uma janela, navegar na Web em outra e usar um terceiro aplicativo para reproduzir música. Quanto mais aplicativos são executados ao mesmo tempo, maior será a carga para o sistema. Se mais aplicativos ou serviços forem executados em segundo plano, isso deixará o sistema sobrecarregado, o que pode resultar em uma experiência reduzida para o usuário. Por exemplo, o aplicativo de música pode ser desligado repentinamente.

Para diminuir a chance desses problemas, o Android 8.0 estabelece restrições sobre o que os aplicativos podem fazer enquanto os usuários não estão interagindo diretamente com eles. Os aplicativos são limitados de duas maneiras:

- Restrições de serviços em segundo plano: enquanto um aplicativo está ocioso, há limites para o uso dos serviços em segundo plano. Isso não se aplica aos serviços em primeiro plano, que são mais evidentes para o usuário.
- Restrições de transmissão: com exceções limitadas, os aplicativos não podem usar os manifestos para registrar-se em transmissões implícitas. Os aplicativos ainda poderão se registrar para essas transmissões no tempo de execução e usar o manifesto para registro em transmissões explícitas direcionadas especificamente a eles.

Restrições de serviços em segundo plano

Os serviços executados em segundo plano podem consumir recursos do dispositivo, gerando uma experiência ruim para o usuário. Para mitigar esse problema, o sistema aplica diversas restrições aos serviços.

O sistema distingue entre aplicativos de primeiro plano e de segundo plano. A definição de segundo plano para fins de restrições de serviço é diferente da definição usada pelo gerenciamento de memória. Um aplicativo pode estar no segundo plano em termos de gerenciamento de memória, mas em primeiro plano em termos da capacidade de iniciar serviços. Um aplicativo será considerado como estando em primeiro plano se qualquer uma das condições a seguir for verdadeira:

- Ele tem uma atividade visível, independentemente de ela estar em andamento ou pausada.
- Ele tem um serviço de primeiro plano.
- Outro aplicativo está conectado a ele, seja por vinculação de um dos serviços ou pelo uso de um dos provedores de conteúdo. Por exemplo, o aplicativo estará em primeiro plano se outro aplicativo se vincular ao:
 - IME
 - Serviço de plano de fundo
 - Detector de notificações
 - Serviço de voz ou texto

Se nenhuma dessas condições for verdadeira, o aplicativo será considerado como estando em segundo plano.

Restrições de transmissão

Se um aplicativo se registra para receber transmissões, o receptor do aplicativo consome recursos sempre que a transmissão é enviada. Isso pode causar problemas se muitos aplicativos se registraram para receber transmissões com base em eventos do sistema. Um evento do sistema que aciona uma transmissão pode fazer com que todos esses aplicativos consumam recursos rapidamente, prejudicando a experiência do usuário. Para mitigar esse problema, o Android 7.0 (API de nível 24) estabeleceu restrições para transmissões, conforme descrito em Otimização do segundo plano. O Android 8.0 (API de nível 26) torna essas limitações mais rigorosas.

Os aplicativos direcionados ao Android 8.0 ou superior não podem mais registrar broadcast receivers para transmissões implícitas nos manifestos. Uma transmissão implícita é aquela que não é direcionada especificamente ao aplicativo. Por exemplo, o ACTION_PACKAGE_REPLACED é uma transmissão implícita, pois ele é enviado a todos os detectores registrados, informando que alguns pacotes no dispositivo foram substituídos. Entretanto, o ACTION_MY_PACKAGE_REPLACED não é uma transmissão implícita porque ele é

enviado somente ao aplicativo cujo pacote foi substituído, não importando quantos outros aplicativos tenham detectores registrados para a transmissão em questão.

Os aplicativos podem continuar a se registrar para transmissões explícitas nos manifestos.

Os aplicativos podem usar o `Context.registerReceiver()` no tempo de execução para registrar um receptor para qualquer transmissão, seja ela implícita ou explícita.

Essas restrições não se aplicam a transmissões que exigem uma permissão de assinatura, já que elas são enviadas somente para aplicativos assinados com o mesmo certificado, não para todos os aplicativos no dispositivo.

Guia de migração

A partir da versão 8.0 do Android, houve mudanças que afetam a forma como os aplicativos em segundo plano podem fazer requisições quando há conexão com a internet. Isso foi feito para evitar que essas requisições consumam muita bateria e causem erros nos aplicativos.

Essas alterações se aplicam principalmente a aplicativos que são desenvolvidos para o Android 8.0 ou versões mais recentes. No entanto, os usuários têm a opção de ativar essas restrições para qualquer aplicativo nas configurações do dispositivo, mesmo que o aplicativo tenha sido desenvolvido para versões mais antigas do Android. Nesse caso, pode ser necessário atualizar o aplicativo para se adequar a essas novas limitações.

Se o seu aplicativo utiliza serviços que rodam em segundo plano quando o aplicativo está ocioso, é necessário fazer algumas adaptações. Algumas soluções possíveis são: usar o método `"startForegroundService()"` ao invés de `"startService()"` se o serviço precisar ser executado em primeiro plano, tornar o serviço em primeiro plano se for perceptível para o usuário (como um serviço de reprodução de áudio), utilizar um job programado para duplicar a funcionalidade do serviço, usar o FCM (Firebase Cloud Messaging) para ativar o aplicativo de forma seletiva quando ocorrerem eventos de rede, adiar o trabalho em segundo plano até que o aplicativo esteja em primeiro plano naturalmente.

Além disso, é importante analisar os broadcast receivers definidos no manifesto do aplicativo. Se o manifesto declarar um receptor para uma transmissão implícita, será necessário fazer algumas substituições. Por exemplo, é possível criar o receptor em tempo de execução usando o método `"Context.registerReceiver()"` ao invés de declará-lo no manifesto, ou utilizar um job programado para verificar a condição que acionaria a transmissão implícita.

Essas medidas foram implementadas para melhorar o desempenho dos dispositivos Android e evitar o consumo excessivo de recursos pelos aplicativos em segundo plano. Os desenvolvedores devem estar cientes dessas alterações e adaptar seus aplicativos conforme necessário para garantir um funcionamento adequado nas versões mais recentes do Android.

Para entender o conceito de `Manifest.permission`, é importante lembrar que o Android é um sistema operacional com recursos avançados que permitem que os aplicativos interajam com o dispositivo e seus recursos. O arquivo de manifesto é uma parte fundamental do desenvolvimento de aplicativos Android, pois contém informações sobre o aplicativo, suas permissões e configurações.

Dentro do manifesto, existem permissões específicas que um aplicativo pode solicitar para acessar recursos protegidos do dispositivo, como a câmera, a localização ou a conexão com a internet. Essas permissões são

definidas utilizando a classe `Manifest.permission`, que é um conjunto de constantes pré-definidas pelo sistema Android.

Quando um aplicativo solicita uma permissão, o usuário é notificado sobre quais recursos o aplicativo precisa acessar. O usuário pode então conceder ou negar essas permissões com base em sua preferência e nível de confiança no aplicativo.

As permissões do `Manifest.permission` são importantes para garantir a segurança e a privacidade dos usuários, pois evitam que aplicativos maliciosos acessem informações sensíveis sem o conhecimento ou consentimento do usuário.

Manifest.permission

Na documentação do Android, é destacado que, sempre que um aplicativo precisa acessar um recurso protegido por uma permissão, deve-se declarar essa necessidade no arquivo `Manifest`. O arquivo `Manifest` é um componente fundamental de um aplicativo Android, onde são definidas diversas informações essenciais para o funcionamento correto do aplicativo.

Para declarar a necessidade de uma permissão no arquivo `Manifest`, utiliza-se o elemento `<uses-permission>`. Esse elemento deve ser colocado dentro da tag `<manifest>` e antes da tag `<application>`. Ele possui um atributo chamado `android:name`, que indica o nome da permissão que está sendo solicitada.

Por exemplo, se um aplicativo precisa acessar a câmera do dispositivo, a permissão necessária é `android.permission.CAMERA`. Portanto, no arquivo `Manifest`, deve-se incluir a seguinte declaração:

```
<uses-permission android:name="android.permission.CAMERA" />
```

Ao incluir essa declaração, o sistema operacional Android reconhecerá que o aplicativo necessita da permissão para acessar a câmera e solicitará ao usuário que conceda essa permissão durante a instalação ou execução do aplicativo.

Essa declaração informa ao sistema operacional quais recursos o aplicativo precisa acessar e solicita a permissão correspondente ao usuário. Dessa forma, o aplicativo pode solicitar acesso a recursos como câmera, localização, internet, entre outros. Ao analisar a lista de permissões solicitadas pelo aplicativo no `Manifest`, é possível determinar quais recursos o aplicativo pode acessar.

(FGV – Banestes / 2018) Sempre que um aplicativo precisa de acesso a um recurso protegido por uma permissão no sistema operacional Android, ele precisa declarar essa necessidade incluindo um elemento `<uses-permission>` no arquivo `Manifest` do aplicativo.

A permissão que deve ser incluída no arquivo `Manifest` para que o aplicativo possa identificar se o aparelho está conectado a uma rede e qual o tipo de conexão é:

- a) `android.permission.ACCESS_FINE_LOCATION`;
- b) `android.permission.ACCESS_NETWORK_STATE`;
- c) `android.permission.INTERNET`;
- d) `android.permission.READ_SYNC_SETTINGS`;

e) android.permission.STATUS_BAR.

Comentários: A permissão que deve ser incluída no arquivo Manifest do aplicativo para identificar se o aparelho está conectado a uma rede e obter informações sobre o tipo de conexão é a opção b) android.permission.ACCESS_NETWORK_STATE.

Ao declarar essa permissão no Manifest, o aplicativo terá a capacidade de verificar o estado da conexão de rede do dispositivo, como se está conectado ou desconectado, e obter detalhes sobre o tipo de conexão, como Wi-Fi, dados móveis ou nenhuma conexão.

Essa permissão é essencial para aplicativos que dependem de conectividade de rede para funcionar corretamente, permitindo que eles ajustem seu comportamento com base na disponibilidade e no tipo de conexão disponível. (Gabarito: Letra B)

Pessoal, a classe Manifest.permission contém uma ampla variedade de permissões, totalizando aproximadamente 310 permissões. Obviamente, não é viável listá-las aqui. Porém, se você tiver interesse em conhecê-las detalhadamente, recomendo visitar o link: <https://developer.android.com/reference/android/Manifest.permission>.

No entanto, selecionei as 20 permissões **mais utilizadas e populares**, incluindo aquela que foi abordada no concurso do Banestes. É importante destacar que **entender o propósito de cada permissão é mais valioso do que memorizá-las indiscriminadamente**:

PERMISSÃO	DESCRIÇÃO
ANDROID.PERMISSION.INTERNET	Permite o acesso à internet.
ANDROID.PERMISSION.ACCESS_NETWORK_STATE	Permite verificar o estado da conexão de rede.
ANDROID.PERMISSION.READ_EXTERNAL_STORAGE	Permite ler arquivos armazenados externamente.
ANDROID.PERMISSION.WRITE_EXTERNAL_STORAGE	Permite escrever arquivos em armazenamento externo.
ANDROID.PERMISSION.CAMERA	Permite o acesso à câmera do dispositivo.
ANDROID.PERMISSION.ACCESS_FINE_LOCATION	Permite o acesso à localização precisa (usando GPS).
ANDROID.PERMISSION.ACCESS_COARSE_LOCATION	Permite o acesso à localização aproximada (usando redes móveis e Wi-Fi).
ANDROID.PERMISSION.RECORD_AUDIO	Permite o acesso ao microfone do dispositivo.
ANDROID.PERMISSION.READ_CONTACTS	Permite ler os contatos armazenados no dispositivo.
ANDROID.PERMISSION.WRITE_CONTACTS	Permite escrever ou modificar os contatos armazenados no dispositivo.
ANDROID.PERMISSION.READ_CALENDAR	Permite ler eventos e informações do calendário.
ANDROID.PERMISSION.WRITE_CALENDAR	Permite escrever ou modificar eventos e informações do calendário.

ANDROID.PERMISSION.READ_SMS	Permite ler mensagens SMS armazenadas no dispositivo.
ANDROID.PERMISSION.SEND_SMS	Permite enviar mensagens SMS.
ANDROID.PERMISSION.RECEIVE_SMS	Permite receber e processar mensagens SMS.
ANDROID.PERMISSION.READ_PHONE_STATE	Permite acessar informações do estado do telefone, como o número de telefone e o status da chamada.
ANDROID.PERMISSION.CALL_PHONE	Permite fazer chamadas telefônicas.
ANDROID.PERMISSION.READ_CALL_LOG	Permite ler o histórico de chamadas.
ANDROID.PERMISSION.WRITE_CALL_LOG	Permite escrever ou modificar o histórico de chamadas.
ANDROID.PERMISSION.PROCESS_OUTGOING_CALLS	Permite processar chamadas de saída.

Manifest.permission_group

A classe `Manifest.permission_group` é uma parte do sistema Android que agrupa permissões relacionadas a um determinado conjunto de recursos ou funcionalidades. Ela oferece uma maneira organizada de gerenciar e conceder permissões específicas para diferentes aspectos do aplicativo.

Essa classe possui várias permissões que abrangem diferentes áreas de funcionalidade. Aqui estão algumas das permissões mais comuns encontradas na classe `Manifest.permission_group`:

PERMISSÃO	DESCRIÇÃO
ACTIVITY_RECOGNITION	Utilizada para permissões relacionadas ao reconhecimento de atividades
CALENDAR	Utilizada para permissões relacionadas ao calendário do usuário
CALL_LOG	Utilizada para permissões relacionadas aos registros de chamadas
CAMERA	Utilizada para permissões relacionadas ao acesso à câmera do dispositivo
CONTACTS	Utilizada para permissões relacionadas aos contatos e perfis do usuário
LOCATION	Utilizada para permissões que permitem acessar a localização do dispositivo
MICROPHONE	Utilizada para permissões relacionadas ao acesso ao microfone do dispositivo
NEARBY_DEVICES	Requerida para ser capaz de descobrir e conectar-se a dispositivos Bluetooth próximos
NOTIFICATIONS	Utilizada para permissões relacionadas à exibição de notificações
PHONE	Utilizada para permissões relacionadas a recursos de telefonia
READ_MEDIA_AUDIO	Requerida para ser capaz de ler arquivos de áudio armazenados no armazenamento compartilhado
READ_MEDIA_VISUAL	Requerida para ser capaz de ler arquivos de imagem e vídeo armazenados no armazenamento

SENSORS	Utilizada para permissões relacionadas ao acesso a sensores corporais ou ambientais
SMS	Utilizada para permissões relacionadas a mensagens SMS do usuário
STORAGE	Utilizada para permissões relacionadas ao armazenamento externo compartilhado

Classes

As classes são elementos fundamentais no desenvolvimento de aplicativos Android. Elas são estruturas que encapsulam propriedades e comportamentos relacionados, permitindo a criação de objetos reutilizáveis e organizados.

No desenvolvimento Android, existem várias classes importantes fornecidas pela plataforma para facilitar a criação de **interfaces de usuário**, **manipulação de eventos**, **acesso a recursos do dispositivo** e muito mais. O site oficial do desenvolvedor Android (<https://developer.android.com/>) oferece uma documentação completa sobre todas essas classes e seus recursos.

Existem várias classes de layout disponíveis no Android que permitem organizar e posicionar os elementos da interface do usuário de maneira eficiente. Cada classe possui características distintas que se adequam a diferentes necessidades de design.

Uma das classes de layout mais básicas é o `FrameLayout`. Essa classe permite empilhar elementos uns sobre os outros, sendo útil quando se deseja exibir apenas um único elemento por vez, como uma imagem de fundo ou uma sobreposição. Já o `LinearLayout` organiza os elementos de forma linear, seja na vertical ou na horizontal. É uma opção popular para criar interfaces simples e alinhadas, onde os elementos são colocados um após o outro em uma única direção.

Para exibir dados em formato de tabela, a classe `TableLayout` é utilizada. Ela permite criar uma tabela com linhas e colunas, sendo útil quando se deseja exibir informações tabulares de maneira organizada. Outra classe comumente utilizada é o `RelativeLayout`, que permite posicionar os elementos com base em suas relações. Com o `RelativeLayout`, é possível definir as posições dos elementos de forma relativa uns aos outros, criando layouts flexíveis e adaptáveis a diferentes tamanhos de tela.

O `ConstraintLayout` é outra opção de layout bastante flexível. Ele oferece recursos avançados para posicionar e dimensionar os elementos, permitindo criar interfaces complexas e responsivas. O `CoordinatorLayout` é uma classe de layout que foi introduzida no Material Design. Ele é usado para controlar o comportamento de elementos interativos, como botões flutuantes, ao responder a eventos de rolagem e toque.

Quando se deseja exibir conteúdo rolável, a classe `ScrollView` é apropriada. Ela permite criar áreas de exibição que podem ser roladas verticalmente para mostrar todo o conteúdo. O `GridLayout` é ideal para organizar elementos em linhas e colunas, oferecendo suporte a tamanhos flexíveis e alinhamentos personalizados. É útil para criar layouts complexos e responsivos. O `CardView` é uma classe de layout usada para exibir informações ou conteúdo de forma elegante e consistente, com um design semelhante a cartões. Ele pode ser utilizado para exibir itens individuais ou agrupar elementos relacionados.

Por fim, o `ViewPager` é usado para criar interfaces com guias ou abas, permitindo que os usuários naveguem entre diferentes fragmentos ou páginas de conteúdo com gestos de deslizar.

CLASSE	DESCRIÇÃO
FRAMELAYOUT	Layout simples que contém um único elemento filho. É útil para exibir um único elemento em tela cheia ou sobreposto a outros elementos.

LINEARLAYOUT	Layout que organiza elementos filhos em uma única direção (horizontal ou vertical). É útil para criar interfaces de usuário simples e lineares.
TABLELAYOUT	Layout que organiza elementos filhos em uma grade bidimensional de linhas e colunas. É útil para exibir dados tabulares.
RELATIVELAYOUT	Layout que permite posicionar elementos filhos em relação uns aos outros ou em relação às bordas do layout pai. É útil para criar interfaces de usuário flexíveis e adaptáveis.
CONSTRAINTLAYOUT	Layout altamente flexível e poderoso que permite criar interfaces de usuário complexas e responsivas, usando restrições para definir as relações de posicionamento entre as visualizações filhas.
COORDINATORLAYOUT	Layout usado principalmente em conjunto com a biblioteca Design Support para criar efeitos de comportamento interativo entre visualizações, como rolagem, recuo e ancoragem. É útil para criar interfaces de usuário com efeitos de sobreposição e comportamentos especiais.
SCROLLVIEW	Layout que permite que o conteúdo dentro dele seja rolado verticalmente quando excede o tamanho da tela. É útil quando o conteúdo não cabe completamente na tela e precisa ser rolado para visualização completa.
GRIDLAYOUT	Layout que organiza elementos filhos em uma grade bidimensional, com linhas e colunas. É útil quando você precisa posicionar elementos em uma grade fixa ou quando a estrutura precisa se adaptar dinamicamente ao número de elementos.
CARDVIEW	Visualização que permite criar cartões estilizados com sombras, cantos arredondados e outros efeitos visuais. É útil para destacar visualmente certos elementos na interface do usuário.
VIEWPAGER	Visualização que permite a exibição de várias visualizações filhas em uma interface deslizável, permitindo a navegação entre elas com gestos de deslizar. É útil para criar interfaces com várias telas ou guias, onde o usuário pode percorrer horizontalmente para alternar entre as visualizações.

A classe **Intent** é uma **descrição abstrata de uma operação a ser realizada no Android**. Ela é usada para iniciar atividades, enviar broadcasts para componentes BroadcastReceiver e se comunicar com serviços em segundo plano. A Intent atua como uma estrutura de dados passiva que contém uma descrição abstrata de uma ação a ser executada.

A Intent permite a ligação dinâmica em tempo de execução entre o código de diferentes aplicativos. A classe **Intent é usada para iniciar componentes dentro do sistema operacional Android** ou entre diferentes aplicativos, permitindo a comunicação e interação entre eles. Sua principal utilização é no lançamento de atividades, onde ela funciona como uma espécie de **conexão entre as atividades**. Basicamente, é uma estrutura de dados que contém informações sobre a ação a ser executada.

Através da classe Intent, é possível realizar uma variedade de operações, como abrir atividades, enviar mensagens de broadcast, iniciar serviços e muito mais. Ela possui propriedades importantes, como a ação a ser executada e os dados em que a operação será realizada.

CLASSE	DESCRIÇÃO
ASYNQUERYHANDLER	Classe auxiliar que facilita o manuseio de consultas assíncronas no ContentResolver.
ASYNCTASKLOADER	Classe abstrata que fornece uma implementação base para carregadores de tarefas assíncronas.
CLIPDATA	Classe que representa um dado recortado na área de transferência no Android.
CONTEXT	Classe abstrata que representa um contexto no Android, fornecendo acesso a informações globais sobre o aplicativo.
ENTITY	Classe final que representa uma entidade.
LOADER	Classe que representa um carregador no Android, usado para carregar dados assincronamente.
LOCUSID	Classe final que representa um identificador de local no Android.
RESTRICTIONENTRY	Classe que representa uma entrada de restrição no Android, usada para expor restrições configuráveis para um usuário restrito em um dispositivo multiusuário.
SYNCCONTEXT	Classe que representa o contexto de sincronização.
QUICKVIEWCONSTANTS	Classe que define constantes relacionadas à visualização rápida.

Classes do Anadroid App

Activity

Activity é uma classe no Android que representa **uma única e focada ação que o usuário pode realizar**. A maioria das atividades interage com o usuário e a classe Activity cuida de criar uma janela na qual você pode colocar a interface do usuário usando o método `setContentView(View)`. As atividades geralmente são apresentadas ao usuário como janelas em tela cheia, mas também podem ser usadas de outras formas, como janelas flutuantes, modo de várias janelas ou incorporadas em outras janelas.

O método `onCreate(Bundle)` é usado para **inicializar a atividade**, ou seja, `onCreate` é utilizado para a criação de uma activity, onde você normalmente define o layout da interface do usuário com `setContentView(int)` e recupera os widgets dessa interface usando `findViewById(int)` para interagir com eles programaticamente.

O método `onPause()` é utilizado para lidar com a pausa da interação ativa do usuário com a atividade. Nesse momento, quaisquer alterações feitas pelo usuário devem ser salvas (geralmente em um `ContentProvider` que mantém os dados). A atividade ainda é visível na tela nesse estado. Para poder ser usado com `Context.startActivity()`, todas as classes de atividade devem ter uma declaração correspondente no arquivo `AndroidManifest.xml` do pacote.

(AOCF – PRODEB / 2018) Android é um sistema operacional (SO) amplamente utilizado em dispositivos móveis como por exemplo smartphones. A programação para este SO utiliza a linguagem Java e permite a criação e a manipulação de vários objetos como as activities. Levando em consideração este objeto da programação para Android, escolha a alternativa que representa o comando utilizado para a criação de uma activity?

- a) `OnStart`
- b) `OnResume`
- c) `OnPause`
- d) `OnCreate`
- e) `OnDestroy`

Comentários:

A resposta correta é a alternativa D: `OnCreate`. No contexto da programação para Android, uma "activity" é uma classe que representa uma tela ou uma janela com a qual o usuário interage. Quando uma activity é criada, o método `onCreate()` é chamado para inicializar a activity e configurar seus componentes. (Gabarito: Letra D)

Dialog

A classe **Dialog** é uma classe base no Android utilizada para criar caixas de diálogo na interface do usuário. Ela fornece uma estrutura para exibir informações, solicitar ações do usuário ou exibir opções em um formato de janela pop-up.

A classe **Dialog** é uma classe abstrata que pode ser estendida para criar diferentes tipos de caixas de diálogo, como alertas, diálogos de confirmação, seleção de itens e muitos outros. Esses tipos de diálogos podem ser personalizados de acordo com as necessidades do aplicativo.

A criação de uma instância de Dialog envolve a definição do conteúdo e do layout do diálogo, a configuração dos botões e a resposta aos eventos do usuário. A classe Dialog também oferece métodos para exibir e fechar o diálogo.

Além disso, as atividades no Android possuem métodos específicos, como `onCreateDialog(int)`, `onPrepareDialog(int, Dialog)`, `showDialog(int)` e `dismissDialog(int)`, que permitem gerenciar a criação, exibição e fechamento de diálogos de forma mais conveniente. Esses métodos são úteis para associar um diálogo a uma atividade específica e lidar com a interação entre o diálogo e a atividade.

Fragment

Um Fragment é uma parte da interface do usuário ou comportamento de um aplicativo que pode ser inserido em uma Activity. A interação com os fragments é feita por meio do `FragmentManager`, que pode ser obtido por meio de `Activity.getFragmentManager()` e `Fragment.getFragmentManager()`.

A classe Fragment pode ser utilizada de várias maneiras para alcançar uma ampla variedade de resultados. Em sua essência, ela representa uma operação ou interface específica que está sendo executada dentro de uma Activity maior. Um Fragment está intimamente ligado à Activity em que está inserido e não pode ser usado separadamente. Embora o Fragment defina seu próprio ciclo de vida, esse ciclo depende da sua activity: se a activity for interrompida, nenhum fragmento dentro dela poderá ser iniciado; quando a activity for destruída, todos os fragments também serão destruídos.

Todas as subclasses de Fragment devem incluir um construtor público sem argumentos. O framework frequentemente reinstitanciará uma classe de fragmento quando necessário, especialmente durante a restauração do estado, e precisa ser capaz de encontrar esse construtor para instanciá-lo. Se o construtor sem argumentos não estiver disponível, ocorrerá uma exceção em tempo de execução em alguns casos durante a restauração do estado.

GameManager e GameState

O GameManager permite que aplicativos do sistema modifiquem e consultem o modo de jogo de outros aplicativos. O GameState é o estado do jogo que é passado para o GameManager. Isso inclui um estado de alto nível para o jogo, indicando se o jogo pode ser interrompido sem interferir no conteúdo que não pode ser pausado. Como o conteúdo pode ser carregado em qualquer estado, o GameState inclui um indicador booleano independente para indicar o status de carregamento. Além disso, o aplicativo pode especificar metadados em formato livre (como um Bundle) e uma descrição em formato de texto no GameState.

Instrumentation

Instrumentation é uma classe base para implementar código de instrumentação de aplicativos. Quando a instrumentação está ativada, essa classe é instanciada antes de qualquer código do aplicativo, permitindo monitorar toda a interação do sistema com o aplicativo. Uma implementação de Instrumentation é descrita para o sistema por meio da tag `<instrumentation>` no arquivo `AndroidManifest.xml`.

IntentService

O `IntentService` é uma extensão da classe de componente `Service` que lida com solicitações assíncronas (expressas como `Intents`) sob demanda. Os clientes enviam solicitações por meio de chamadas `Context.startService(Intent)`; o serviço é iniciado quando necessário, lida com cada `Intent` em sequência usando uma `thread` de trabalho e para quando não houver mais trabalho a ser feito.

Esse padrão de "processador de fila de trabalho" é comumente usado para descarregar tarefas da `thread` principal de um aplicativo. A classe `IntentService` existe para simplificar esse padrão e cuidar da mecânica envolvida. Para usá-lo, estenda o `IntentService` e implemente `onHandleIntent(android.content.Intent)`. O `IntentService` receberá as `Intents`, lançará uma `thread` de trabalho e interromperá o serviço conforme apropriado.

Todas as solicitações são tratadas em uma única `thread` de trabalho - elas podem levar o tempo necessário (e não bloquearão o loop principal do aplicativo), mas apenas uma solicitação será processada por vez.

Notification

A classe `Notification` no Android é usada para criar e exibir notificações para o usuário. Ela representa uma notificação que pode ser exibida na barra de status, na tela de bloqueio ou como uma notificação flutuante, dependendo das configurações do dispositivo. A classe `Notification` permite configurar vários elementos da notificação, como o ícone, o título, o texto, as ações, o som e muito mais. Além disso, ela fornece métodos para gerenciar o comportamento da notificação, como cancelar ou atualizar uma notificação existente. O `NotificationManager` é responsável por lidar com as notificações e é usado para exibi-las e controlar seu comportamento.

Person

A classe `Person` no Android fornece uma referência imutável a uma entidade que aparece repetidamente em diferentes superfícies da plataforma. Por exemplo, ela pode representar o remetente de uma mensagem. A classe `Person` é usada para identificar e exibir informações consistentes sobre essa entidade em vários contextos, como notificações, conversas ou outros elementos da interface do usuário. Ela permite que os desenvolvedores especifiquem o nome, o ícone e outros detalhes relevantes da entidade para garantir uma experiência coerente e reconhecível em toda a plataforma.

Android Enterprise

O Android Enterprise é uma solução desenvolvida pelo Google para gerenciar o uso de dispositivos Android no ambiente corporativo. Ele oferece um conjunto de recursos e ferramentas que permitem às empresas controlar e proteger os dispositivos móveis utilizados pelos funcionários, garantindo a segurança das informações corporativas.

Com o Android Enterprise, as empresas podem aproveitar uma ampla variedade de casos de uso para estabelecer políticas de segurança e gerenciamento eficazes para seus dispositivos móveis. Essas funcionalidades permitem um controle mais granular sobre os dispositivos, garantindo a proteção dos dados corporativos e a privacidade dos funcionários.

Um dos casos de uso mais comuns é a configuração de perfis separados para uso pessoal e profissional nos dispositivos. Isso permite que os funcionários utilizem seus dispositivos pessoais para fins de trabalho, enquanto mantêm seus dados pessoais separados dos dados corporativos. Com a separação clara entre os perfis, as empresas podem aplicar políticas de segurança específicas ao perfil profissional, como restrições de acesso a determinados aplicativos ou recursos, exigência de autenticação adicional e criptografia de dados.

Além disso, o Android Enterprise facilita a implantação simplificada de aplicativos corporativos. As empresas podem distribuir e atualizar os softwares necessários para os funcionários de forma eficiente, garantindo que todos tenham acesso às versões mais recentes dos aplicativos corporativos.

O Android Enterprise também oferece recursos avançados de segurança, como criptografia de dados, controle de acesso, proteção contra malware e monitoramento remoto de dispositivos. A criptografia de dados ajuda a proteger as informações confidenciais armazenadas nos dispositivos, garantindo que somente as pessoas autorizadas tenham acesso a elas.

O controle de acesso permite às empresas controlar quais aplicativos e recursos os funcionários podem acessar, garantindo a conformidade com as políticas de segurança estabelecidas. A proteção contra malware ajuda a prevenir e detectar possíveis ameaças, protegendo os dispositivos e dados corporativos. O monitoramento remoto dos dispositivos permite que as empresas rastreiem e gerenciem os dispositivos de forma centralizada, identificando problemas de segurança, realizando atualizações e aplicando políticas em tempo real.

Casos de Uso Android Enterprise

Existem vários casos de uso comuns, como citado anteriormente, no contexto do Android Enterprise para o gerenciamento de dispositivos móveis no ambiente corporativo. Alguns desses casos de uso incluem:

- Gerenciamento de dispositivos pessoais: O Android Enterprise permite que os funcionários usem seus próprios dispositivos pessoais no ambiente de trabalho. Com a solução, é possível criar perfis separados para uso pessoal e profissional no mesmo dispositivo. Isso garante a segregação de aplicativos e dados pessoais e corporativos, protegendo a privacidade de cada usuário.

- Dispositivos corporativos com perfis separados: Para dispositivos de propriedade da empresa, o Android Enterprise suporta a criação de perfis pessoais e profissionais separados. Isso permite que os funcionários usem o dispositivo para fins pessoais e profissionais, mantendo os dados e aplicativos separados para garantir a segurança e privacidade das informações corporativas.
- Dispositivos corporativos com perfil profissional: O Android Enterprise também suporta a configuração de dispositivos de propriedade da empresa que têm apenas um perfil profissional. Nesse caso, o dispositivo é totalmente gerenciado pela empresa, com políticas de segurança, aplicativos e configurações controlados pela organização.
- Dispositivos dedicados para tarefas específicas: O Android Enterprise é adequado para dispositivos usados para tarefas especializadas, como leitores de código de barras ou terminais de ponto de venda. Esses dispositivos geralmente possuem apenas um perfil profissional e são totalmente gerenciados pela empresa, com acesso restrito a aplicativos específicos necessários para a tarefa em questão.
- Dispositivos para serviços dedicados: Nesse caso de uso, os dispositivos são destinados a serviços específicos, como dispositivos de campo para técnicos ou funcionários externos. Eles suportam apenas um perfil profissional, são totalmente gerenciados pela empresa e geralmente trabalham com um conjunto limitado de aplicativos, relevantes para a função ou serviço prestado.

(FUNDATEC – CRF-PR / 2021) Android Enterprise é uma solução para dispositivos Android que visa gerenciar o uso de dispositivos móveis no ambiente corporativo. Analise os casos de uso citados abaixo e assinale a opção que NÃO é suportada pelo Android Enterprise.

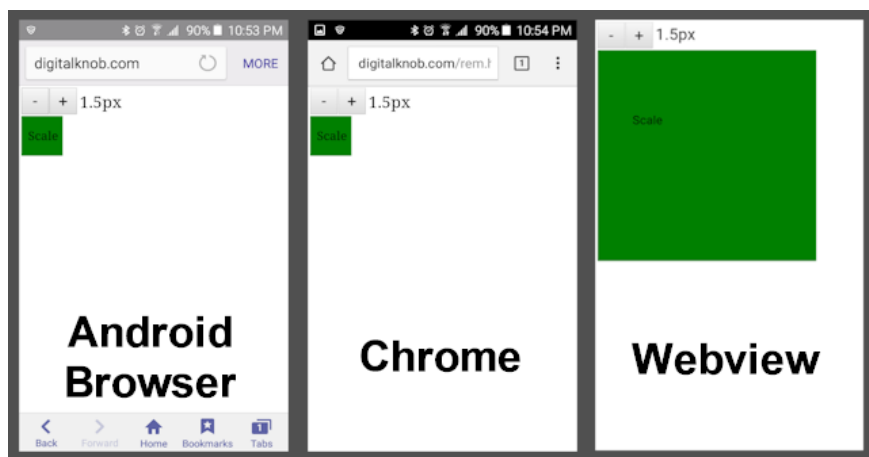
- a) Dispositivos de propriedade do empregado, que suportam um perfil pessoal e outro profissional armazenados separadamente.
- b) Dispositivos de propriedade da empresa, que suportam dois perfis (pessoal e profissional) separados e adicionalmente oferece um controle maior de políticas de segurança.
- c) Dispositivos de propriedade da empresa, que suportam somente um perfil profissional e totalmente gerenciado pela empresa.
- d) Dispositivos para serviços dedicados, que suportam somente perfil profissional, totalmente gerenciado pela empresa e com a característica de geralmente trabalhar somente alguns poucos aplicativos.
- e) Dispositivos de propriedade da empresa, que suportam dois perfis (pessoal e profissional), nos quais a empresa tem acesso a ambos, por questões de segurança.

Comentários:

A opção que NÃO é suportada pelo Android Enterprise é: E) Dispositivos de propriedade da empresa, que suportam dois perfis (pessoal e profissional), nos quais a empresa tem acesso a ambos, por questões de segurança.

No Android Enterprise, o controle é fornecido principalmente em dispositivos de propriedade da empresa, onde a empresa gerencia e controla o perfil profissional. O perfil pessoal é mantido separado e é controlado pelo proprietário do dispositivo. O Android Enterprise não suporta a opção em que a empresa tem acesso a ambos os perfis por questões de segurança. (Gabarito: Letra E)

WebKit



O WebKit, uma biblioteca de renderização utilizada em navegadores web, como o Google Chrome e o Safari, desempenha um papel fundamental no processamento e exibição de conteúdo HTML, CSS e JavaScript em páginas da web. Com algoritmos avançados, ele interpreta e renderiza os elementos da página, proporcionando uma experiência visual de qualidade aos usuários. Adicionalmente, o WebKit é uma tecnologia essencial para o

desenvolvimento de navegadores e aplicativos que exibem conteúdo web, suportando recursos como layout flexível, animações, manipulação de eventos e interatividade, o que enriquece a experiência de navegação.

Além de seu papel na renderização de páginas da web, o WebKit é altamente compatível com os padrões web, incluindo HTML5 e CSS3, permitindo que os desenvolvedores criem páginas e aplicativos modernos e responsivos. Sua compatibilidade estende-se a tecnologias como AJAX, que possibilita a atualização de conteúdo em tempo real, sem a necessidade de recarregar toda a página. Isso proporciona uma experiência mais dinâmica e interativa aos usuários, permitindo a exibição de informações atualizadas sem interrupções. Além disso, ele oferece recursos avançados de renderização, suporte a múltiplas janelas, tratamento de eventos e comunicação entre o aplicativo e a página web.

Essa biblioteca redenzadora de páginas é especialmente notável por sua compatibilidade com tecnologias como DOOM e AJAX. Com suporte a DOOM (Document Object Model), o WebKit permite que os desenvolvedores **acessem, manipulem e atualizem dinamicamente o conteúdo da página**, criando interações avançadas e interfaces interativas. Além disso, com o suporte a **AJAX** (Asynchronous JavaScript and XML), o WebKit possibilita **atualizações de conteúdo em tempo real**, sem a necessidade de recarregar completamente a página. Isso contribui para uma experiência de navegação mais dinâmica e responsiva.

A classe `WebViewAssetLoader.PathHandler` é uma interface no WebKit que produz respostas para um caminho registrado. Ela desempenha um papel fundamental ao lidar com requisições de recursos locais dentro de um `WebView`. Essa interface permite que desenvolvedores personalizem o comportamento de resposta para caminhos específicos, facilitando o carregamento de ativos e recursos estáticos do aplicativo.

A interface `WebViewCompat.VisualStateCallback` é fornecida como um callback para o método `postVisualStateCallback`. Ela desempenha um papel importante ao receber notificações sobre o estado visual do `WebView`. Ao registrar essa interface, os desenvolvedores podem ser informados sobre eventos

relevantes, como a conclusão do carregamento da página ou alterações visuais no WebView, permitindo que tomem ações apropriadas com base nessas informações.

Por fim, a interface `WebViewCompat.WebMessageListener` é responsável por receber mensagens enviadas no objeto JavaScript injetado por meio do método `addWebMessageListener`. Essa interface é essencial para a comunicação entre o JavaScript e o WebView, permitindo a troca de informações e interações entre as duas partes. Ao implementar essa interface, os desenvolvedores podem definir a lógica de tratamento para as mensagens recebidas, possibilitando uma integração eficiente e personalizada entre o JavaScript e o WebView.

INTERFACES	DESCRIÇÃO
WEBVIEWASSETLOADER.PATHHANDLER	Um manipulador que produz respostas para um caminho registrado.
WEBVIEWCOMPAT.VISUALSTATECALLBACK	Interface de callback fornecida para o método <code>postVisualStateCallback</code> , usada para receber notificações sobre o estado visual do WebView.
WEBVIEWCOMPAT.WEBMESSAGELISTENER	Esse ouvinte recebe mensagens enviadas no objeto JavaScript injetado por meio do método <code>addWebMessageListener</code> .

No desenvolvimento de aplicativos com WebView no WebKit, várias classes desempenham papéis fundamentais, fornecendo recursos avançados e funcionalidades importantes. Entre essas classes, destacam-se o `CookieManagerCompat`, o `ProxyConfig`, o `ProxyController`, o `TracingConfig` e o `WebSettingsCompat`. Cada uma delas desempenha funções específicas, permitindo o gerenciamento de cookies, configuração de proxy, rastreamento de atividades, ajuste de configurações do WebView e muito mais.

A classe `CookieManagerCompat` é uma versão de compatibilidade do `CookieManager`, permitindo o gerenciamento de cookies durante a navegação. Adicionando, removendo e obtendo informações sobre os cookies são recursos disponíveis com essa classe. Isso é especialmente valioso em dispositivos Android com diferentes versões do sistema operacional, pois garante a compatibilidade em dispositivos mais antigos e mais recentes.

Outra classe relevante é a `ProxyConfig`, que assume a responsabilidade de configurar as opções de proxy no WebKit. Com a `ProxyConfig`, é possível definir configurações personalizadas para o proxy, como host, porta e regras específicas. Isso permite o controle preciso sobre como as solicitações de rede feitas pelo WebView serão manipuladas através do proxy. É uma solução flexível que permite adaptar as configurações de proxy de acordo com as necessidades específicas do aplicativo.

Além disso, o `ProxyController` é responsável por gerenciar a configuração e limpeza das substituições de proxy específicas do processo no WebKit. Essa classe tem a função de controlar as configurações de proxy em todo o sistema Android para as solicitações de rede feitas pelo WebView em um determinado processo. Ao utilizar o `ProxyController`, é possível definir e limpar essas substituições de proxy de maneira eficiente, proporcionando um controle mais preciso e personalizado do comportamento de rede do WebView.

No contexto do WebKit, a classe TracingConfig armazena informações de configuração e configurações predefinidas para o TracingController. Ela oferece métodos para definir opções de rastreamento, como os eventos a serem rastreados e os recursos a serem coletados durante a depuração e análise de desempenho do WebView. Ao utilizar o TracingConfig, é possível personalizar o rastreamento das atividades do WebView de maneira mais precisa e eficiente, facilitando a identificação de problemas e otimização do desempenho.

Por fim, a classe WebSettingsCompat é uma versão de compatibilidade das configurações do WebView no WebKit. Com ela, é possível controlar várias configurações do WebView, como a habilitação ou desabilitação do JavaScript, definição do modo de renderização, gerenciamento do cache e configuração de opções de segurança. O WebSettingsCompat permite aos desenvolvedores personalizar e ajustar o comportamento do WebView de acordo com as necessidades específicas do aplicativo, oferecendo uma experiência de navegação mais personalizada e segura.

CLASSES	DESCRIÇÃO
COOKIEMANAGERCOMPAT	Versão de compatibilidade do android.webkit.CookieManager
DROPDATACONTENTPROVIDER	O WebView fornece suporte parcial para arrastar e soltar do Android, permitindo que imagens, texto e links sejam arrastados para fora de um WebView.
JAVASCRIPTREPLYPROXY	Esta classe representa o objeto JavaScript injetado por WebViewCompat#addWebMessageListener.
PROCESSGLOBALCONFIG	Configuração global do processo para WebView.
PROXYCONFIG	Configuração para setProxyOverride.
PROXYCONFIG.BUILDER	Construtor ProxyConfig.
PROXYCONFIG.PROXYRULE	Classe que contém um filtro de esquema e um proxy de URL.
PROXYCONTROLLER	Gerencia a configuração e a limpeza de uma substituição específica do processo para as configurações de proxy de todo o sistema Android que controlam as solicitações de rede feitas pelo android.webkit.WebView.
SAFEBROWSINGRESPONSECOMPAT	Versão de compatibilidade do SafeBrowsingResponse.
SERVICEWORKERCLIENTCOMPAT	Classe base para clientes capturarem retornos de chamada relacionados ao Service Worker;
SERVICEWORKERCONTROLLERCOMPAT	Gerencia Service Workers usados pelo WebView.
SERVICEWORKERWEBSETTINGSCOMPAT	Gerencia as configurações de estado para todos os Service Workers.
TRACINGCONFIG	Contém informações de configuração de rastreamento e configurações predefinidas para TracingController.
TRACINGCONFIG.BUILDER	Construtor usado para criar objetos TracingConfig.
TRACINGCONTROLLER	Gerencia o rastreamento de WebViews.
WEBMESSAGECOMPAT	A representação Java do evento HTML5 PostMessage.

WEBMESSAGEPORTCOMPAT	A representação Java das portas de mensagem HTML5.
WEBMESSAGEPORTCOMPAT.WEBMESSAGECALLBACKCOMPAT	O ouvinte para lidar com eventos MessagePort.
WEBRESOURCEERRORCOMPAT	Versão de compatibilidade do WebResourceError.
WEBRESOURCEREQUESTCOMPAT	Versão de compatibilidade de WebResourceRequest.
WEBSETTINGSCOMPAT	Versão de compatibilidade de android.webkit.WebSettings
WEBVIEWASSETLOADER	Classe auxiliar para carregar arquivos locais, incluindo ativos e recursos estáticos do aplicativo usando URLs http(s):// dentro de uma classe android.webkit.WebView.
WEBVIEWASSETLOADER.ASSETPATHHANDLER	Classe do manipulador para abrir um arquivo do diretório de ativos no APK do aplicativo.
WEBVIEWASSETLOADER.BUILDER	Uma classe de construtor para construir objetos WebViewAssetLoader.
WEBVIEWASSETLOADER.INTERNALSTOREPATHHANDLER	Classe do manipulador para abrir arquivos do armazenamento interno do aplicativo.
WEBVIEWASSETLOADER.RESOURCESPATHHANDLER	Classe do manipulador para abrir um arquivo do diretório de recursos no APK do aplicativo.
WEBVIEWCLIENTCOMPAT	Versão de compatibilidade do android.webkit.WebViewClient.
WEBVIEWCOMPAT	Versão de compatibilidade do android.webkit.WebView
WEBVIEWFEATURE	Classe de utilitário para verificar quais recursos da WebView Support Library são suportados no dispositivo.
WEBVIEWRENDERPROCESS	WebViewRenderProcess fornece um identificador opaco para um renderizador WebView.
WEBVIEWRENDERPROCESSCLIENT	Usado para receber retornos de chamada em eventos do renderizador WebView.

Extras!

Após aprender tudo (ou quase tudo) sobre desenvolvimento mobile, vamos entender como é o processo de publicar o app? Publicar um aplicativo Android envolve prepará-lo para o lançamento e disponibilizá-lo aos usuários. Durante a preparação, uma versão de lançamento do app é criada. Em seguida, o app pode ser lançado em uma loja de apps, como o Google Play, onde é possível divulgá-lo, vendê-lo e distribuí-lo para dispositivos Android. Também é possível lançar o app em um site, disponibilizando o arquivo APK para download e instalação pelos usuários.

A preparação para lançamento envolve configurações do app, como desativar a geração de registros e definir informações da versão. É necessário **criar e assinar uma versão de lançamento do app**, além de testá-la em

dispositivos de destino. É importante garantir que todos os recursos do app estejam atualizados e que os servidores ou serviços externos nos quais o app depende estejam prontos para produção.

O lançamento pode ser feito em uma loja de apps, como o **Google Play**, que oferece uma plataforma robusta para a divulgação e venda de apps Android em escala global. O Google Play permite configurar opções e recursos de upload, além de disponibilizar ferramentas para análise de vendas e tendências de mercado. Alternativamente, o app pode ser lançado em um site próprio, onde os usuários podem fazer o download e instalação. É importante considerar as configurações de segurança do dispositivo, como permitir a instalação de apps de fontes desconhecidas, caso o app seja disponibilizado fora de uma loja oficial.

O **controle de versões** do aplicativo fornece informações aos usuários sobre a versão instalada e as atualizações disponíveis. Além disso, outros aplicativos e serviços precisam consultar a versão do seu aplicativo para garantir a compatibilidade e identificar dependências. O sistema Android usa o **versionCode** para **evitar downgrades**, enquanto o **versionName** é usado para **exibir a versão aos usuários**. Definir corretamente essas informações no arquivo de configuração **build.gradle ou build.gradle.kts do seu projeto é essencial para garantir um controle adequado das versões**.

Ao definir as informações da versão do aplicativo, é importante especificar o **versionCode** e o **versionName**. O **versionCode** é um número inteiro positivo usado internamente para determinar a versão mais recente do aplicativo, enquanto o **versionName** é o valor exibido aos usuários. É possível definir esses valores no arquivo **build.gradle ou build.gradle.kts**, incluindo-os no bloco **defaultConfig {}**. Essas configurações serão mescladas no arquivo de manifesto do aplicativo durante o processo de compilação.

Cada versão sucessiva do aplicativo deve ter um valor de **versionCode** maior que o da versão anterior. Esses valores de versão são fundamentais para garantir a correta atualização e compatibilidade do aplicativo, tanto para os usuários quanto para outros aplicativos e serviços que dependem dele. Portanto, é importante adotar uma estratégia de controle de versões semântico e definir corretamente as informações de versão no seu projeto Android.

A **assinatura do aplicativo** é um requisito essencial no Android, **garantindo a autenticidade e integridade dos APKs antes de serem instalados ou atualizados em um dispositivo**. Ao lançar um aplicativo na Google Play Store, **é necessário assinar o pacote do aplicativo com uma chave de upload e configurar a Assinatura de Apps do Google Play**. As etapas envolvem gerar uma chave de upload, assinar o aplicativo com essa chave, configurar a assinatura no Google Play e fazer o upload do aplicativo. Essas medidas garantem a segurança do aplicativo durante a distribuição.

Se o aplicativo já estiver publicado na Google Play Store usando uma chave de assinatura ou se você desejar usar uma nova chave em vez da gerada pelo Google, é possível assinar o aplicativo com a chave de assinatura e fazer o upload dessa chave na Assinatura de Apps do Google Play. Recomenda-se gerar e registrar um certificado de upload para futuras atualizações. Além disso, a página também aborda a opção de gerenciar suas próprias chaves ao enviar o aplicativo para outras lojas de aplicativos. Se você preferir assinar o aplicativo na linha de comando ou não estiver usando o Android Studio, pode aprender a utilizar a ferramenta **apksigner**.

O code signing, ou assinatura de código refere-se ao processo de assinar digitalmente um aplicativo Android com um certificado para verificar sua autenticidade e integridade. **O Android exige que todos os APKs (Android Package) sejam assinados antes de serem instalados em dispositivos ou atualizados**. A assinatura

do aplicativo no Android envolve o uso de um keystore, que é um arquivo contendo uma chave privada e o certificado correspondente. A chave privada é usada para gerar uma assinatura digital única para o aplicativo, enquanto o certificado contém informações sobre o desenvolvedor ou fabricante do aplicativo.

O code signing, ou assinatura de código, é um processo utilizado para assinar digitalmente software ou código a fim de verificar sua autenticidade e integridade. Isso envolve o uso de um certificado digital emitido por uma autoridade de certificação confiável para assinar o código. A assinatura digital funciona como um selo à prova de violação que identifica o autor ou a organização responsável pelo código e garante que ele não tenha sido modificado ou adulterado desde a assinatura.

O objetivo do code signing é fornecer garantia aos usuários de que o software que estão instalando ou executando é proveniente de uma fonte confiável e não foi alterado de forma maliciosa. Quando um usuário encontra um código assinado, seu sistema operacional ou software de segurança pode verificar a assinatura usando a chave pública do certificado. Se a assinatura for válida e o certificado puder ser confiável, o código é considerado autêntico e não foi adulterado.

Ao seguir esses procedimentos de assinatura, os desenvolvedores garantem a autenticidade do seu aplicativo, protegendo-o contra alterações indesejadas e fornecendo aos usuários uma experiência segura e confiável durante a instalação e atualização do app. A assinatura do aplicativo é uma medida de segurança essencial, que permite aos usuários verificar a procedência do aplicativo e ter a garantia de que não foi adulterado por terceiros mal-intencionados.

(CEBRASPE – ABIN / 2018) Para garantir que o software gerado no servidor chegue ao usuário final, utiliza-se um certificado code signing, que altera o software e também insere uma assinatura do desenvolvedor ou fabricante.

Comentários:

Na verdade, o certificado Code Signing não altera o software em si, mas adiciona uma assinatura digital ao software já existente. A assinatura digital é um mecanismo de segurança que atesta a autenticidade e integridade do software, garantindo que o código não tenha sido modificado após a assinatura ter sido aplicada. O certificado Code Signing serve para verificar a identidade do desenvolvedor ou fabricante do software, permitindo que o usuário final saiba quem o criou e se o software não foi alterado por terceiros mal-intencionados. A assinatura do desenvolvedor ou fabricante, inserida pelo certificado Code Signing, fornece confiança e segurança ao usuário final, mas não envolve alterações no software em si. (Gabarito: Errado)

REFERÊNCIAS

Kotlin for Android Developers

Antonio Leiva. Kotlin for Android Developers: Learn Kotlin the Easy Way While Developing an Android App. 2ª Edição, 2018.

Kotlin Language Documentation on Wikipedia

Wikipedia. Kotlin (programming language). Disponível em: <https://pt.wikipedia.org/wiki/Kotlin>

Kotlin by Example

JetBrains. Kotlin by Example. Disponível em: <https://kotlinlang.org/docs/examples.html>

Kotlin in Action

Dmitry Jemerov, Svetlana Isakova. Kotlin in Action. 2017.

Android Manifest Documentation

Google. *Manifesto do Android: Introdução ao Manifesto*. Disponível em: <https://developer.android.com/guide/topics/manifest/manifest-intro?hl=pt-br>

Temok Blog

Temok. *Blog de Hospedagem e Marketing Online*. Disponível em: <https://www.temok.com/blog/>

Android Enterprise Documentation

Google. *Documentação Android Enterprise*. Disponível em: <https://www.androidenterprise.dev/s/>

Android Studio Publishing

Google. *Publicação no Android Studio*. Disponível em: <https://developer.android.com/studio/publish?hl=pt-br>

Apple Developer Documentation

Apple. *Documentação do Apple Developer*. Disponível em: <https://developer.apple.com/documentation/>

Flutter Official Website

Flutter. *Flutter: Beautiful native apps in record time*. Disponível em: <https://flutter.dev>

Flutter Wikipedia

Wikipedia. *Flutter*. Disponível em: <https://pt.wikipedia.org/wiki/Flutter>

Flutter API Documentation

Flutter. *Documentação da API Flutter*. Disponível em: <https://api.flutter.dev>

Swift Documentation

Apple. *The Swift Programming Language Documentation*. Disponível em: <https://docs.swift.org/swift-book/documentation/the-swift-programming-language/>

Flutter Docs

Flutter. *Flutter Documentation*. Disponível em: <https://docs.flutter.dev>

QUESTÕES COMENTADAS

Android

1. (AOCP – PRODEB/2018) Considerando as linguagens de programação mobile, qual das dispostas a seguir foi criada pela Apple e pode ser utilizada para o desenvolvimento das suas aplicações?

- a) Kotlin
- b) Swift
- c) C#
- d) Python
- e) IOS

Comentários:

Com base nas informações fornecidas pela Apple, a linguagem de programação Swift foi desenvolvida especificamente pela empresa para a criação de aplicativos em seus dispositivos, como iOS, Mac, Apple TV e Apple Watch. A Apple destaca que o objetivo ao criar o Swift foi proporcionar maior liberdade aos desenvolvedores, oferecendo uma linguagem consistente, intuitiva e fácil de usar.

Uma característica interessante do Swift é sua disponibilidade como código aberto, o que permite que qualquer pessoa com uma boa ideia possa utilizar a linguagem e fazer coisas surpreendentes. Isso também estimula a participação da comunidade de desenvolvedores no aprimoramento contínuo da linguagem.

Gabarito: Letra B

2. (FGV – IBGE / 2016) Um desenvolvedor Android deseja inserir a funcionalidade de backup em uma aplicação móvel para, de tempos em tempos, armazenar dados automaticamente. A classe da API de Backup (versão 6.0 ou superior) a ser utilizada é a:

- a) BkpAgent;
- b) BkpHelper;
- c) BackupManager;
- d) BackupOutputData;
- e) BackupDataStream.

Comentários:

A classe BackupManager da API de Backup do Android é fundamental para implementar o backup automático em aplicativos móveis. Com ela, os desenvolvedores podem agendar e coordenar

operações de backup e restauração de dados, simplificando o armazenamento e proteção das informações. É possível definir intervalos de tempo e personalizar quais dados serão incluídos no backup. Essa funcionalidade é essencial para garantir a segurança dos dados dos usuários, especialmente em aplicativos com informações sensíveis. A API de Backup do Android oferece um mecanismo poderoso para preservar a integridade dos dados, permitindo sua restauração em situações como reinstalação ou migração de dispositivo.

Gabarito: Letra C

3. (FGV – Banestes / 2018) Sempre que um aplicativo precisa de acesso a um recurso protegido por uma permissão no sistema operacional Android, ele precisa declarar essa necessidade incluindo um elemento `<uses-permission>` no arquivo Manifest do aplicativo.

A permissão que deve ser incluída no arquivo Manifest para que o aplicativo possa identificar se o aparelho está conectado a uma rede e qual o tipo de conexão é:

- a) `android.permission.ACCESS_FINE_LOCATION`;
- b) `android.permission.ACCESS_NETWORK_STATE`;
- c) `android.permission.INTERNET`;
- d) `android.permission.READ_SYNC_SETTINGS`;
- e) `android.permission.STATUS_BAR`.

Comentários:

A permissão que deve ser incluída no arquivo Manifest do aplicativo para identificar se o aparelho está conectado a uma rede e obter informações sobre o tipo de conexão é a opção b) `android.permission.ACCESS_NETWORK_STATE`.

Ao declarar essa permissão no Manifest, o aplicativo terá a capacidade de verificar o estado da conexão de rede do dispositivo, como se está conectado ou desconectado, e obter detalhes sobre o tipo de conexão, como Wi-Fi, dados móveis ou nenhuma conexão.

Essa permissão é essencial para aplicativos que dependem de conectividade de rede para funcionar corretamente, permitindo que eles ajustem seu comportamento com base na disponibilidade e no tipo de conexão disponível.

Gabarito: Letra B

4. (FUNDATEC – CRF-PR / 2021) Android Enterprise é uma solução para dispositivos Android que visa gerenciar o uso de dispositivos móveis no ambiente corporativo. Analise os casos de uso citados abaixo e assinale a opção que NÃO é suportada pelo Android Enterprise.

- a) Dispositivos de propriedade do empregado, que suportam um perfil pessoal e outro profissional armazenados separadamente.
- b) Dispositivos de propriedade da empresa, que suportam dois perfis (pessoal e profissional) separados e adicionalmente oferece um controle maior de políticas de segurança.
- c) Dispositivos de propriedade da empresa, que suportam somente um perfil profissional e totalmente gerenciado pela empresa.
- d) Dispositivos para serviços dedicados, que suportam somente perfil profissional, totalmente gerenciado pela empresa e com a característica de geralmente trabalhar somente alguns poucos aplicativos.
- e) Dispositivos de propriedade da empresa, que suportam dois perfis (pessoal e profissional), nos quais a empresa tem acesso a ambos, por questões de segurança.

Comentários:

A opção que NÃO é suportada pelo Android Enterprise é:

E) Dispositivos de propriedade da empresa, que suportam dois perfis (pessoal e profissional), nos quais a empresa tem acesso a ambos, por questões de segurança.

No Android Enterprise, o controle é fornecido principalmente em dispositivos de propriedade da empresa, onde a empresa gerencia e controla o perfil profissional. O perfil pessoal é mantido separado e é controlado pelo proprietário do dispositivo. O Android Enterprise não suporta a opção em que a empresa tem acesso a ambos os perfis por questões de segurança.

Gabarito: Letra E

5. (AOCP – Pref Novo Hamburgo / 2020) São considerados sistemas operacionais móveis:

- a) Windows Phone e Windows XP.
- b) Red Hat Enterprise Linux e Android.
- c) iOS e OSX.
- d) Android e Minix.
- e) Symbian OS e Windows Phone.

Comentários:

a) Windows Phone e Windows XP: Essa opção não é correta. Embora o Windows Phone seja um sistema operacional móvel, o Windows XP é um sistema operacional para computadores pessoais e não é considerado um sistema operacional móvel.

b) Red Hat Enterprise Linux e Android: Essa opção não é correta. O Red Hat Enterprise Linux é um sistema operacional voltado para servidores e não é específico para dispositivos móveis. No entanto, o Android é um sistema operacional móvel desenvolvido pelo Google e amplamente utilizado em smartphones e tablets.

c) iOS e OSX: Essa opção não é correta. O iOS é o sistema operacional móvel desenvolvido pela Apple e usado em dispositivos como iPhones e iPads. Já o OSX (agora chamado de macOS) é o sistema operacional usado em computadores Mac e não é voltado para dispositivos móveis.

d) Android e Minix: Essa opção é correta. O Android é um sistema operacional móvel desenvolvido pelo Google e usado em uma ampla variedade de dispositivos, incluindo smartphones, tablets e até mesmo smart TVs. Minix, por outro lado, é um sistema operacional de código aberto usado principalmente em dispositivos incorporados e também pode ser encontrado em algumas caixas de TV Android.

e) Symbian OS e Windows Phone: Essa opção não é correta. O Symbian OS foi um sistema operacional móvel desenvolvido pela Nokia, mas não é amplamente utilizado atualmente. O Windows Phone, por sua vez, foi um sistema operacional móvel desenvolvido pela Microsoft, mas também foi descontinuado e substituído pelo Windows 10 Mobile.

Portanto, a alternativa correta é a d) Android e Minix, pois ambos são sistemas operacionais móveis.

Gabarito: Letra D

6. (IBADE – Pref Jaru/ 2019) O sistema operacional indicado para dispositivos móveis é denominado:

- a) Windows Server.
- b) Linux.
- c) IOS.
- d) Z/OS.
- e) AIX.

Comentários:

a) Windows Server: Essa opção não é correta. O Windows Server é um sistema operacional da Microsoft voltado para servidores e não é especificamente desenvolvido para dispositivos móveis.

b) Linux: Essa opção é parcialmente correta. O Linux é um sistema operacional de código aberto amplamente utilizado em uma variedade de dispositivos, incluindo servidores, computadores pessoais e também dispositivos móveis. Existem distribuições do Linux, como o Android, que são específicas para dispositivos móveis.

c) iOS: Essa opção é correta. O iOS é o sistema operacional desenvolvido pela Apple exclusivamente para seus dispositivos móveis, como iPhones e iPads. Ele é conhecido por sua interface intuitiva e integração perfeita com o ecossistema da Apple.

d) Z/OS: Essa opção não é correta. O Z/OS é um sistema operacional desenvolvido pela IBM para mainframes, que não é destinado a dispositivos móveis.

e) AIX: Essa opção não é correta. O AIX é um sistema operacional desenvolvido pela IBM para seus servidores da linha IBM Power Systems e não é projetado especificamente para dispositivos móveis.

Portanto, a alternativa correta é a c) iOS, pois é o sistema operacional desenvolvido pela Apple para dispositivos móveis. No entanto, é importante ressaltar que existem outras opções de sistemas operacionais móveis, como o Android (baseado no Linux), que também são amplamente utilizados em dispositivos móveis.

Gabarito: Letra C

7. (UNESC – Pref Criciúma/ 2019) O sistema android possibilita que seus usuários possam agrupar seus aplicativos em pastas de acordo com sua necessidade. Qual a ação necessária para a criação de uma pasta no android:

- a) Pressionar com o dedo na tela e manter pressionado até o aparecimento do menu pasta, posteriormente clicar em nova pasta.
- b) Arrastar um aplicativo e sobrepô-lo a outro.
- c) Pressionar o botão menu e acessar as configurações.
- d) Pressionar os botões menu e voltar ao mesmo tempo.

Comentários:

A ação necessária para a criação de uma pasta no Android é a opção b) Arrastar um aplicativo e sobrepô-lo a outro. Explicação: Para criar uma pasta no Android, basta seguir os seguintes passos:

1. Na tela inicial do seu dispositivo Android, localize o aplicativo que deseja colocar em uma pasta.
2. Pressione e segure o ícone do aplicativo com o dedo.
3. Arraste o ícone do aplicativo em direção a outro aplicativo que você deseja agrupar na mesma pasta.
4. Ao sobrepor o ícone do primeiro aplicativo sobre o segundo, o Android criará automaticamente uma pasta e os dois aplicativos serão colocados dentro dela.
5. A pasta será criada com um nome padrão, geralmente baseado na categoria dos aplicativos. Você pode renomear a pasta para algo mais descritivo, se desejar.
6. Para adicionar mais aplicativos à pasta, basta arrastá-los para dentro da pasta da mesma maneira que você fez anteriormente.
7. Você também pode arrastar aplicativos para fora da pasta se desejar movê-los para a tela inicial novamente.

Essa é a maneira comum de criar pastas no Android, permitindo que os usuários organizem seus aplicativos de acordo com suas necessidades e preferências.

Gabarito: Letra B

8. (UNESC – Pref Criciúma/ 2019) As primeiras versões do sistema operacional android apresentavam a barra de navegação de forma física. Com o surgimento de novas versões do sistema a barra de navegação ficou virtual. Os botões que compõem a barra de navegação do android são?

- a) Desligar, aumentar e diminuir volume.
- b) Menu, voltar e aplicativos abertos.
- c) Voltar, iniciar e aplicativos abertos.
- d) Menu, volta, Iniciar.

Comentários:

Os botões que compõem a barra de navegação do Android são: b) Menu, voltar e aplicativos abertos. Nas versões mais recentes do sistema operacional Android, a barra de navegação é composta pelos seguintes botões: Botão Voltar: O botão voltar permite que você retorne à tela anterior, seja dentro de um aplicativo ou na navegação geral do sistema. Botão Iniciar: O botão Iniciar, também conhecido como Home, leva você de volta à tela inicial do dispositivo. Pressioná-lo enquanto estiver em um aplicativo ou em qualquer outra tela leva você à tela inicial. Botão de Aplicativos Abertos: Esse botão, representado por um ícone de quadrado ou listra, permite visualizar e alternar rapidamente entre os aplicativos recentemente usados. Ao pressioná-lo, uma lista dos aplicativos abertos ou em segundo plano é exibida, permitindo que você escolha o aplicativo desejado para acessá-lo novamente. Esses botões facilitam a interação com os aplicativos, a navegação entre telas e o acesso rápido aos aplicativos recentes. É importante notar que as versões mais recentes do Android podem ter variações na interface e na aparência dos botões de navegação, mas a funcionalidade básica geralmente permanece a mesma.

Gabarito: Letra B

9. (FAU UNICENTRO – IF PR/ 2019) No desenvolvimento para dispositivos móveis utilizando Android podemos utilizar alguns tipos de layout que facilitam o desenvolvimento das telas de aplicativos. Relacione os tipos de layouts e suas descrições e assinale a alternativa com a sequência correta:

- 1 - AbsoluteLayout.
- 2 - FrameLayout.
- 3 - LinearLayout.

4 - TableLayout.

5 - RelativeLayout.

() Permite posicionar um componente relativo a outro, por exemplo, abaixo ou acima de um componente existente.

() Utilizado quando necessário que um componente preencha a tela inteira do dispositivo automaticamente.

() Permite posicionar componentes, fornecendo as coordenadas x e y.

() Utilizado para organizar os componentes na vertical ou horizontal.

() Utilizado para organizar os componentes em uma tabela, com linhas e colunas.

A sequência correta de cima para baixo é:

a) 2, 3, 4, 1, 5.

b) 3, 5, 1, 2, 4.

c) 1, 4, 5, 2, 3.

d) 5, 2, 1, 3, 4.

e) 5, 1, 2, 3, 4

Comentários:

(5) RelativeLayout: Permite posicionar um componente relativo a outro, por exemplo, abaixo ou acima de um componente existente.

(2) FrameLayout: Utilizado quando necessário que um componente preencha a tela inteira do dispositivo automaticamente.

(1) AbsoluteLayout: Permite posicionar componentes, fornecendo as coordenadas x e y.

(3) LinearLayout: Utilizado para organizar os componentes na vertical ou horizontal.

(4) TableLayout: Utilizado para organizar os componentes em uma tabela, com linhas e colunas.

Portanto, a sequência correta é:

D. 5, 2, 1, 3, 4.

Gabarito: Letra D

- 10. (CEBRASPE – ABIN / 2018)** O Android disponibiliza um banco de dados público local, orientado a objetos, para o armazenamento de dados estruturados, o que possibilita o gerenciamento das aplicações e dos dados de forma rápida e segura.

Comentários:

Na verdade, o Android oferece um mecanismo de banco de dados embutido chamado SQLite, que é um banco de dados relacional, não orientado a objetos. O SQLite é uma biblioteca de código aberto que fornece um conjunto de recursos para o armazenamento, gerenciamento e consulta de dados estruturados em dispositivos Android.

O SQLite é amplamente utilizado no desenvolvimento de aplicativos Android para armazenar dados estruturados de forma eficiente. Ele oferece suporte a consultas SQL completas, transações ACID (Atomicidade, Consistência, Isolamento e Durabilidade), integridade de dados e recuperação em caso de falhas. O SQLite é leve e otimizado para uso em dispositivos móveis, tornando-o uma escolha popular para o armazenamento local de dados em aplicativos Android.

Embora o SQLite seja um banco de dados relacional, os desenvolvedores podem usar estratégias e padrões de programação orientados a objetos para interagir com ele. Isso inclui o uso de bibliotecas de mapeamento objeto-relacional (ORM), como o Room Persistence Library, que simplifica a interação com o SQLite e permite aos desenvolvedores trabalhar com objetos e classes em vez de escrever consultas SQL diretas.

Gabarito: Errado

- 11. (CEBRASPE – ABIN / 2018)** Mesmo controlando o login e a senha do usuário via contas Google, um aplicativo pode capturar e enviar arquivos armazenados no cartão SD do celular que utiliza o sistema Android.

Comentários:

Embora o controle do login e senha do usuário via contas Google possa fortalecer a segurança, é importante considerar que um aplicativo com permissão de leitura/escrita no armazenamento externo do dispositivo Android pode potencialmente acessar e modificar arquivos armazenados no cartão SD. Esse risco é especialmente relevante quando um aplicativo realiza o download de atualizações ou processamento, momento em que arquivos podem ficar temporariamente em uma área compartilhada, permitindo que um app malicioso insira código malicioso. Portanto, além do controle de autenticação, é fundamental avaliar as permissões concedidas aos aplicativos e estar atento à segurança ao lidar com arquivos no cartão SD.

Gabarito: Correto

- 12. (CEBRASPE – ABIN / 2018)** Quando a Internet está disponível, os aplicativos executados em segundo plano podem efetuar requisições, que utilizam muita carga da bateria e podem

ocasionar erros nos aplicativos, por isso, na versão 8.0 do sistema, os manifestos não podem ocorrer para transmissões implícitas

Comentários:

Na versão 8.0 do sistema operacional Android, foi introduzida uma restrição nos manifestos dos aplicativos, impedindo a ocorrência de transmissões implícitas quando a internet está disponível. Essa restrição foi implementada devido ao fato de que os aplicativos em segundo plano podem realizar requisições que consomem muita carga da bateria e podem levar a erros nos aplicativos. Essa medida visa melhorar o desempenho do dispositivo, evitando requisições desnecessárias em segundo plano que podem impactar negativamente a vida útil da bateria e a estabilidade dos aplicativos. Os desenvolvedores são incentivados a adotar métodos mais eficientes para lidar com eventos e requisições de rede, levando em consideração essa restrição nos manifestos do Android 8.0.

Gabarito: Correto

13. (CEBRASPE – ABIN / 2018) Para garantir que o software gerado no servidor chegue ao usuário final, utiliza-se um certificado code signing, que altera o software e também insere uma assinatura do desenvolvedor ou fabricante.

Comentários:

Na verdade, o certificado Code Signing não altera o software em si, mas adiciona uma assinatura digital ao software já existente. A assinatura digital é um mecanismo de segurança que atesta a autenticidade e integridade do software, garantindo que o código não tenha sido modificado após a assinatura ter sido aplicada.

O certificado Code Signing serve para verificar a identidade do desenvolvedor ou fabricante do software, permitindo que o usuário final saiba quem o criou e se o software não foi alterado por terceiros mal-intencionados. A assinatura do desenvolvedor ou fabricante, inserida pelo certificado Code Signing, fornece confiança e segurança ao usuário final, mas não envolve alterações no software em si.

Gabarito: Errado

14. (CONSULPLAN – Câmara de Belo Horizonte - MG / 2018) "Classe do Android que pode ser utilizada para enviar uma mensagem para o sistema operacional; solicitar ao sistema operacional que ligue para determinado número de celular etc." Assinale-a.

- a) Intent.
- b) Service.
- c) Content.
- d) Receiver.

Comentários:

A classe do Android que pode ser utilizada para enviar uma mensagem para o sistema operacional, solicitar ao sistema operacional que ligue para determinado número de celular, entre outras ações, é a opção a) Intent. A classe Intent é usada para iniciar componentes dentro do sistema operacional Android ou entre diferentes aplicativos, permitindo a comunicação e interação entre eles.

Gabarito: Letra A

15. (AOCP – PRODEB / 2018) Android é um sistema operacional (SO) amplamente utilizado em dispositivos móveis como por exemplo smartphones. A programação para este SO utiliza a linguagem Java e permite a criação e a manipulação de vários objetos como as activitys. Levando em consideração este objeto da programação para Android, escolha a alternativa que representa o comando utilizado para a criação de uma activity?

- a) OnStart
- b) OnResume
- c) OnPause
- d) OnCreate
- e) OnDestroy

Comentários:

A resposta correta é a alternativa D: OnCreate. No contexto da programação para Android, uma "activity" é uma classe que representa uma tela ou uma janela com a qual o usuário interage. Quando uma activity é criada, o método onCreate() é chamado para inicializar a activity e configurar seus componentes.

Gabarito: Letra D

16. (IBADE – Câmara de Vilhena - RO / 2018) Os aplicativos desenvolvidos para dispositivos móveis como celulares e tablets são denominados:

- a) Android.
- b) IOS.
- c) Kindle.
- d) APP.
- e) REXX.

Comentários:

Questão teste de pressão arterial: questão para verificar se o candidato está vivo. Os aplicativos desenvolvidos para dispositivos móveis, como celulares e tablets, são geralmente referidos como "apps". "APP" é uma abreviação comum para "aplicativo" em inglês se refere aos programas

desenvolvidos para dispositivos móveis, oferecendo funcionalidades específicas aos usuários. Portanto, a alternativa D é a resposta correta. Vamos usar a questão para fazer uma revisão: a) Android: Android é um sistema operacional móvel desenvolvido pelo Google, amplamente utilizado em smartphones e tablets. b) iOS: iOS é o sistema operacional móvel desenvolvido pela Apple para seus dispositivos, como iPhones e iPads. c) Kindle: O Kindle é uma linha de dispositivos de leitura de livros eletrônicos desenvolvida pela Amazon, permitindo que os usuários acessem e leiam uma ampla variedade de livros digitais. e) REXX: REXX é uma linguagem de programação de alto nível, criada para automatizar tarefas e facilitar o processamento de dados em sistemas mainframe.

Gabarito: Letra D

17. (AOCF – CODEM - PA / 2017) O Sistema Operacional Android é

- a) baseado em núcleo Linux e é utilizado exclusivamente em celulares.
- b) um sistema operacional proprietário e é utilizado principalmente em dispositivos móveis.
- c) um sistema operacional proprietário e é utilizado exclusivamente em celulares.
- d) um sistema operacional livre e é utilizado exclusivamente em celulares.
- e) baseado em núcleo Linux e é utilizado principalmente em dispositivos móveis.

Comentários:

Questão didática! Vamos analisar cada item em detalhes. Primeiramente, a alternativa A está incorreta, pois o Android é baseado em núcleo Linux, porém não é utilizado exclusivamente em celulares. Além dos celulares, ele também é usado em tablets, smartwatches, smart TVs e outros dispositivos inteligentes. A alternativa B também está incorreta, uma vez que o Android é um sistema operacional de código aberto, ou seja, não é proprietário. Da mesma forma, a alternativa C está equivocada. O Android não é exclusivamente utilizado em celulares, abrangendo uma variedade de dispositivos móveis e inteligentes. Já a alternativa D não é correta. Embora o Android seja um sistema operacional de código aberto, ele não é restrito ao uso exclusivo em celulares. Por fim, a alternativa E é a correta. O Android é baseado no núcleo Linux e é utilizado principalmente em dispositivos móveis, tais como celulares, tablets, smartwatches e outros dispositivos inteligentes. Sua flexibilidade e popularidade contribuíram para que ele se tornasse um dos sistemas operacionais mais amplamente utilizados no mundo móvel.

Gabarito: Letra E

18. (Quadrix – CFO-DF / 2017) O Android possui um emulador que permite simular o sistema operacional real. Contudo, não é possível executar operações como excluir e(ou) recuperar arquivos do emulador.

Comentários:

O Android possui o Android Emulator, um emulador oficial que permite simular um dispositivo Android no computador. É amplamente usado por desenvolvedores para testar aplicativos sem a necessidade de um dispositivo físico. Contrariando a afirmação anterior, é possível realizar operações como excluir e recuperar arquivos no emulador. Ele oferece recursos avançados, incluindo acesso ao sistema de arquivos, permitindo criar, excluir, modificar e recuperar arquivos e pastas dentro do emulador, reproduzindo as ações que seriam feitas em um dispositivo real. Essa funcionalidade é de extrema importância no desenvolvimento e teste de aplicativos, permitindo aos desenvolvedores simular diferentes cenários e comportamentos do sistema operacional Android.

Gabarito: Errado

19.(CEBRASPE – TRE-PI / 2016) Com relação à plataforma Android, assinale a opção correta.

- a) Webkit é uma biblioteca redenzadora de páginas para navegadores com suporte a DOOM e AJAX.
- b) Dalvik é um gerenciador de banco de dados para o armazenamento de dados estruturados.
- c) A camada RunTime, na arquitetura Android, fica acima de todas as outras camadas e é nela que as aplicações Java são executadas.
- d) Na arquitetura Android, a Activity Manager, presente na camada Libraries, gerencia a execução de uma activity, incluindo sua iniciação e seu término.
- e) A Content Providers, na arquitetura Android, gerencia as apresentações de janelas e os tratamentos gráficos das aplicações.

Comentários:

A alternativa correta é: A) WebKit é uma biblioteca redenzadora de páginas para navegadores com suporte a DOOM e AJAX. O WebKit é um motor de renderização utilizado em navegadores web para renderizar páginas. Ele é amplamente adotado por navegadores populares, como o Google Chrome e o Safari. O WebKit possui suporte para tecnologias web avançadas, incluindo HTML, CSS e JavaScript, permitindo a exibição correta e interativa de conteúdo web. Embora o suporte a DOOM e AJAX não esteja diretamente relacionado ao WebKit, a biblioteca oferece recursos que possibilitam a execução dessas tecnologias em navegadores que o utilizam. Vejamos os erros dos demais itens: b) Dalvik não é um gerenciador de banco de dados para o armazenamento de dados estruturados. Na verdade, o Dalvik é uma máquina virtual baseada em registradores, projetada e escrita por Dan Bornstein com contribuições de outros engenheiros do Google como parte da plataforma Android para telefones celulares, para executar aplicativos Android. Ele foi substituído pela máquina virtual Android Runtime (ART) a partir do Android 5.0. O Dalvik e o ART são responsáveis por executar o código bytecode dos aplicativos Android. O item c está incorreto. A afirmação de que a camada RunTime fica acima de todas as outras camadas na arquitetura Android e é nela que as aplicações Java são executadas está incorreta. Na arquitetura do Android, a camada RunTime não é uma camada individual. O tempo de execução é fornecido pela máquina virtual Android Runtime (ART), que está localizada na camada de aplicação. É nessa camada que as aplicações Java são executadas, interagindo com as outras camadas do sistema

operacional Android. O item d também está errado. A Activity Manager não está presente na camada Libraries na arquitetura Android. Na verdade, a Activity Manager faz parte do sistema operacional Android e está localizada na camada de sistema (System Layer). A Activity Manager é responsável por gerenciar as atividades do aplicativo, controlando o ciclo de vida das atividades, gerenciando a pilha de atividades e garantindo uma transição suave entre as diferentes atividades. Por fim, a afirmação de que os Content Providers gerenciam as apresentações de janelas e os tratamentos gráficos das aplicações na arquitetura Android também está incorreta. Os Content Providers são componentes do sistema Android que permitem o compartilhamento estruturado de dados entre aplicativos. Eles não estão diretamente envolvidos na gerência das apresentações de janelas ou tratamentos gráficos das aplicações. Essas responsabilidades são da camada de interface do usuário (User Interface Layer) e dos componentes de interface do usuário, como activities, fragments e views.

Gabarito: Letra A

20. (CEBRASPE – TCE-PA / 2016) O Android, sistema operacional Linux multiusuário em que cada aplicativo é visto como um usuário diferente, atribui a cada aplicativo uma identidade de usuário exclusiva.

Comentários:

Perfeita questão! O Android, como sistema operacional Linux multiusuário, possui uma abordagem de segurança em que cada aplicativo é tratado como um usuário individual. Essa abordagem é essencial para garantir a proteção e o isolamento entre os aplicativos.

Ao iniciar um app Android, ele é ativado em sua própria sandbox de segurança, que é um ambiente isolado. O sistema operacional atribui a cada aplicativo um código de usuário exclusivo, desconhecido para o aplicativo em si. Isso significa que somente o código de usuário atribuído ao aplicativo pode acessar seus arquivos e recursos.

Além disso, cada aplicativo é executado em seu próprio processo do Linux, com sua própria máquina virtual (VM). Isso proporciona um nível adicional de isolamento, garantindo que o código de um aplicativo seja executado separadamente dos demais. O Android inicia o processo de um aplicativo quando necessário e encerra-o quando não está mais em uso, liberando memória para outros aplicativos.

Gabarito: Correto

21. (IADES – Perito Criminal (PC DF)/ 2016) Quer saber o que é um celular Android? O Android é um sistema operacional para aparelhos móveis – celulares (nesse caso, smartphones) e tablets. O Android foi desenvolvido por um consórcio de empresas liderado pelo Google e, com várias funcionalidades, permite que você realize muitas atividades com um celular em mãos. Por exemplo: com um smartphone Android, você pode aproveitar muitos serviços do Google. Se

quer encontrar uma rua, basta buscar no Google Maps. Para receber e enviar e-mails no dia a dia, o Gmail é sempre bem-vindo. E, se você quiser ainda ter momentos de descontração, pode assistir a vários vídeos no YouTube.

Um dos destaques do Android é que ele é um sistema aberto. Isso quer dizer que diversos desenvolvedores podem atuar nesse sistema, entregando melhorias conforme novas tecnologias vão surgindo.

Disponível em: <<http://www.zoom.com.br/celular/deumzoom/o-que-e-umcelular-android>>.
Acesso em: 12 abr. 2016, com adaptações.

Acerca do controle de processos do Sistema Operacional Android, assinale a alternativa correta.

- a) Broadcast receivers são componentes responsáveis por receber e tratar eventos oriundos do sistema ou de outras aplicações e não possuem interface com o usuário, apesar de poderem lançar notificações de alerta.
- b) Para permitir que um aplicativo interfira com a sandbox de outro aplicativo, acesse dados privados, ou execute quaisquer funções que não estão diretamente relacionadas com a aplicação em si, deve-se declarar a permissão em arquivo XML específico.
- c) Uma activity é uma descrição abstrata de uma operação a ser executada. Ela representa uma mensagem, um pedido que é encaminhado ao sistema operacional, e pode ativar um broadcast ou enviar uma mensagem para aplicações que executam em outros processos, entre outros.
- d) As permissões que cada aplicação possui no Android são dinamicamente declaradas, e é responsabilidade do usuário determinar o nível de permissão que a aplicação que está sendo instalada possui.
- e) Uma intent é basicamente um elemento de gestão da aplicação Android para a interface com o usuário. Todo aplicativo Android começa por ela.

Comentários:

Vamos analisar cada item. A alternativa A, em relação à definição de broadcast receivers, pode-se afirmar que está correta. Broadcast receivers são componentes cruciais no sistema Android responsáveis por receber e lidar com eventos provenientes do sistema ou de outras aplicações. Eles atuam como observadores, aguardando eventos específicos para realizar ações. Embora não tenham interface com o usuário, eles têm a capacidade de lançar notificações de alerta quando necessário. Essa funcionalidade permite que os aplicativos reajam a eventos importantes em tempo real, como receber uma nova mensagem ou notificação. b) A afirmação de que um aplicativo precisa declarar permissões em um arquivo XML específico para interferir na sandbox de outro aplicativo, acessar dados privados ou executar funções não relacionadas à aplicação em si está incorreta. Na verdade, a sandbox de segurança do Android impede que um aplicativo acesse dados

ou execute funções além do escopo da sua própria aplicação, a menos que permissões explícitas sejam concedidas pelo usuário. c) A descrição apresentada sobre uma activity no Android está incorreta. Uma activity não é uma descrição abstrata de uma operação a ser executada, nem representa uma mensagem ou pedido encaminhado ao sistema operacional. Na verdade, uma activity é uma unidade fundamental de interação com o usuário, representando uma tela com interface gráfica em um aplicativo Android. d) A afirmação de que as permissões que cada aplicação possui no Android são dinamicamente declaradas e que é responsabilidade do usuário determinar o nível de permissão está equivocada. No Android, as permissões são estaticamente declaradas pelo desenvolvedor durante a criação do aplicativo, por meio de arquivos XML. O usuário tem a responsabilidade de revisar e conceder as permissões solicitadas durante a instalação do aplicativo, mas não determinar o nível de permissão. e) A informação de que uma intent é um elemento de gestão da aplicação Android para a interface com o usuário e que todo aplicativo Android começa por meio dela está incorreta. Na verdade, uma intent é um objeto usado para iniciar componentes do Android, como activities, serviços e broadcast receivers, e para facilitar a comunicação e interação entre esses componentes. A execução de um aplicativo Android pode começar por meio de uma intent, mas ela não é exclusivamente responsável por isso.

Gabarito: Letra A

22.(FUNRIO – IF-PA/ 2016) Pode-se dividir o software em três categorias bem específicas: aplicativos e/ou utilitários; linguagens de programação e/ou compiladores e sistemas operacionais.

Assinale a alternativa que apresenta apenas sistemas operacionais para dispositivos móveis.

- a) WinZip, Windows Phone e Mac OS.
- b) Linux, Acrobat Reader e Android.
- c) MS Office Access, WinZip, Acrobat Reader.
- d) iOS, Android e Windows Phone.
- e) Windows, Linux, Mac OS.

Comentários:

Questão boa para dar uma revisada nessa sopa de letrinhas, vamos entender o que significa cada termo utilizado pela banca. A) WinZip, Windows Phone e Mac OS: WinZip é um aplicativo/utilitário de compactação e descompactação de arquivos, não é um sistema operacional. Windows Phone é um sistema operacional para dispositivos móveis, portanto, está correto. Mac OS é um sistema operacional para computadores da Apple, não é específico para dispositivos móveis. B) Linux, Acrobat Reader e Android: Linux é um sistema operacional de código aberto utilizado em diversas plataformas, incluindo dispositivos móveis, então está correto. Acrobat Reader é um aplicativo/utilitário para visualização de documentos PDF, não é um sistema operacional. Android é um sistema operacional para dispositivos móveis, então está correto. C) MS Office Access, WinZip, Acrobat Reader: MS Office Access é um aplicativo/utilitário de banco de dados, não é um sistema operacional. WinZip já foi falado. Acrobat Reader também já foi explicado no item B. D) iOS,

Android e Windows Phone: iOS é um sistema operacional desenvolvido pela Apple exclusivamente para seus dispositivos móveis, então está correto. Android é um sistema operacional para dispositivos móveis, então está correto. Windows Phone é um sistema operacional para dispositivos móveis, então está correto. E) Windows, Linux, Mac OS: Windows já explicamos. Linux também. Mac OS também foi explicado no item A. Portanto, a alternativa correta é a D) iOS, Android e Windows Phone, pois são sistemas operacionais específicos para dispositivos móveis.

Gabarito: Letra D

23. (FGV – DPE-MT / 2015) A segurança é um fator especialmente importante em dispositivos móveis, dada a quantidade de informações pessoais tipicamente armazenadas e transitadas neles. Por esse motivo, o sistema operacional iOS não permite a execução de aplicativos sem uma assinatura digital certificada pela Apple Inc. Considerando essa limitação, a instalação de um perfil de Provisionamento de Distribuição Corporativa em um aparelho com iOS permite

- a) a execução de aplicativos corporativos obtidos da App Store.
- b) a instalação de aplicativos a partir de um endereço corporativo.
- c) a assinatura digital de um aplicativo independente da Apple Inc.
- d) a execução de aplicativos assinados obtidos fora da App store.
- e) a instalação de aplicativos sem uma assinatura digital.

Comentários:

A instalação de um perfil de Provisionamento de Distribuição Corporativa em um aparelho com iOS permite a execução de aplicativos assinados obtidos fora da App Store. No sistema operacional iOS, os aplicativos normalmente são obtidos e instalados através da App Store, onde passam por um processo de revisão e são assinados digitalmente pela Apple Inc. Essa assinatura digital garante a autenticidade e integridade dos aplicativos, fornecendo uma camada de segurança adicional aos usuários. No entanto, através do perfil de Provisionamento de Distribuição Corporativa, é possível instalar aplicativos assinados fora da App Store, desde que possuam uma assinatura digital certificada pela Apple Inc. Esse tipo de perfil é utilizado por empresas e organizações para distribuir aplicativos internos ou desenvolvidos exclusivamente para uso corporativo, sem a necessidade de publicá-los na App Store.

Assim, a opção correta é a letra D) a execução de aplicativos assinados obtidos fora da App Store. Isso significa que, com um perfil de Provisionamento de Distribuição Corporativa instalado, é possível executar aplicativos que foram assinados digitalmente e obtidos por outros meios, sem a restrição de depender exclusivamente da App Store. Isso oferece maior flexibilidade e controle para empresas e desenvolvedores na distribuição de aplicativos iOS.

Gabarito: Letra D

24. (FGV – DPE-MT / 2015) Apesar dos incrementos das capacidades de memória e de processamento dos dispositivos móveis, limites são sempre atingidos em razão da demanda

crescente dos usuários por aplicativos com as mais variadas funções, muitas vezes envolvendo o uso de múltiplos sensores e recursos gráficos de alta qualidade.

Caso o conjunto de aplicativos utilizados exceda o total de memória disponível, o sistema operacional Android irá

- a) solicitar ao usuário que termine um dos aplicativos em uso.
- b) terminar automaticamente o aplicativo de menor prioridade.
- c) bloquear toda a execução do sistema, requerendo reinício.
- d) terminar todos os aplicativos.
- e) terminar apenas o aplicativo em uso.

Comentários:

Caso o conjunto de aplicativos utilizados exceda o total de memória disponível, o sistema operacional Android irá automaticamente terminar o aplicativo de menor prioridade. Os dispositivos móveis possuem recursos limitados de memória, e a demanda por aplicativos cada vez mais complexos e com diversas funcionalidades pode levar ao esgotamento dessa capacidade. O sistema operacional Android é projetado para gerenciar essa situação de forma eficiente. Quando a memória disponível começa a se esgotar, o Android utiliza um mecanismo chamado "Gerenciador de Processos" para determinar a prioridade de cada aplicativo em execução. Cada aplicativo é atribuído a uma categoria de prioridade com base em fatores como a interação do usuário, o estado atual do aplicativo e outras considerações do sistema. Ao atingir o limite de memória, o Android tomará a decisão de encerrar automaticamente o aplicativo de menor prioridade em execução. Isso significa que o aplicativo que está sendo menos utilizado ou que possui menor relevância para o usuário será finalizado, liberando assim recursos de memória para os aplicativos de maior prioridade. Essa abordagem permite que o sistema operacional Android otimize o uso da memória disponível, priorizando os aplicativos mais importantes e garantindo um desempenho adequado para as funcionalidades essenciais do dispositivo móvel. Dessa forma, mesmo com limitações de memória, o Android busca equilibrar a execução dos aplicativos de acordo com suas prioridades.

Gabarito: Letra B

LISTA DE QUESTÕES

Android

1. **(AOCP – PRODEB/2018)** Considerando as linguagens de programação mobile, qual das dispostas a seguir foi criada pela Apple e pode ser utilizada para o desenvolvimento das suas aplicações?

a) Kotlin
b) Swift
c) C#
d) Python
e) IOS

2. **(FGV – IBGE / 2016)** Um desenvolvedor Android deseja inserir a funcionalidade de backup em uma aplicação móvel para, de tempos em tempos, armazenar dados automaticamente. A classe da API de Backup (versão 6.0 ou superior) a ser utilizada é a:

a) BkpAgent;
b) BkpHelper;
c) BackupManager;
d) BackupOutputData;
e) BackupDataStream.

3. **(FGV – Banestes / 2018)** Sempre que um aplicativo precisa de acesso a um recurso protegido por uma permissão no sistema operacional Android, ele precisa declarar essa necessidade incluindo um elemento <uses-permission> no arquivo Manifest do aplicativo.

A permissão que deve ser incluída no arquivo Manifest para que o aplicativo possa identificar se o aparelho está conectado a uma rede e qual o tipo de conexão é:

a) android.permission.ACCESS_FINE_LOCATION;
b) android.permission.ACCESS_NETWORK_STATE;
c) android.permission.INTERNET;
d) android.permission.READ_SYNC_SETTINGS;
e) android.permission.STATUS_BAR.

4. **(FUNDATEC – CRF-PR / 2021)** Android Enterprise é uma solução para dispositivos Android que visa gerenciar o uso de dispositivos móveis no ambiente corporativo. Analise os casos de uso citados abaixo e assinale a opção que NÃO é suportada pelo Android Enterprise.

- a) Dispositivos de propriedade do empregado, que suportam um perfil pessoal e outro profissional armazenados separadamente.
- b) Dispositivos de propriedade da empresa, que suportam dois perfis (pessoal e profissional) separados e adicionalmente oferece um controle maior de políticas de segurança.
- c) Dispositivos de propriedade da empresa, que suportam somente um perfil profissional e totalmente gerenciado pela empresa.
- d) Dispositivos para serviços dedicados, que suportam somente perfil profissional, totalmente gerenciado pela empresa e com a característica de geralmente trabalhar somente alguns poucos aplicativos.
- e) Dispositivos de propriedade da empresa, que suportam dois perfis (pessoal e profissional), nos quais a empresa tem acesso a ambos, por questões de segurança.

5. (AOCP – Pref Novo Hamburgo / 2020) São considerados sistemas operacionais móveis:

- a) Windows Phone e Windows XP.
- b) Red Hat Enterprise Linux e Android.
- c) iOS e OSX.
- d) Android e Minix.
- e) Symbian OS e Windows Phone.

6. (IBADE – Pref Jarú/ 2019) O sistema operacional indicado para dispositivos móveis é denominado:

- a) Windows Server.
- b) Linux.
- c) IOS.
- d) Z/OS.
- e) AIX.

7. (UNESC – Pref Criciúma/ 2019) O sistema android possibilita que seus usuários possam agrupar seus aplicativos em pastas de acordo com sua necessidade. Qual a ação necessária para a criação de uma pasta no android:

- a) Pressionar com o dedo na tela e manter pressionado até o aparecimento do menu pasta, posteriormente clicar em nova pasta.
- b) Arrastar um aplicativo e sobrepô-lo a outro.
- c) Pressionar o botão menu e acessar as configurações.

d) Pressionar os botões menu e voltar ao mesmo tempo.

8. (UNESC – Pref Criciúma/ 2019) As primeiras versões do sistema operacional android apresentavam a barra de navegação de forma física. Com o surgimento de novas versões do sistema a barra de navegação ficou virtual. Os botões que compõem a barra de navegação do android são?

- a) Desligar, aumentar e diminuir volume.
- b) Menu, voltar e aplicativos abertos.
- c) Voltar, iniciar e aplicativos abertos.
- d) Menu, volta, Iniciar.

9. (FAU UNICENTRO – IF PR/ 2019) No desenvolvimento para dispositivos móveis utilizando Android podemos utilizar alguns tipos de layout que facilitam o desenvolvimento das telas de aplicativos. Relacione os tipos de layouts e suas descrições e assinale a alternativa com a sequência correta:

1 - AbsoluteLayout.

2 - FrameLayout.

3 - LinearLayout.

4 - TableLayout.

5 - RelativeLayout.

() Permite posicionar um componente relativo a outro, por exemplo, abaixo ou acima de um componente existente.

() Utilizado quando necessário que um componente preencha a tela inteira do dispositivo automaticamente.

() Permite posicionar componentes, fornecendo as coordenadas x e y.

() Utilizado para organizar os componentes na vertical ou horizontal.

() Utilizado para organizar os componentes em uma tabela, com linhas e colunas.

A sequência correta de cima para baixo é:

- a) 2, 3, 4, 1, 5.

- b) 3, 5, 1, 2, 4.
- c) 1, 4, 5, 2, 3.
- d) 5, 2, 1, 3, 4.
- e) 5, 1, 2, 3, 4

- 10. (CEBRASPE – ABIN / 2018)** O Android disponibiliza um banco de dados público local, orientado a objetos, para o armazenamento de dados estruturados, o que possibilita o gerenciamento das aplicações e dos dados de forma rápida e segura.
- 11. (CEBRASPE – ABIN / 2018)** Mesmo controlando o login e a senha do usuário via contas Google, um aplicativo pode capturar e enviar arquivos armazenados no cartão SD do celular que utiliza o sistema Android.
- 12. (CEBRASPE – ABIN / 2018)** Quando a Internet está disponível, os aplicativos executados em segundo plano podem efetuar requisições, que utilizam muita carga da bateria e podem ocasionar erros nos aplicativos, por isso, na versão 8.0 do sistema, os manifestos não podem ocorrer para transmissões implícitas
- 13. (CEBRASPE – ABIN / 2018)** Para garantir que o software gerado no servidor chegue ao usuário final, utiliza-se um certificado code signing, que altera o software e também insere uma assinatura do desenvolvedor ou fabricante.
- 14. (CONSULPLAN – Câmara de Belo Horizonte - MG / 2018)** “Classe do Android que pode ser utilizada para enviar uma mensagem para o sistema operacional; solicitar ao sistema operacional que ligue para determinado número de celular etc.” Assinale-a.
- a) Intent.
 - b) Service.
 - c) Content.
 - d) Receiver.
- 15. (AOCP – PRODEB / 2018)** Android é um sistema operacional (SO) amplamente utilizado em dispositivos móveis como por exemplo smartphones. A programação para este SO utiliza a linguagem Java e permite a criação e a manipulação de vários objetos como as activitys. Levando em consideração este objeto da programação para Android, escolha a alternativa que representa o comando utilizado para a criação de uma activity?
- a) onStart
 - b) onResume
 - c) onPause
 - d) onCreate
 - e) onDestroy

16. (IBADE – Câmara de Vilhena - RO / 2018) Os aplicativos desenvolvidos para dispositivos móveis como celulares e tablets são denominados:

- a) Android.
- b) IOS.
- c) Kindle.
- d) APP.
- e) REXX.

17. (AOCP – CODEM - PA / 2017) O Sistema Operacional Android é

- a) baseado em núcleo Linux e é utilizado exclusivamente em celulares.
- b) um sistema operacional proprietário e é utilizado principalmente em dispositivos móveis.
- c) um sistema operacional proprietário e é utilizado exclusivamente em celulares.
- d) um sistema operacional livre e é utilizado exclusivamente em celulares.
- e) baseado em núcleo Linux e é utilizado principalmente em dispositivos móveis.

18. (Quadrix – CFO-DF / 2017) O Android possui um emulador que permite simular o sistema operacional real. Contudo, não é possível executar operações como excluir e(ou) recuperar arquivos do emulador.

19. (CEBRASPE – TRE-PI / 2016) Com relação à plataforma Android, assinale a opção correta.

- a) Webkit é uma biblioteca redenzadora de páginas para navegadores com suporte a DOOM e AJAX.
- b) Dalvik é um gerenciador de banco de dados para o armazenamento de dados estruturados.
- c) A camada RunTime, na arquitetura Android, fica acima de todas as outras camadas e é nela que as aplicações Java são executadas.
- d) Na arquitetura Android, a Activity Manager, presente na camada Libraries, gerencia a execução de uma activity, incluindo sua iniciação e seu término.
- e) A Content Providers, na arquitetura Android, gerencia as apresentações de janelas e os tratamentos gráficos das aplicações.

20. (CEBRASPE – TCE-PA / 2016) O Android, sistema operacional Linux multiusuário em que cada aplicativo é visto como um usuário diferente, atribui a cada aplicativo uma identidade de usuário exclusiva.

21. (IADES – Perito Criminal (PC DF) / 2016) Quer saber o que é um celular Android? O Android é um sistema operacional para aparelhos móveis – celulares (nesse caso, smartphones) e tablets. O Android foi desenvolvido por um consórcio de empresas liderado pelo Google e, com várias

funcionalidades, permite que você realize muitas atividades com um celular em mãos. Por exemplo: com um smartphone Android, você pode aproveitar muitos serviços do Google. Se quer encontrar uma rua, basta buscar no Google Maps. Para receber e enviar e-mails no dia a dia, o Gmail é sempre bem-vindo. E, se você quiser ainda ter momentos de descontração, pode assistir a vários vídeos no YouTube.

Um dos destaques do Android é que ele é um sistema aberto. Isso quer dizer que diversos desenvolvedores podem atuar nesse sistema, entregando melhorias conforme novas tecnologias vão surgindo.

Disponível em: <<http://www.zoom.com.br/celular/deumzoom/o-que-e-umcelular-android>>.
Acesso em: 12 abr. 2016, com adaptações.

Acerca do controle de processos do Sistema Operacional Android, assinale a alternativa correta.

- a) Broadcast receivers são componentes responsáveis por receber e tratar eventos oriundos do sistema ou de outras aplicações e não possuem interface com o usuário, apesar de poderem lançar notificações de alerta.
- b) Para permitir que um aplicativo interfira com a sandbox de outro aplicativo, acesse dados privados, ou execute quaisquer funções que não estão diretamente relacionadas com a aplicação em si, deve-se declarar a permissão em arquivo XML específico.
- c) Uma activity é uma descrição abstrata de uma operação a ser executada. Ela representa uma mensagem, um pedido que é encaminhado ao sistema operacional, e pode ativar um broadcast ou enviar uma mensagem para aplicações que executam em outros processos, entre outros.
- d) As permissões que cada aplicação possui no Android são dinamicamente declaradas, e é responsabilidade do usuário determinar o nível de permissão que a aplicação que está sendo instalada possui.
- e) Uma intent é basicamente um elemento de gestão da aplicação Android para a interface com o usuário. Todo aplicativo Android começa por ela.

22.(FUNRIO – IF-PA/ 2016) Pode-se dividir o software em três categorias bem específicas: aplicativos e/ou utilitários; linguagens de programação e/ou compiladores e sistemas operacionais.

Assinale a alternativa que apresenta apenas sistemas operacionais para dispositivos móveis.

- a) WinZip, Windows Phone e Mac OS.

- b) Linux, Acrobat Reader e Android.
- c) MS Office Access, WinZip, Acrobat Reader.
- d) iOS, Android e Windows Phone.
- e) Windows, Linux, Mac OS.

23. (FGV – DPE-MT / 2015) A segurança é um fator especialmente importante em dispositivos móveis, dada a quantidade de informações pessoais tipicamente armazenadas e transitadas neles. Por esse motivo, o sistema operacional iOS não permite a execução de aplicativos sem uma assinatura digital certificada pela Apple Inc. Considerando essa limitação, a instalação de um perfil de Provisionamento de Distribuição Corporativa em um aparelho com iOS permite

- a) a execução de aplicativos corporativos obtidos da App Store.
- b) a instalação de aplicativos a partir de um endereço corporativo.
- c) a assinatura digital de um aplicativo independente da Apple Inc.
- d) a execução de aplicativos assinados obtidos fora da App store.
- e) a instalação de aplicativos sem uma assinatura digital.

24. (FGV – DPE-MT / 2015) Apesar dos incrementos das capacidades de memória e de processamento dos dispositivos móveis, limites são sempre atingidos em razão da demanda crescente dos usuários por aplicativos com as mais variadas funções, muitas vezes envolvendo o uso de múltiplos sensores e recursos gráficos de alta qualidade.

Caso o conjunto de aplicativos utilizados exceda o total de memória disponível, o sistema operacional Android irá

- a) solicitar ao usuário que termine um dos aplicativos em uso.
- b) terminar automaticamente o aplicativo de menor prioridade.
- c) bloquear toda a execução do sistema, requerendo reinício.
- d) terminar todos os aplicativos.
- e) terminar apenas o aplicativo em uso.

Kotlin

1. **(CESGRANRIO – Banco do Brasil/ 2023)** Kotlin é uma linguagem de programação usada no desenvolvimento Android.

Entre suas características, está um grau de compatibilidade com Java, que permite

- a) chamar funções feitas em Java, apenas, mas não permite que suas funções Kotlin sejam chamadas por Java.
 - b) ler dados que foram salvos por apps Java, apenas.
 - c) ler e escrever dados que podem ser lidos e escritos por apps Java, apenas.
 - d) ter suas funções chamadas por Java, apenas, mas não consegue chamar funções feitas em Java.
 - e) construir apps com código parcialmente em Java e parcialmente em Kotlin, sem restrições.
2. **(FCM – CEFETMINAS / 2022)** Com relação ao desenvolvimento de aplicativos móveis, relacione as linguagens de programação com suas características.

LINGUAGEM DE PROGRAMAÇÃO

- 1 - Java
- 2 - JavaScript
- 3 - Kotlin
- 4 - TypeScript
- 5 - Objective-C

CARACTERÍSTICAS

() É um superconjunto da linguagem de programação C, ou seja, agrega recursos ao C. Ele possibilita o uso do paradigma programação orientada a objetos, contendo sintaxe para a criação de métodos e classes.

() É uma linguagem mais recente e que tem ganhado força nos últimos tempos. É desenvolvida e mantida pela JetBrains. Utiliza o paradigma orientado a objetos e tem suporte ao paradigma funcional, com o uso de expressões lambda (anônimas).

() É uma linguagem de programação usada principalmente para controlar o Hypertext Markup Language (HTML) e o Cascading Style Sheets (CSS) e manipular comportamentos em uma página web. É mantido pela European Computer Manufacturer's Association (ECMA). Originalmente, foi criada para o desenvolvimento de aplicações no lado cliente, mas evoluiu para possibilitar o desenvolvimento de aplicações desktop e no lado servidor.

() Mantida pela Oracle, é composta por uma linguagem de programação e uma plataforma computacional utilizada como base por muitas aplicações. É orientada a objetos, o que significa ser baseada na modelagem e comunicação entre os objetos. Também é uma linguagem estaticamente tipada, ou seja, o usuário precisa declarar o tipo de dados que será armazenado em cada variável declarada.

() É uma linguagem de programação desenvolvida pela Microsoft e que possui tipagem. A tipagem possibilita que o desenvolvedor declare o tipo de uma variável, como numérico, textual ou data, por exemplo. Ele permite desenvolver aplicações tanto do lado do cliente como do lado do servidor.

A sequência correta dessa associação é

- a) 5, 3, 2, 1, 4.
- b) 4, 3, 2, 1, 5.
- c) 5, 1, 3, 2, 4.
- d) 1, 2, 3, 5, 4.
- e) 4, 5, 1, 2, 3.

3. (FGV – TJ RO/ 2021) Analise o código Kotlin exibido a seguir.

```
fun main() {  
    val classe = ?  
    val resposta = when (classe) {  
        1 -> "Premium"  
        2 -> "Superior"  
        3 -> "Econômica"  
        4, 5 -> when (classe % 2 == 0)  
            {true -> "Gold" false -> "Iron"}  
        else -> "Dado inválido."  
    }  
    println(resposta)  
}
```

O par que indica corretamente o valor a ser atribuído à variável classe, na segunda linha, e o valor computado e exibido pelo comando println é:

- a) 3 / Iron
- b) 4 / Iron
- c) 5 / Iron
- d) 5 / Gold
- e) 9 / Gold

4. **(Cesgranrio – Caixa / 2021)** Na linguagem de programação Kotlin, é possível criar uma variável cujo valor nunca pode ser mudado, na prática, uma constante, com o nome `idademinima`, do tipo básico inteiro de 32 bits, com o valor 18.

Para que isso aconteça, qual das seguintes instruções deve ser usada?

- a) `val idademinima : Int = 18`
- b) `val idademinima : Integer = 18`
- c) `val idademinima = 18 : Integer`
- d) `var idademinima : Int = 18`
- e) `var idademinima : Integer = 18`

5. **(CESPE – PGDF/ 2021)** O Spring WebFlux é compatível com Java 8 lambdas e Kotlin e tem a vantagem de permitir a criação de microsserviços com requisitos menos complexos.

6. **(CESPE – PGDF/ 2021)** O JUnit 5 é formado por JUnit Platform, JUnit Jupiter e JUnit Vintage; o JUnit Jupiter pode ser utilizado em programas escritos em Kotlin.

7. **(AOCP – Prefeitura de Novo Hamburgo - RS / 2020)** Considerando o ambiente de desenvolvimento Android Studio, assinale a alternativa que apresenta apenas linguagens válidas ao adicionar uma nova `activity` ao projeto.

- a) Java e Swift.
- b) C e Java.
- c) JavaScript e Python.
- d) C# e PHP.
- e) Java e Kotlin.

8. **(CESGRANRIO – Banco do Brasil / 2021)** Foi solicitado a um programador de sistemas de informação que transformasse uma classe escrita em Java em uma classe equivalente, para ser executada em um programa Kotlin.

O código da classe Java é:

```
public class AlunoJava{
```

```
private String codigo;  
private String nome;  
private int numero=0;  
private String texto= "EscolaX";  
public AlunoJava (String codigo,String nome)  
    { this.codigo = codigo;  
      this.nome = nome; }  
    }
```

A classe em Kotlin equivalente à classe Java acima é

a) public class AlunoKotlin (private String: nome , private String: codigo)
 { private:
 numero int = 0
 texto String = "EscolaX" }

b) public class AlunoKotlin (private var nome; codigo: String)
 { private var numero = 0
 private var texto = "EscolaX" }

c) class AlunoKotlin (val nome: String, val codigo: String)
 { private this.nome = nome
 private this.codigo=codigo
 private var int numero = 0
 private var String texto = "EscolaX" }

d) class AlunoKotlin (var nome: String, var codigo: String)
 { private var numero = 0
 private var texto = "EscolaX"
 private AlunoKotlin.nome, AlunoKotlin.codigo }

e)class AlunoKotlin (private val nome: String, private val codigo: String)
 { private var numero = 0
 private var texto = "EscolaX" }

9. (AOCP – SANESUL/ 2021) Qual é o nome de uma API de Fluxo do Kotlin para programação android que é considerada uma ótima opção para classes que precisam manter um estado mutável observável?

- a) ClassFlow
- b) MutableFlow

- c) QuickFlow
- d) StateFlow
- e) SharedFlow

10.(UNICENTRO – UNICENTRO / 2019) Assinale a alternativa que contenha o código na linguagem de programação Kotlin que NÃO será executado na plataforma Android por erro de compilação:

a) `text!!.let {
 val tamanho = text.length
}`

b) `var res = when(pontos) {
 9, 10 -> "Excelente"
 in 6..8 -> "Bom"
 4, 5 -> "Ok"
 in 1..3 -> "Ruim"
 else -> "Ruim"
}`

c) `for (i in 1..10) {
 println(i)
}`

d) `for (i in 10 upTo 0) {
 println(i)
}`

e) `for (i in 10 downTo 1 step 2) {
 println(i)
}`

React Native

1. **(CESGRANRIO – Banco do Brasil / 2023)** O React Native 0.59 introduziu o conceito de Hooks. Entre os Hooks, tem-se o usestate, que permite
 - a) calcular o estado de um CEP ou ZIP de acordo com o Locale.
 - b) chamar estados específicos do engine React para alterar seu comportamento.
 - c) declarar uma classe que segue o padrão de design state.
 - d) criar uma enumeration que representa estados.
 - e) manter um estado local em uma função de um componente funcional.
2. **(FCC – PGE AM/ 2022)** Considere que um desenvolvedor está criando um aplicativo usando React e React Native e deseja criar um elemento h1 contendo o título Amazonas, aplicando a classe de estilo CSS de nome tit e armazenando em uma constante chamada elemento. Para realizar esta tarefa, ele terá que utilizar a instrução
 - a) `let elemento = ReactNative.createElement('h1',{className:'tit'},'Amazonas');`
 - b) `let const elemento = React.createElement('h1'.tit},'Amazonas');`
 - c) `const elemento = ReactNative.createElement('h1',{className:'tit'},'Amazonas');`
 - d) `const elemento = React.createElement('h1',{className:'tit'},'Amazonas');`
 - e) `const elemento = React.createElement('h1'.tit,'Amazonas');`
3. **(Consulplan – SEED-PR/ 2022)** O React Native é uma plataforma baseada no React que possibilita o desenvolvimento de aplicativos mobile híbridos, ou seja, que rodam tanto no iOS quanto no Android. Assinale, a seguir, a funcionalidade do React Native que permite fazer com que o programa fique rodando constantemente em background e, a cada vez que o código é alterado, ele é interpretado, seu build é feito e as suas alterações são mostradas rapidamente na tela.
 - a) Expo.
 - b) Snack.
 - c) Hot Reloading.
 - d) Desenvolvimento paralelo.
4. **(CEBRASPE – SLU-DF / 2019)** React Native utiliza componentes nativos em vez de componentes da Web como blocos de construção, existindo dois tipos de dados que controlam um componente: state, definido pelo pai e fixado durante todo o tempo de vida de um componente; e props, utilizado para os dados que irão mudar.

5. (UFC – UFC/ 2019 – Adaptada) O React.js é um framework de código aberto usado para desenvolver aplicativos para Android, iOS e UWP.
6. (FCC – METRÔ-SP / 2019) Um Analista precisa desenvolver um aplicativo móvel para celulares com sistemas operacionais Android e iOS. Para isso, poderá utilizar o framework desenvolvido pela equipe do Facebook, que possibilita o desenvolvimento de aplicações mobile utilizando bibliotecas JavaScript para criar interfaces de usuário. Esse framework é conhecido como
- a) Ionic Builder.
 - b) Flutter Script.
 - c) Cordova.
 - d) Xamarin Core.
 - e) React Native.

Swift

1. **(CESGRANRIO – Banco do Brasil / 2023)** Em um programa em Swift, o programador deseja incluir o resultado de uma operação dentro de uma string. Nesse contexto, considere o seguinte código:

```
let quantidade = 4 let valor = 10
```

Dado o código acima, o programador deseja uma string saída cujo valor seja

```
"valor total = 40"
```

Para isso, o programador deve utilizar o seguinte fragmento de código Swift:

- a) `let saida := "valor total = \(quantidade*valor)"`
- b) `let saida := "valor total = {\(quantidade*valor)}"`
- c) `let saida = "valor total = %[quantidade*valor]"`
- d) `let saida = "valor total = \((quantidade*valor)"`
- e) `let saida = "valor total = \[quantidade*valor]"`

2. **(AOCP – PRODEB / 2018)** Considerando as linguagens de programação mobile, qual das dispostas a seguir foi criada pela Apple e pode ser utilizada para o desenvolvimento das suas aplicações?

- a) Kotlin
- b) Swift
- c) C#
- d) Python
- e) IOS

3. **(CEBRASPE – TRE-BA / 2017)** Na linguagem Swift do IOS, ao se declarar o código `var fruta=["maça", "banana", "abacaxi"]`, a linguagem automaticamente entenderá que fruta é array de

- a) integer.
- b) floatings.
- c) strings.
- d) imutáveis.
- e) double.

4. (IF-SE – IF-SE / 2016) Em relação aos ambientes de desenvolvimento de software, analise as afirmativas abaixo.

I. O XCode é o ambiente de desenvolvimento da Apple e permite trabalhar com as linguagens Swift e C#.

II. O ambiente Eclipse suporta diversas linguagens de programação, tais como, Java, C/C++, AspectJ e PHP.

III. O ambiente NetBeans é gratuito, porém com código fechado.

IV. O Visual Studio suporta diversas linguagens de programação, tais como, C#, C++, F#, Python e Visual Basic.

De acordo com as afirmativas, marque a alternativa CORRETA:

- a) As afirmativas II e III estão corretas.
- b) Apenas as afirmativas II e IV estão corretas.
- c) Apenas a afirmativa I está incorreta
- d) Apenas a afirmativa III está incorreta.

5. (AOCP – IBGE / 2019) Compreender o ciclo de vida das views das aplicações é extremamente importante, sobretudo quando falamos de aplicações para dispositivos móveis. Sobre o ciclo de vida das aplicações iOS com Swift, assinale a alternativa que apresenta um método que é chamado toda vez que uma visão vai aparecer na tela, podendo ser chamado mais de uma vez, e é muito usado para acionar quaisquer operações que precisem ocorrer antes que a ViewController seja apresentada na tela, como atualizar os dados do usuário.

- a) viewDidLoad()
- b) viewDidAppear()
- c) viewWillAppear()
- d) viewWillLoad()
- e) viewRestart()

6. (AOCP – IBGE / 2019) Você está desenvolvendo um aplicativo iOS usando Swift, que é uma agenda de controle de tarefas do funcionário do departamento de Tecnologia da Informação que presta o serviço de manutenção e suporte para os usuários da corporação. Nesse momento do desenvolvimento, é preciso fornecer uma maneira para nosso usuário sair da listagem de tarefas e ir para a tela de nova tarefa, ou seja, é necessário trabalhar com a navegação entre telas, pois o usuário precisa navegar entre a tela de listagem e a de nova tarefa, tanto a ida quanto a volta. Para isso, é necessário ter uma barra de navegação. Assinale a alternativa que apresenta o que você deve utilizar para implementar essa ação.

- a) NavController
- b) Navigation Jetpacks
- c) Navigation Component
- d) Navigation Controller
- e) Routing Navigation

7. **(AOCP – IBGE / 2019)** Com o swift no desenvolvimento para iOS, a Apple adotou novas características e capacidades para a linguagem de programação, como o uso de protocolos. Estes trabalham de uma maneira que visa estender a funcionalidade de uma classe ou estrutura existente. Um protocolo pode ser pensado como um escopo ou interface que define um conjunto de propriedades e métodos. Um dos protocolos mais utilizados nessa linguagem de programação é o que tem a capacidade de determinar quando dois objetos são iguais e, com extensões condicionais a esse protocolo, é possível fornecer funcionalidade específica para tipos específicos de objetos em conformidade com um protocolo. Assinale a alternativa que apresenta corretamente o nome desse protocolo.

- a) Equatable.
- b) CollectionType.
- c) ViewController.
- d) NSCodering.
- e) MyDelegate.

8. **(CESGRANRIO – Banco do Brasil / 2021)** Um programador de aplicativos para dispositivos Apple com iOS recebeu a seguinte parte de um código, escrito na linguagem swift:

```
var i: Int
var texto: String
var num: Int = 0
var frase: String = ""

for i in 1...3 {
    num = num + 1 + i * 2
    switch num {
    case 2...6:
        texto = "a "
    case 7...9:
        texto = "casa "
    case 10...13:
        texto = "carro "
    case 14...16:
        texto = "eh "
```

```
case 17...20:
    texto = "o "
case 21...23:
    texto = "forte "
default:
    texto = "não eh "
}
frase = frase + texto
}
```

print(frase)

A execução dessa parte do código produz como resposta

- a) o carro eh
- b) a casa eh
- c) o carro não eh
- d) a casa eh forte
- e) o carro eh forte

9. (FEPESE – CELESC / 2022) Caso seja necessário o desenvolvimento de aplicativos para dispositivos móveis, mais especificamente para o sistema operacional IOS, assinale a alternativa que apresenta corretamente uma linguagem de programação e um ambiente de desenvolvimento que podem ser utilizados para este fim.

- a) Swift e XCODE
- b) Swift e Objective-D
- c) Objective-D e iOSCODE
- d) Objective-D e VSCODE
- e) XCODE e VSCODE

10. (CEBRASPE – TRT - 7ª Região (CE) / 2017) Assinale a opção que apresenta a linguagem de programação disponível, grátis e em código aberto, para desenvolvedores sob a licença Apache 2.0 e desenvolvida pela Apple para a criação de aplicativos para IOS.

- a) Xcode
- b) Swift
- c) Objective-C
- d) Python

11. (AOCP – PRODEB / 2018) Considerando as linguagens de programação mobile, qual das dispostas a seguir foi criada pela Apple e pode ser utilizada para o desenvolvimento das suas aplicações?

- a) Kotlin
- b) Swift
- c) C#
- d) Python
- e) IOS

12. (IBADE – Prefeitura de Itapemirim - ES / 2019) A Apple desenvolveu uma linguagem de programação própria para desenvolvimento de aplicações sob IOS. Ela se chama:

- a) C++.
- b) PHP.
- c) Python.
- d) Swift.
- e) Java Script.

Flutter

1. **(CEBRASPE – TRT - 8ª Região (PA e AP) / 2022)** O widget básico do Flutter que permite criar layouts flexíveis nas direções horizontal e vertical, com design de objetos baseado no modelo de layout flexbox da Web é o
- a) Text.
 - b) Row, Column.
 - c) Stack.
 - d) Expanded.
 - e) Container.
2. **(IBFC – EBSE RH / 2020)** Quanto aos frameworks mais destacados para o desenvolvimento de aplicativos mobile, analise as afirmativas abaixo quanto a existência dos mesmos e dê valores Verdadeiro (V) ou Falso (F).

() Flutter () Corona SDK () JQuery Mobile

Assinale a alternativa que apresenta a sequência correta de cima para baixo:

- a) V, V, V
 - b) V, V, F
 - c) V, F, V
 - d) F, F, V
 - e) F, F, F
3. **(UFC – UFC / 2019 – Adaptada)** Sobre o desenvolvimento de aplicações móveis, assinale a alternativa correta.

Flutter é um SDK de código aberto criado pelo Google para o desenvolvimento de aplicativos para dispositivos móveis utilizado para desenvolver aplicativos para Android e iOS.

GABARITO

Android

- | | | |
|------------|-------------|-------------|
| 1. Letra B | 9. Letra D | 18. Errado |
| 2. Letra C | 10. Errado | 19. Letra A |
| 3. Letra B | 11. Correto | 20. Correto |
| 4. Letra E | 12. Correto | 21. Letra A |
| 5. Letra D | 13. Errado | 22. Letra D |
| 6. Letra C | 14. Letra A | 23. Letra D |
| 7. Letra B | 15. Letra D | 24. Letra B |
| 8. Letra B | 16. Letra D | |
| | 17. Letra E | |

KOTLIN

Conceitos Básicos



Kotlin é uma linguagem de programação de código aberto que foi desenvolvida para ser compatível com a programação orientada a objetos, funcional, desenvolvida e mantida pela [JetBrains](#). Ela oferece uma sintaxe moderna e concisa, combinando conceitos e recursos de outras linguagens populares, como C#, Java e Scala.

De acordo com os desenvolvedores, Kotlin é uma linguagem de programação moderna, porém já madura, projetada para tornar os desenvolvedores mais felizes. 😊 Ela é concisa, segura, interoperável com Java e outras linguagens, e oferece diversas maneiras de reutilizar código entre múltiplas plataformas para programação produtiva.

Uma das principais características do Kotlin é sua capacidade de interoperabilidade com o Java. Isso significa que é possível utilizar bibliotecas e frameworks Java existentes em projetos escritos em Kotlin, bem como chamar código Kotlin a partir de código Java. Isso facilita a adoção gradual do Kotlin em projetos já existentes e permite que desenvolvedores aproveitem os benefícios da linguagem sem a necessidade de reescrever todo o código.

Além disso, o Kotlin oferece recursos poderosos, como a inferência de tipos, que permite ao compilador deduzir automaticamente o tipo de uma variável com base no contexto. Isso reduz a necessidade de declarações de tipo explícitas e torna o código mais conciso e legível.

Outro recurso interessante do Kotlin é a capacidade de escrever código mais seguro em relação a erros comuns, como a ocorrência de valores nulos. O Kotlin introduziu o conceito de tipos anuláveis e não anuláveis, o que ajuda a evitar erros de `NullPointerException` em tempo de compilação. Isso é alcançado por meio do uso do operador de segurança de chamada `?.` e do operador de coalescência nula `?:`.

Além disso, o Kotlin oferece suporte a recursos avançados da programação funcional, como funções de ordem superior, [lambdas](#) e expressões de função. Esses recursos permitem escrever código mais conciso e expressivo, além de facilitar a implementação de padrões de projeto como [Programação Orientada a Objetos](#) e Programação Funcional.

A tecnologia [Kotlin Multiplatform](#) foi projetada para simplificar o desenvolvimento de projetos multiplataforma. Ela reduz o tempo gasto escrevendo e mantendo o mesmo código para diferentes plataformas, ao mesmo tempo em que mantém a flexibilidade e os benefícios da programação nativa. O desenvolvimento móvel Android tem priorizado o Kotlin desde a conferência Google I/O em 2019.

Mais de 50% dos desenvolvedores profissionais de Android utilizam Kotlin como sua linguagem principal, enquanto apenas 30% usam Java como sua principal linguagem. 70% dos desenvolvedores que têm Kotlin como sua linguagem principal afirmam que o Kotlin os torna mais produtivos.

Ao utilizar Kotlin para o desenvolvimento Android, você pode se beneficiar de:

- Menos código combinado com maior legibilidade. Gaste menos tempo escrevendo seu código e compreendendo o código de outros desenvolvedores.
- Menos erros comuns. Aplicativos construídos com Kotlin têm 20% menos chances de apresentar falhas, de acordo com dados internos do Google.
- Suporte ao Kotlin em bibliotecas Jetpack. O Jetpack Compose é o kit de ferramentas moderno recomendado pelo Android para construir interfaces de usuário nativas em Kotlin. As extensões KTX adicionam recursos da linguagem Kotlin, como coroutines, funções de extensão, lambdas e parâmetros nomeados, às bibliotecas Android existentes.
- Suporte para desenvolvimento multiplataforma. O Kotlin Multiplatform permite o desenvolvimento não apenas para Android, mas também para iOS, backend e aplicações web. Algumas bibliotecas Jetpack já são multiplataforma. O Compose Multiplatform, o framework declarativo de interface do usuário da [JetBrains](#) baseado em Kotlin e Jetpack Compose, torna possível compartilhar interfaces de usuário entre plataformas - iOS, Android, desktop e web.
- Linguagem e ambiente maduros. Desde sua criação em 2011, o Kotlin tem se desenvolvido continuamente, não apenas como uma linguagem, mas como um ecossistema completo com ferramentas robustas. Agora, está integrado perfeitamente ao Android Studio e é amplamente utilizado por várias empresas no desenvolvimento de aplicativos Android.

- Interoperabilidade com Java. Você pode usar Kotlin juntamente com a linguagem de programação Java em seus aplicativos sem precisar migrar todo o seu código para Kotlin.
- Fácil aprendizado. O Kotlin é muito fácil de aprender, especialmente para desenvolvedores Java.

Lembram que eu disse que Kotlin oferece uma **ótima interoperabilidade com Java**? Isso significa que você pode utilizar funções escritas em Kotlin a partir de código Java e vice-versa. Essa compatibilidade entre as duas linguagens **é um dos principais pontos fortes do Kotlin**! Imagine que você está desenvolvendo um aplicativo em Java, mas gostaria de aproveitar alguns recursos avançados do Kotlin. Com o Kotlin, você pode facilmente adicionar trechos de código Kotlin ao seu projeto Java existente. Isso permite que você migre gradualmente de Java para Kotlin, aproveitando as vantagens da linguagem sem ter que reescrever todo o código.

Além disso, o Kotlin é capaz de ler e escrever dados salvos por aplicativos Java. Isso significa que se você tiver informações armazenadas em um formato específico pelo seu aplicativo Java, o Kotlin será capaz de acessar esses dados sem problemas. Da mesma forma, o **Kotlin também pode salvar e ler seus próprios dados, sem restrições**.

Outro ponto importante é que o Kotlin permite que suas **funções sejam chamadas por código Java**. Isso significa que você pode **utilizar as funcionalidades implementadas em Kotlin diretamente em seu código Java existente**. Da mesma forma, você também pode chamar funções feitas em Java a partir de código Kotlin. Essa interoperabilidade bidirecional facilita a combinação de ambas as linguagens em um mesmo projeto.

Uma vantagem interessante do Kotlin é a flexibilidade que ele oferece na construção de aplicativos. Você pode utilizar **Kotlin e Java juntos**, combinando trechos de código de ambas as linguagens de forma natural. Não há restrições quanto à quantidade de Kotlin que você pode usar em seu projeto. Você pode escolher utilizar apenas algumas partes do código em Kotlin ou até mesmo migrar completamente para o Kotlin, dependendo das suas necessidades e preferências. Isso permite que você aproveite as funcionalidades de ambas as linguagens, chamando funções de um lado para o outro e trabalhando com dados armazenados por aplicativos Java. A flexibilidade do Kotlin permite que você construa aplicativos combinando código escrito em Kotlin e Java, sem restrições. Com o Kotlin, você pode migrar gradualmente de Java para Kotlin ou utilizar ambas as linguagens em conjunto, explorando o melhor de cada uma delas. 😊

Native

Kotlin/Native é uma tecnologia para compilar código Kotlin em binários nativos que podem ser executados sem uma máquina virtual. O Kotlin/Native inclui um backend baseado em LLVM para o compilador Kotlin e uma implementação nativa da biblioteca padrão do Kotlin.

Mas Paolla, por que usar o Kotlin/Native? De acordo com a documentação, o Kotlin/Native foi projetado principalmente para permitir **a compilação em plataformas em que máquinas virtuais não são desejáveis ou possíveis**, como dispositivos embarcados ou iOS. É ideal para situações em que um desenvolvedor precisa produzir um programa autossuficiente que não requer uma máquina virtual ou tempo de execução adicional.

Plataformas de destino O Kotlin/Native suporta as seguintes plataformas:

- macOS
- iOS, tvOS, watchOS
- Linux
- Windows (MinGW)
- Android NDK

O Kotlin/Native suporta interoperabilidade bidirecional com linguagens de programação nativas para diferentes sistemas operacionais. O compilador cria:

- um executável para várias plataformas
- uma biblioteca estática ou dinâmica com cabeçalhos C para projetos C/C++
- um framework Apple para projetos Swift e Objective-C

O Kotlin/Native suporta interoperabilidade para usar bibliotecas existentes diretamente do Kotlin/Native:

- bibliotecas C estáticas ou dinâmicas
- frameworks C, Swift e Objective-C

É fácil incluir código Kotlin compilado em projetos existentes escritos em C, C++, Swift, Objective-C e outras linguagens. Também é fácil usar código nativo existente, bibliotecas C estáticas ou dinâmicas, frameworks Swift/Objective-C, motores gráficos e qualquer outra coisa diretamente do Kotlin/Native.

As bibliotecas do Kotlin/Native ajudam a compartilhar código Kotlin entre projetos. Bibliotecas populares como POSIX, gzip, OpenGL, Metal, Foundation e muitas outras bibliotecas e frameworks da Apple já estão pré-importadas e incluídas como bibliotecas do Kotlin/Native no pacote do compilador.

Pessoal, vamos direto ao ponto, o que precisamos saber para responder questões? a Sintaxe básica, os comandos básicos, e a essência dessa linguagem. Então vamos lá, sem perder tempo ver as diferenças dessa linguagem.

Sintaxe básica

Definição de pacote e importações

A especificação do pacote deve estar no topo do arquivo de origem.

```
package my.demo

import kotlin.text.*
```

```
// ...
```

Não é necessário que os diretórios correspondam aos pacotes: os arquivos de origem podem ser colocados arbitrariamente no sistema de arquivos.

O ponto de entrada do programa de um aplicativo Kotlin é a função main.

```
fun main() {  
    println("Olá mundo!")  
}
```

Outra forma de main aceita um número variável de argumentos String.

```
fun main(args: Array<String>) {  
    println(args.contentToString())  
}
```

Impressão na saída padrão

```
print imprime seu argumento na saída padrão.  
  
print("Olá ")  
print("mundo!")
```

`println` imprime seus argumentos e adiciona uma quebra de linha, para que a próxima coisa que você imprima apareça na próxima linha.

```
println("Hello world!")  
println(42)
```

Funções

Uma função com dois parâmetros Int e tipo de retorno Int.

```
fun sum(a: Int, b: Int): Int {  
    return a + b  
}
```

O corpo de uma função pode ser uma expressão. Seu tipo de retorno é inferido.

```
fun sum(a: Int, b: Int) = a + b
```

Uma função que não retorna um valor significativo.

```
fun printSum(a: Int, b: Int): Unit {  
    println("soma de $a e $b é ${a + b}")  
}
```

```
}
```

O tipo de retorno Unit pode ser omitido.

```
fun printSum(a: Int, b: Int) {  
    println("soma de $a e $b é ${a + b}")  
}
```

Variáveis

Variáveis locais somente leitura são definidas usando a palavra-chave **val**. Elas podem receber um valor apenas uma vez. Essas variáveis podem receber um valor **somente uma vez**, e após a atribuição inicial, **não podem mais ser modificadas**. Por exemplo, ao definir a constante "a" com um valor fixo, utiliza-se a instrução "val" para indicar que essa variável é imutável. Isso significa que não é possível atribuir um novo valor a ela após a sua inicialização. Essa abordagem permite criar constantes que garantem a integridade dos dados e promovem uma programação mais segura, evitando alterações acidentais ou indesejadas nos valores.

```
val a: Int = 1 // atribuição imediata  
val b = 2 // o tipo Int é inferido  
val c: Int // Tipo necessário quando nenhum inicializador é fornecido  
c = 3 // atribuição posterior
```

Variáveis que podem ser reatribuídas usam a palavra-chave var. Ao contrário das variáveis somente leitura (val), as variáveis var podem ter seu valor modificado após a sua inicialização. Isso significa que é possível atribuir um novo valor a uma variável var durante a execução do programa. Por exemplo, podemos declarar uma variável var chamada "x" e atribuir um valor inicial a ela. Em seguida, podemos alterar seu valor conforme necessário durante a execução do código.

```
var x = 5 // o tipo Int é inferido  
x += 1
```

Você pode declarar variáveis no nível superior.

```
val PI = 3.14  
var x = 0  
  
fun incrementX() {  
    x += 1  
}
```

(Cesgranrio – Caixa / 2021) Na linguagem de programação Kotlin, é possível criar uma variável cujo valor nunca pode ser mudado, na prática, uma constante, com o nome idademinima, do tipo básico inteiro de 32 bits, com o valor 18.

Para que isso aconteça, qual das seguintes instruções deve ser usada?

- a) val idademinima : Int = 18
- b) val idademinima : Integer = 18
- c) val idademinima = 18 : Integer
- d) var idademinima : Int = 18
- e) var idademinima : Integer = 18

Comentários:

Na linguagem de programação Kotlin, para criar uma variável cujo valor nunca pode ser alterado, ou seja, uma constante, deve-se utilizar a instrução "val". Para definir a constante "idademinima", que possui um valor fixo e não pode ser modificado, deve-se utilizar a instrução "val". Essa instrução indica que a variável é imutável e não é possível atribuir um novo valor a ela após a sua inicialização. No caso específico da constante "idademinima", que possui o tipo básico inteiro de 32 bits, a opção correta é a alternativa A: "val idademinima: Int = 18". Essa declaração cria uma constante chamada "idademinima" do tipo "Int" e atribui o valor 18 a ela. É importante notar que as alternativas B, C, D e E são incorretas, pois utilizam a instrução "var" em vez de "val". A instrução "var" é utilizada para criar variáveis mutáveis, ou seja, variáveis cujo valor pode ser alterado ao longo do código. Portanto, a resposta correta para criar uma constante imutável em Kotlin é a alternativa A: "val idademinima: Int = 18". (Gabarito: Letra A)

Vamos aprofundar um pouco mais? Agora veremos classes, instâncias, comentários, expressões condicionais como for, while e when. Bora! 🚀

Criando classes e instâncias

Para definir uma classe, use a palavra-chave class.

```
class Shape
```

As propriedades de uma classe podem ser listadas em sua declaração ou corpo.

```
class Rectangle(var height: Double, var length: Double) {  
    var perimeter = (height + length) * 2  
}
```

O construtor padrão com parâmetros listados na declaração da classe está disponível automaticamente.

```
val rectangle = Rectangle(5.0, 2.0)  
println("O perímetro é ${rectangle.perimeter}")
```

A herança entre classes é declarada por dois pontos (:). As classes são finais por padrão; para tornar uma classe herdável, marque-a como open.

```
open class Shape
```

```
class Rectangle(var height: Double, var length: Double): Shape() {  
    var perimeter = (height + length) * 2  
}
```

Comentários

Assim como a maioria das linguagens modernas, o Kotlin suporta comentários de linha única (ou no final da linha) e comentários de várias linhas (bloco). Além disso, comentários em bloco no Kotlin podem ser aninhados.

```
// Este é um comentário no final da linha  
  
/* Este é um comentário em bloco  
   em várias linhas. */  
  
/* O comentário começa aqui  
/* contém um comentário aninhado */  
e termina aqui. */
```

Templates de string

Os templates de string em Kotlin são uma forma de inserir valores de variáveis ou expressões dentro de uma string de forma conveniente. Eles são representados pelo símbolo "\$" seguido pelo nome da variável ou da expressão entre chaves "{}".

```
var a = 1  
// Nome simples no template:  
val s1 = "a é $a"  
  
a = 2  
// Expressão arbitrária no template:  
val s2 = "${s1.replace("é", "foi")}, mas agora é $a"
```

No exemplo fornecido, a variável "a" é declarada como sendo igual a 1. Em seguida, é criada uma string chamada "s1" que contém o template "\$a". Isso significa que o valor de "a" será inserido na posição do template dentro da string. Neste caso, "s1" será igual a "a é 1".

Depois, o valor da variável "a" é alterado para 2. Em seguida, é criada outra string chamada "s2" que utiliza um template mais complexo. Dentro desse template, é usada uma expressão para substituir a palavra "é" por "foi" na string "s1", que é referenciada usando "\${s1.replace("é", "foi")}". Além disso, o valor atualizado de "a" é adicionado ao final do template. Portanto, "s2" será igual a "a foi 1, mas agora é 2".

Expressões condicionais

Vamos estudar o uso da instrução "if" em Kotlin, que não apenas é uma instrução, mas também uma expressão que retorna um valor. Isso elimina a necessidade de um operador ternário

separado e torna a lógica condicional mais intuitiva e simplificada. Em seguida, veremos o loop "for", que é utilizado para percorrer coleções de elementos, como listas e arrays, de forma sequencial, simplificando o código e facilitando o acesso aos elementos.

Também exploraremos os loops "while" e "do-while", que executam seu corpo repetidamente enquanto uma condição for satisfeita, com a diferença de que o "do-while" garante a execução do corpo pelo menos uma vez. Além disso, aprenderemos sobre o uso dos operadores "break" e "continue" para controlar o fluxo de execução dentro dos loops. Por fim, abordaremos o comando "when", que é uma expressão condicional com múltiplos ramos, semelhante ao "switch" em outras linguagens. O "when" permite comparar um argumento com cada ramo até que uma condição seja satisfeita, oferecendo maior flexibilidade no tratamento de diferentes casos. Vamos lá! 🚀

If

Vamos relembrar o uso do IF? No Kotlin, a instrução if não é apenas uma instrução, mas também uma expressão que retorna um valor. Isso significa que ela pode ser usada para atribuir um valor ou retornar diretamente um resultado com base em uma condição, eliminando a necessidade de um operador ternário separado, como em algumas outras linguagens de programação. A flexibilidade da expressão if permite um código conciso e legível, tornando a lógica condicional mais intuitiva e simplificada no Kotlin.

```
fun maxOf(a: Int, b: Int): Int {  
    if (a > b) {  
        return a  
    } else {  
        return b  
    }  
}
```

```
fun maxOf(a: Int, b: Int) = if (a > b) a else b // If usado como uma expressão.
```

Loop for

O loop "for" é usado para iterar sobre uma coleção de elementos. Ele permite percorrer os elementos de uma lista, array ou qualquer objeto que seja iterável. A sintaxe do loop "for" é simples: você declara uma variável de iteração e especifica a coleção que será percorrida. Em cada iteração, a variável assume o valor de um elemento da coleção, permitindo que você execute operações ou acesse informações relacionadas a esse elemento.

O loop "for" é especialmente útil quando você precisa percorrer todos os elementos de uma coleção de forma sequencial. Ele simplifica o código, tornando-o mais legível e fácil de entender. No exemplo fornecido, o primeiro loop "for" itera sobre a lista de itens e imprime cada elemento. O segundo loop "for" utiliza o índice da lista para acessar cada elemento e exibe o índice e o valor correspondente.

```
val items = listOf("maçã", "banana", "kiwi")
```

```
for (item in items) {  
    println(item)  
}
```

```
val items = listOf("maçã", "banana", "kiwi")  
for (index in items.indices) {  
    println("o item no índice $index é ${items[index]}")  
}
```

Loop while

Em Kotlin, os loops while e do-while executam seu corpo continuamente enquanto sua condição for satisfeita. A diferença entre eles está no momento em que a condição é verificada:

O while verifica a condição e, se ela for satisfeita, executa o corpo e depois retorna à verificação da condição.

O do-while executa o corpo primeiro e, em seguida, verifica a condição. Se a condição for satisfeita, o loop se repete. Portanto, o corpo do do-while é executado pelo menos uma vez, independentemente da condição.

Em Kotlin, também é possível utilizar os operadores tradicionais break e continue nos loops, permitindo interromper a execução do loop ou pular para a próxima iteração, respectivamente. Esses operadores oferecem controle adicional sobre o fluxo de execução dentro dos loops.

```
val items = listOf("maçã", "banana", "kiwi")  
var index = 0  
while (index < items.size) {  
    println("o item no índice $index é ${items[index]}")  
    index++  
}
```

Expressão when

O when em Kotlin é uma expressão condicional com múltiplos ramos. Ele é semelhante ao comando switch em linguagens como C. O when compara seu argumento com cada ramo sequencialmente até que uma condição de ramo seja satisfeita.

O when pode ser usado tanto como uma expressão quanto como um comando. Se for usado como uma expressão, o valor do primeiro ramo correspondente se torna o valor da expressão como um todo. Se for usado como um comando, os valores dos ramos individuais são ignorados. Assim como acontece com o if, cada ramo pode ser um bloco de código, e seu valor é o valor da última expressão no bloco. O ramo else é avaliado quando nenhuma das outras condições de ramo é satisfeita.

Se o when for usado como uma expressão, o ramo else é obrigatório, a menos que o compilador possa provar que todos os casos possíveis são abrangidos pelas condições dos ramos, por exemplo, em casos de entradas de classes enumeradas e subtipos de classes seladas.

```
fun describe(obj: Any): String =  
    when (obj) {  
        1          -> "Um"  
        "Hello"    -> "Saudação"  
        is Long     -> "Longo"  
        !is String -> "Não é uma string"  
        else       -> "Desconhecido"  
    }
```

(FGV – TJ RO/ 2021) Analise o código Kotlin exibido a seguir.

```
fun main() {  
    val classe = ?  
    val resposta = when (classe) {  
        1 -> "Premium"  
        2 -> "Superior"  
        3 -> "Econômica"  
        4, 5 -> when (classe % 2 == 0)  
            {true -> "Gold" false -> "Iron"}  
        else -> "Dado inválido."  
    }  
    println(resposta)  
}
```

O par que indica corretamente o valor a ser atribuído à variável classe, na segunda linha, e o valor computado e exibido pelo comando println é:

- a) 3 / Iron
- b) 4 / Iron
- c) 5 / Iron
- d) 5 / Gold
- e) 9 / Gold

Comentários:

A combinação válida entre as alternativas é a seguinte: c) 5 / Iron. O algoritmo apresentado é um código escrito na linguagem de programação Kotlin. Vamos entender passo a passo o que esse código faz: O código começa com a função main(), que é a função principal do programa. É a partir dessa função que o programa será executado. Na segunda linha, declara-se uma variável chamada classe com um valor indefinido, indicado pelo ponto de interrogação ?. O valor dessa variável será fornecido posteriormente durante a execução do programa. Na terceira linha, declara-se uma variável chamada resposta e utiliza-se a estrutura when para fazer uma seleção condicional com base no valor da variável classe. Dentro do bloco when, são especificados os possíveis valores de classe e as respostas correspondentes a cada um deles. Se o valor de classe

for igual a 1, a resposta atribuída à variável resposta será "Premium". Se o valor de classe for igual a 2, a resposta atribuída à variável resposta será "Superior". Se o valor de classe for igual a 3, a resposta atribuída à variável resposta será "Econômica". Se o valor de classe for igual a 4 ou 5, o código entra em outro bloco when para fazer uma nova seleção condicional com base no resultado da expressão `classe % 2 == 0`. Essa expressão verifica se o valor de classe é divisível por 2 (ou seja, se é par). Se a expressão `classe % 2 == 0` for avaliada como verdadeira (par), a resposta atribuída à variável resposta será "Gold". Se a expressão `classe % 2 == 0` for avaliada como falsa (ímpar), a resposta atribuída à variável resposta será "Iron". Caso o valor de classe não corresponda a nenhum dos casos anteriores, ou seja, não seja igual a 1, 2, 3, 4 ou 5, a resposta atribuída à variável resposta será "Dados inválidos." Por fim, a resposta contida na variável resposta é impressa no console utilizando o comando `println`. Ao analisar o código Kotlin fornecido, podemos identificar que o valor a ser atribuído à variável "classe" é 5, conforme indicado na segunda linha do código. Em seguida, o código utiliza a estrutura "when" para avaliar o valor da variável "classe" e determinar o valor da variável "resposta". Assim, quando o valor de "classe" é igual a 5, o código entra no bloco correspondente e avalia a expressão `classe % 2 == 0`. No entanto, o resto da divisão de 5 por 2 é igual a 1, o que não atende à condição de ser igual a 0. Portanto, o código segue para o bloco alternativo e atribui o valor "Iron" à variável "resposta". Finalmente, o valor de "resposta" é exibido utilizando o comando `println`, resultando na impressão de "5/Iron" como resposta final. (Gabarito: Letra C)

Coleções

A Biblioteca Padrão do Kotlin oferece **um conjunto abrangente de ferramentas para gerenciar coleções** - grupos de um número variável de itens (possivelmente zero) que são significativos para o problema sendo resolvido e que são comumente operados. As coleções são um conceito comum na maioria das linguagens de programação, então se você está familiarizado, por exemplo, com as coleções em Java ou Python, pode pular esta introdução e prosseguir para as seções detalhadas.

Uma coleção geralmente contém um número de objetos (esse número também pode ser zero) do mesmo tipo. Os objetos em uma coleção são chamados de **elementos ou itens**. Por exemplo, todos os estudantes em um departamento formam uma coleção que pode ser usada para calcular a idade média deles.

Os seguintes **tipos de coleção** são relevantes para o Kotlin:

- A **List** é uma coleção ordenada com acesso aos elementos por meio de índices - números inteiros que refletem sua posição. Os elementos podem ocorrer mais de uma vez em uma lista. Um exemplo de lista é um número de telefone: é um grupo de dígitos, sua ordem é importante e eles podem se repetir.
- O **Set** é uma coleção de elementos únicos. Ele reflete a abstração matemática de um conjunto: um grupo de objetos sem repetições. Geralmente, a ordem dos elementos de um conjunto não tem importância. Por exemplo, os números em bilhetes de loteria formam um conjunto: eles são únicos e sua ordem não é importante.
- O **Map (ou dicionário)** é um conjunto de pares chave-valor. As chaves são únicas e cada uma delas está associada a exatamente um valor. Os valores podem ser duplicados. Os

Maps são úteis para armazenar conexões lógicas entre objetos, por exemplo, o ID de um funcionário e sua posição.

O Kotlin permite manipular coleções independentemente do tipo exato de objetos armazenados nelas. Em outras palavras, você pode adicionar uma String a uma lista de Strings da mesma maneira que faria com Ints ou uma classe definida pelo usuário. Portanto, a Biblioteca Padrão do Kotlin oferece interfaces genéricas, classes e funções para criar, preencher e gerenciar coleções de qualquer tipo.

Tipos de coleção

A Biblioteca Padrão do Kotlin fornece implementações para tipos básicos de coleção: sets, lists e maps. Um par de interfaces representa cada tipo de coleção:

Uma interface somente leitura que fornece operações para acessar os elementos da coleção. Uma interface mutável que estende a interface somente leitura correspondente com operações de escrita: adicionar, remover e atualizar seus elementos.

Observe que alterar uma coleção mutável não requer que ela seja uma variável mutável: as operações de escrita modificam o mesmo objeto de coleção mutável, então a referência não muda. No entanto, se você tentar reatribuir uma coleção val, ocorrerá um erro de compilação.

Collection

A `Collection<T>` é a raiz da hierarquia de coleções. Essa interface representa o comportamento comum de uma coleção somente leitura: recuperar o tamanho, verificar a pertinência de um item, entre outros. A `Collection` herda da interface `Iterable<T>`, que define as operações para iterar sobre os elementos. Você pode usar a `Collection` como parâmetro de uma função que se aplica a diferentes tipos de coleções.

```
val numbers: Collection<Int> = listOf(1, 2, 3, 4, 5)
println(numbers.size) // Output: 5
println(numbers.contains(3)) // Output: true
```

Frisando um detalhe importante! Ambas as formas estão corretas em Kotlin. `val numbers: Collection<Int> = listOf(1, 2, 3, 4, 5)` e `val numbers = listOf(1, 2, 3, 4, 5)`. A diferença está no tipo declarado para a variável `numbers`.

Na primeira opção, temos a declaração explícita do tipo da variável `numbers` como `Collection<Int>`, ou seja, uma coleção de elementos do tipo `Int`. Isso significa que, por ter o tipo `Collection<Int>`, a variável `numbers` terá apenas as operações e funcionalidades disponíveis para o tipo `Collection`, como acessar elementos por meio de iterações e realizar operações comuns a coleções.

Na segunda opção, não há a declaração explícita do tipo da variável `numbers`. Nesse caso, o compilador do Kotlin infere o tipo com base no valor atribuído a `numbers`, que é `listOf(1, 2, 3, 4, 5)`. O compilador deduz que `numbers` é do tipo `List<Int>`, uma lista de elementos do tipo `Int`. Assim, a variável `numbers` terá as operações e funcionalidades disponíveis para o tipo `List`, que

incluem as funcionalidades de uma coleção, bem como operações específicas de listas, como acessar elementos por índices e realizar operações de adição, remoção e modificação de elementos.

Portanto, ambas as formas são válidas, e a escolha entre elas depende do contexto e da necessidade específica do código. Se você precisar trabalhar com as operações e funcionalidades específicas de uma lista, a segunda opção (`val numbers = listOf(1, 2, 3, 4, 5)`) é mais adequada. Caso precise apenas das funcionalidades básicas de uma coleção, a primeira opção (`val numbers: Collection<Int> = listOf(1, 2, 3, 4, 5)`) pode ser utilizada.

```
fun printAll(strings: Collection<String>) {  
    for(s in strings) print("$s ")  
    println()  
}  
  
fun main() {  
    val stringList = listOf("one", "two", "one")  
    printAll(stringList)  
  
    val stringSet = setOf("one", "two", "three")  
    printAll(stringSet)  
}
```

O exemplo em Kotlin consiste em duas funções: `printAll` e `main`. A função `printAll` recebe como parâmetro uma coleção de strings (`Collection<String>`) chamada `strings`. Essa função itera sobre os elementos da coleção usando um loop `for`, onde cada elemento é armazenado na variável `s`. Em seguida, o valor de `s` é impresso seguido de um espaço em branco. Após iterar todos os elementos da coleção, é chamado `println()` para imprimir uma nova linha, resultando em uma saída formatada de todos os elementos da coleção separados por espaços.

Na função `main`, duas coleções são criadas e passadas como argumentos para a função `printAll`. Primeiro, é criada a lista de strings `stringList` contendo os elementos "one", "two" e "one". Em seguida, a função `printAll` é chamada com `stringList` como argumento, resultando na impressão de "one two one" no console.

Depois, é criado o conjunto de strings `stringSet` contendo os elementos "one", "two" e "three". Novamente, a função `printAll` é chamada com `stringSet` como argumento, resultando na impressão de "one two three" no console.

List

A `List<T>` armazena elementos em uma ordem especificada e fornece acesso indexado a eles. Os índices começam do zero - o índice do primeiro elemento - e vão até `lastIndex`, que é (`list.size - 1`).

Elementos da lista (incluindo nulos) podem ser duplicados: uma lista pode conter qualquer número de objetos iguais ou ocorrências de um único objeto. Duas listas são consideradas iguais se tiverem os mesmos tamanhos e elementos estruturalmente iguais nas mesmas posições.

```
val numbers = listOf("one", "two", "three", "four")
println("Number of elements: ${numbers.size}")
println("Third element: ${numbers.get(2)}")
println("Fourth element: ${numbers[3]}")
println("Index of element \"two\" ${numbers.indexOf("two")}")
```

Neste exemplo em Kotlin, uma lista de strings chamada numbers é criada com os elementos "one", "two", "three" e "four".

A primeira linha de código imprime o número de elementos da lista usando a propriedade size da lista. Ela utiliza uma interpolação de string para inserir o valor retornado dentro da mensagem a ser impressa.

A segunda linha de código imprime o terceiro elemento da lista usando o método get(index), onde index é a posição do elemento desejado. Neste caso, o índice 2 é fornecido para obter o terceiro elemento. Mais uma vez, é utilizada uma interpolação de string para inserir o valor retornado na mensagem a ser impressa.

A terceira linha de código imprime o quarto elemento da lista usando a notação de acesso direto []. Neste caso, o índice 3 é usado para obter o quarto elemento da lista. A interpolação de string é usada para incluir o valor retornado na mensagem.

A última linha de código imprime o índice do elemento "two" na lista usando o método indexOf(element). O elemento "two" é fornecido como argumento para encontrar seu índice na lista. Novamente, uma interpolação de string é usada para incluir o valor retornado na mensagem impressa.

O exemplo demonstra como acessar elementos específicos em uma lista por meio de índices e também como obter o tamanho da lista e encontrar o índice de um elemento específico. Isso ilustra algumas das operações básicas disponíveis para manipular listas em Kotlin.

Set

Um Set<T> armazena elementos únicos, e sua ordem geralmente é indefinida. Elementos nulos também são considerados únicos: um Set pode conter apenas um nulo. Dois conjuntos são iguais se tiverem o mesmo tamanho e, para cada elemento de um conjunto, houver um elemento igual no outro conjunto.

A implementação padrão de MutableSet - LinkedHashSet - preserva a ordem de inserção dos elementos. Portanto, as funções que dependem da ordem, como first() ou last(), retornam resultados previsíveis nesses conjuntos.

```
val numbers = setOf(1, 2, 3, 4)
println("Number of elements: ${numbers.size}")
if (numbers.contains(1)) println("1 is in the set")

val numbersBackwards = setOf(4, 3, 2, 1)
println("The sets are equal: ${numbers == numbersBackwards}")
```

Mapa:

Map<K, V> não é uma herança da interface Collection; no entanto, também é um tipo de coleção no Kotlin. Um Mapa armazena pares de chave-valor (ou entradas); as chaves são únicas, mas chaves diferentes podem ser associadas a valores iguais. A interface Map fornece funções específicas, como acesso ao valor pela chave, busca por chaves e valores, entre outras. Vale ressaltar que, em um mapa, a ordem dos elementos pode não ser preservada, pois ela é baseada nas operações de armazenamento e recuperação dos pares de chave-valor.

```
val numbersMap = mapOf("chave1" to 1, "chave2" to 2, "chave3" to 3, "chave4" to 1)

println("Todas as chaves: ${numbersMap.keys}")
println("Todos os valores: ${numbersMap.values}")
if ("chave2" in numbersMap) println("Valor da chave \"chave2\": ${numbersMap["chave2"]}")
if (1 in numbersMap.values) println("O valor 1 está no mapa")
if (numbersMap.containsValue(1)) println("O valor 1 está no mapa") // mesmo que o anterior
```

No exemplo fornecido, criamos um mapa chamado `numbersMap` que contém pares de chave-valor. As chaves são strings ("chave1", "chave2", "chave3", "chave4") e os valores são inteiros (1, 2, 3, 1).

Em seguida, imprimimos todas as chaves do mapa usando `numbersMap.keys`, que retorna uma coleção contendo todas as chaves presentes no mapa. Da mesma forma, imprimimos todos os valores usando `numbersMap.values`, que retorna uma coleção contendo todos os valores do mapa.

Em seguida, verificamos se a chave "chave2" está presente no mapa usando a expressão "chave2" in numbersMap. Se a chave estiver presente, imprimimos o valor correspondente à chave usando `numbersMap["chave2"]`.

Depois, verificamos se o valor 1 está presente nos valores do mapa usando `1 in numbersMap.values`. Se o valor estiver presente, imprimimos a mensagem "O valor 1 está no mapa".

Por fim, utilizamos a função `numbersMap.containsValue(1)` para verificar se o valor 1 está presente no mapa. Se estiver presente, imprimimos a mensagem "O valor 1 está no mapa". Essa verificação é equivalente à verificação anterior usando `in numbersMap.values`.

Intervalos e Progressões

O Kotlin oferece recursos para criar intervalos e progressões de valores por meio do pacote `kotlin.ranges` e suas funções correspondentes. Vamos explorar essas funcionalidades em detalhes:

Intervalos:

Um intervalo em Kotlin é definido por dois valores de extremidade, que estão incluídos no intervalo. Esses intervalos são criados usando a função `rangeTo()` do pacote `kotlin.ranges` ou seu operador `..`. Normalmente, `rangeTo()` é complementado pelas funções `in` ou `!in` para verificar se um valor está dentro do intervalo. Por exemplo:

```
if (i in 1..4) {  
    print(i)  
}
```

Progressões:

As progressões em Kotlin são usadas para iterar sobre intervalos de valores numéricos. Os tipos de dados inteiros (`IntRange`, `LongRange`, `CharRange`) têm a capacidade adicional de serem iterados. Para iterar em ordem reversa, utiliza-se a função `downTo`. Além disso, é possível especificar um passo arbitrário para a iteração usando a função `step`. Por exemplo:

```
for (i in 1..8 step 2) print(i)  
println()  
for (i in 8 downTo 1 step 2) print(i)
```

Intervalos abertos:

Caso o último elemento não deva ser incluído no intervalo, pode-se usar a função `until`. Por exemplo:

```
for (i in 1 until 10) {  
    print(i)  
}
```

Intervalo personalizado:

Em Kotlin, é possível criar intervalos personalizados para suas próprias classes, chamando a função `rangeTo()` no valor de início do intervalo e fornecendo o valor final como argumento. Por exemplo:

```
val versionRange = Version(1, 11)..Version(1, 30)  
println(Version(0, 9) in versionRange)  
println(Version(1, 20) in versionRange)
```

Progressões personalizadas:

As progressões são tratadas como sequências aritméticas para tipos de dados inteiros em Kotlin. Existem três tipos principais para as progressões: `IntProgression`, `LongProgression` e `CharProgression`. Essas progressões possuem propriedades essenciais, como o primeiro elemento, o último elemento e o passo. É possível definir um passo personalizado usando a função `step`. Por exemplo:

```
for (i in 1..8 step 2) print(i)
```

Kotlin - Java

A equivalência entre Kotlin e Java é um conceito importante para entendermos como as duas linguagens se relacionam e como podemos converter código de uma para a outra. Kotlin é uma linguagem de programação moderna que foi desenvolvida pela JetBrains e adotada pelo Google como uma linguagem oficial para o desenvolvimento de aplicativos Android. Por outro lado, Java é uma linguagem de programação mais antiga e amplamente utilizada em uma variedade de domínios.

Ambas as linguagens possuem semelhanças em termos de sintaxe e estruturas básicas, mas também possuem algumas diferenças. No entanto, é possível converter código de uma linguagem para a outra de forma relativamente direta.

Vamos a um exemplo prático para ajudá-los a entender como é uma classe Java e uma classe Kotlin!

```
public class Classe_Java {  
    private String aula;  
    private String professor;  
    private int d = 0;  
    private String curso = "Estrategia";  
  
    public Classe_Java(String aula, String professor) {  
        this.aula = aula;  
        this.professor = professor;  
    }  
}
```

Em Kotlin:

```
class Classe_Kotlin(private val aula: String, private val professor: String) {  
    private var d = 0  
    private var curso = "Estrategia"  
}
```

Nesse exemplo, a classe Java chamada Classe_Java foi convertida para Kotlin como Classe_Kotlin. Veja as diferenças:

- O modificador de acesso "public" da classe Java não foi especificado em Kotlin, pois o modificador de acesso padrão em Kotlin é mais restrito do que "public". Portanto, a classe Kotlin só pode ser acessada dentro do mesmo módulo.
- Os atributos privados da classe Java foram convertidos para atributos privados em Kotlin, seguindo a mesma sintaxe.
- Os atributos d e curso em Kotlin não possuem um valor padrão definido como em Java. Em Kotlin, eles são inicializados implicitamente como 0 e "Estrategia", respectivamente.

- A classe Kotlin `Classe_Kotlin` não possui um construtor primário declarado explicitamente. No entanto, ela recebe dois parâmetros (aula e professor) no construtor primário, que são automaticamente atribuídos aos atributos correspondentes. Em Kotlin, essa é uma conveniência oferecida pela linguagem, permitindo a declaração direta dos atributos no construtor primário.

É importante mencionar que a conversão de código de uma linguagem para outra nem sempre é automática, especialmente quando se trata de recursos mais avançados ou bibliotecas específicas. No entanto, a maioria dos conceitos e estruturas básicas podem ser convertidos com facilidade.

A equivalência entre Kotlin e Java permite que os desenvolvedores aproveitem os recursos de ambas as linguagens, facilitando a migração gradual de projetos existentes ou a combinação de código Kotlin e Java em um único projeto.

ELEMENTOS-CHAVE DA PROGRAMAÇÃO ORIENTADA A OBJETOS E CONTROLE DE FLUXO	JAVA	KOTLIN
CLASSE	<pre>public class Pessoa { private String nome; private int idade; public Pessoa(String nome, int idade) { this.nome = nome; this.idade = idade; } public void apresentar() { System.out.println("Olá, meu nome é " + nome + " e tenho " + idade + " anos."); } }</pre>	<pre>class Pessoa(private val nome: String, private var idade: Int) { fun apresentar() { println("Olá, meu nome é \$nome e tenho \$idade anos.") } }</pre>
INSTÂNCIAS	<pre>Pessoa pessoa = new Pessoa("João", 25); pessoa.apresentar();</pre>	<pre>val pessoa = Pessoa("João", 25) pessoa.apresentar()</pre>

FUNÇÕES	<pre>public class Calculadora { public static int somar(int a, int b) { return a + b; } public static void main(String[] args) { int resultado = somar(3, 5); System.out.println("O resultado da soma é: " + resultado); } }</pre>	<pre>object Calculadora { fun somar(a: Int, b: Int): Int { return a + b } } fun main() { val resultado = Calculadora.somar(3, 5) println("O resultado da soma é: \$resultado") }</pre>
LOOP FOR	<pre>for (int i = 1; i <= 5; i++) { System.out.println("Número: " + i); }</pre>	<pre>for (i in 1..5) { println("Número: \$i") }</pre>
LOOP WHILE	<pre>int i = 1; while (i <= 5) { System.out.println("Número: " + i); i++; }</pre>	<pre>var i = 1 while (i <= 5) { println("Número: \$i") i++ }</pre>
LOOP WHEN	<pre>int day = 3; switch (day) { case 1: System.out.println("Segunda-feira"); break; case 2:</pre>	<pre>val day = 3 when (day) { 1 -> println("Segunda-feira") 2 -> println("Terça-feira") 3 -> println("Quarta-feira") else -> println("Outro dia da semana") }</pre>

	<pre> System.out.println("Terça-feira"); break; case 3: System.out.println("Quarta-feira"); break; default: System.out.println("Outro dia da semana"); break; } </pre>	}
--	---	---

Essa tabela mostra uma comparação dos tipos de dados e algumas palavras-chave usadas em Java e Kotlin. Enquanto muitos tipos de dados têm nomes semelhantes nas duas linguagens, algumas diferenças existem. Além disso, algumas palavras-chave específicas do Java não têm um equivalente direto em Kotlin.

JAVA	KOTLIN
BYTE	Byte
SHORT	Short
INT	Int
LONG	Long
FLOAT	Float
DOUBLE	Double
CHAR	Char
BOOLEAN	Boolean
VOID	Unit
OBJECT	Any
STRING	String

ARRAY	Array
ENUM	enum class
INTERFACE	interface
CLASS	class
ANNOTATION	annotation
WRAPPER	N/A (não há)
BYTE	Byte

REFERÊNCIAS

Kotlin for Android Developers

Antonio Leiva. Kotlin for Android Developers: Learn Kotlin the Easy Way While Developing an Android App. 2ª Edição, 2018.

Kotlin Language Documentation on Wikipedia

Wikipedia. Kotlin (programming language). Disponível em: <https://pt.wikipedia.org/wiki/Kotlin>

Kotlin by Example

JetBrains. Kotlin by Example. Disponível em: <https://kotlinlang.org/docs/examples.html>

Kotlin in Action

Dmitry Jemerov, Svetlana Isakova. Kotlin in Action. 2017.

Android Manifest Documentation

Google. *Manifesto do Android: Introdução ao Manifesto*. Disponível em: <https://developer.android.com/guide/topics/manifest/manifest-intro?hl=pt-br>

Temok Blog

Temok. *Blog de Hospedagem e Marketing Online*. Disponível em: <https://www.temok.com/blog/>

Android Enterprise Documentation

Google. *Documentação Android Enterprise*. Disponível em: <https://www.androidenterprise.dev/s/>

Android Studio Publishing

Google. *Publicação no Android Studio*. Disponível em: <https://developer.android.com/studio/publish?hl=pt-br>

Apple Developer Documentation

Apple. *Documentação do Apple Developer*. Disponível em: <https://developer.apple.com/documentation/>

Flutter Official Website

Flutter. *Flutter: Beautiful native apps in record time*. Disponível em: <https://flutter.dev>

Flutter Wikipedia

Wikipedia. *Flutter*. Disponível em: <https://pt.wikipedia.org/wiki/Flutter>

Flutter API Documentation

Flutter. *Documentação da API Flutter*. Disponível em: <https://api.flutter.dev>

Swift Documentation

Apple. *The Swift Programming Language Documentation*. Disponível em: <https://docs.swift.org/swift-book/documentation/the-swift-programming-language/>

Flutter Docs

Flutter. *Flutter Documentation*. Disponível em: <https://docs.flutter.dev>

QUESTÕES COMENTADAS

Kotlin

1. (CESGRANRIO – Banco do Brasil/ 2023) Kotlin é uma linguagem de programação usada no desenvolvimento Android.

Entre suas características, está um grau de compatibilidade com Java, que permite

- a) chamar funções feitas em Java, apenas, mas não permite que suas funções Kotlin sejam chamadas por Java.
- b) ler dados que foram salvos por apps Java, apenas.
- c) ler e escrever dados que podem ser lidos e escritos por apps Java, apenas.
- d) ter suas funções chamadas por Java, apenas, mas não consegue chamar funções feitas em Java.
- e) construir apps com código parcialmente em Java e parcialmente em Kotlin, sem restrições.

Comentários:

Muito interessante a questão! Vamos analisar cada item!

- a) Chamar funções feitas em Java, apenas, mas não permite que suas funções Kotlin sejam chamadas por Java.

Essa afirmação está incorreta. Kotlin e Java são interoperáveis, o que significa que é possível chamar funções escritas em Kotlin a partir de código Java e vice-versa. A compatibilidade entre as duas linguagens é um dos principais benefícios do Kotlin, permitindo que desenvolvedores migrem gradualmente de Java para Kotlin ou combinem ambas em um mesmo projeto.

- b) Ler dados que foram salvos por apps Java, apenas.

Essa afirmação também está incorreta. O Kotlin é capaz de ler dados salvos por aplicativos Java, bem como salvar e ler seus próprios dados. Não há restrições em relação à leitura de dados entre as duas linguagens.

- c) Ler e escrever dados que podem ser lidos e escritos por apps Java, apenas.

Essa afirmação também está incorreta. Assim como mencionado anteriormente, o Kotlin é capaz de ler e escrever dados que foram salvos por aplicativos Java, além de lidar com seus próprios dados. Não há restrições nesse aspecto.

- d) Ter suas funções chamadas por Java, apenas, mas não consegue chamar funções feitas em Java.

Essa afirmação está incorreta. O Kotlin permite que suas funções sejam chamadas por código Java, assim como também permite que funções feitas em Java sejam chamadas por código Kotlin. A interoperabilidade entre as duas linguagens permite que o desenvolvedor trabalhe com ambas sem restrições.

e) Construir apps com código parcialmente em Java e parcialmente em Kotlin, sem restrições.

Essa afirmação está correta. O Kotlin oferece a flexibilidade de construir aplicativos combinando código escrito em Java e Kotlin sem restrições. É possível utilizar qualquer quantidade de Kotlin no projeto, combinando-o com código Java de forma natural e aproveitando a interoperabilidade entre as duas linguagens.

Gabarito: Letra E

2. (FCM – CEFETMINAS / 2022) Com relação ao desenvolvimento de aplicativos móveis, relacione as linguagens de programação com suas características.

LINGUAGEM DE PROGRAMAÇÃO

- 1 - Java
- 2 - JavaScript
- 3 - Kotlin
- 4 - TypeScript
- 5 - Objective-C

CARACTERÍSTICAS

() É um superconjunto da linguagem de programação C, ou seja, agrega recursos ao C. Ele possibilita o uso do paradigma programação orientada a objetos, contendo sintaxe para a criação de métodos e classes.

() É uma linguagem mais recente e que tem ganhado força nos últimos tempos. É desenvolvida e mantida pela JetBrains. Utiliza o paradigma orientado a objetos e tem suporte ao paradigma funcional, com o uso de expressões lambda (anônimas).

() É uma linguagem de programação usada principalmente para controlar o Hypertext Markup Language (HTML) e o Cascading Style Sheets (CSS) e manipular comportamentos em uma página web. É mantido pela European Computer Manufacturer's Association (ECMA).

Originalmente, foi criada para o desenvolvimento de aplicações no lado cliente, mas evoluiu para possibilitar o desenvolvimento de aplicações desktop e no lado servidor.

() Mantida pela Oracle, é composta por uma linguagem de programação e uma plataforma computacional utilizada como base por muitas aplicações. É orientada a objetos, o que significa ser baseada na modelagem e comunicação entre os objetos. Também é uma linguagem estaticamente tipada, ou seja, o usuário precisa declarar o tipo de dados que será armazenado em cada variável declarada.

() É uma linguagem de programação desenvolvida pela Microsoft e que possui tipagem. A tipagem possibilita que o desenvolvedor declare o tipo de uma variável, como numérico, textual ou data, por exemplo. Ele permite desenvolver aplicações tanto do lado do cliente como do lado do servidor.

A sequência correta dessa associação é

- a) 5, 3, 2, 1, 4.
- b) 4, 3, 2, 1, 5.
- c) 5, 1, 3, 2, 4.
- d) 1, 2, 3, 5, 4.
- e) 4, 5, 1, 2, 3.

Comentários:

Questão extensa, né? Mas muito didática! Vamos relembrar todos os conceitos nessa questão! E aprender o que você não sabe ainda! Vamos fazer uma revisão de cada item, mesmo que não esteja compreendido na aula, é bom relembrar até porque na prova as questões vem assim, né?

1 - Java: Mantida pela Oracle, Java é uma linguagem de programação amplamente utilizada no desenvolvimento de aplicativos móveis, especialmente para o sistema operacional Android. Java é orientada a objetos, o que significa que se baseia na modelagem e comunicação entre objetos. Além disso, é uma linguagem estaticamente tipada, o que requer que o desenvolvedor declare o tipo de dados que será armazenado em cada variável declarada. Com sua ampla adoção e suporte, Java é uma das principais linguagens para desenvolvimento móvel.

2 - JavaScript: JavaScript é uma linguagem de programação versátil e amplamente utilizada no desenvolvimento web, incluindo aplicativos móveis baseados em tecnologias como Cordova e React Native. Originalmente criada para controlar o HTML, CSS e manipular comportamentos em páginas web, o JavaScript evoluiu para possibilitar o desenvolvimento de aplicações tanto no lado cliente como no lado servidor. Mantida pela ECMA, é uma linguagem dinamicamente tipada, o que significa que não é necessário declarar explicitamente o tipo das variáveis.

3 - Kotlin: Kotlin é uma linguagem de programação moderna e mais recente que tem ganhado popularidade no desenvolvimento de aplicativos móveis, especialmente para Android.

Desenvolvida e mantida pela JetBrains, ela combina elementos da programação orientada a objetos e funcional. Kotlin é uma linguagem concisa e segura, com recursos avançados como suporte a expressões lambda (anônimas) e inferência de tipos. Ela visa melhorar a produtividade do desenvolvedor, oferecendo uma sintaxe mais limpa e reduzindo a possibilidade de erros comuns.

4 - TypeScript: TypeScript é uma linguagem de programação desenvolvida pela Microsoft que adiciona recursos de tipagem estática ao JavaScript. Ela permite que os desenvolvedores declarem explicitamente o tipo de uma variável, fornecendo assim maior segurança e ferramentas de verificação de tipo durante o desenvolvimento. O TypeScript é um superconjunto sintático estrito do JavaScript, o que significa que qualquer código JavaScript válido também é código TypeScript válido. Essa linguagem é amplamente utilizada no desenvolvimento de aplicativos móveis, fornecendo uma maneira mais robusta e escalável de escrever código JavaScript.

5 - Objective-C: Objective-C é uma linguagem de programação usada principalmente para o desenvolvimento de aplicativos iOS. Ela é um superconjunto da linguagem de programação C, o que significa que agrega recursos ao C e adiciona recursos orientados a objetos. A Apple adotou o Objective-C como a linguagem principal para o desenvolvimento de aplicativos iOS antes do lançamento do Swift. Embora o Swift tenha se tornado a linguagem de programação recomendada pela Apple, ainda existem aplicativos legados escritos em Objective-C.

Gabarito: Letra A

3. (FGV – TJ RO/ 2021) Analise o código Kotlin exibido a seguir.

```
fun main( ) {  
    val classe = ?  
    val resposta = when (classe) {  
        1 -> "Premium"  
        2 -> "Superior"  
        3 -> "Econômica"  
        4, 5 -> when (classe % 2 == 0)  
            {true -> "Gold" false -> "Iron"}  
        else -> "Dado inválido."  
    }  
    println(resposta)  
}
```

O par que indica corretamente o valor a ser atribuído à variável classe, na segunda linha, e o valor computado e exibido pelo comando println é:

- a) 3 / Iron
- b) 4 / Iron

- c) 5 / Iron
- d) 5 / Gold
- e) 9 / Gold

Comentários:

A combinação válida entre as alternativas é a seguinte: c) 5 / Iron. O algoritmo apresentado é um código escrito na linguagem de programação Kotlin. Vamos entender passo a passo o que esse código faz: O código começa com a função `main()`, que é a função principal do programa. É a partir dessa função que o programa será executado. Na segunda linha, declara-se uma variável chamada `classe` com um valor indefinido, indicado pelo ponto de interrogação `?`. O valor dessa variável será fornecido posteriormente durante a execução do programa. Na terceira linha, declara-se uma variável chamada `resposta` e utiliza-se a estrutura `when` para fazer uma seleção condicional com base no valor da variável `classe`. Dentro do bloco `when`, são especificados os possíveis valores de `classe` e as respostas correspondentes a cada um deles. Se o valor de `classe` for igual a 1, a resposta atribuída à variável `resposta` será "Premium". Se o valor de `classe` for igual a 2, a resposta atribuída à variável `resposta` será "Superior". Se o valor de `classe` for igual a 3, a resposta atribuída à variável `resposta` será "Econômica". Se o valor de `classe` for igual a 4 ou 5, o código entra em outro bloco `when` para fazer uma nova seleção condicional com base no resultado da expressão `classe % 2 == 0`. Essa expressão verifica se o valor de `classe` é divisível por 2 (ou seja, se é par). Se a expressão `classe % 2 == 0` for avaliada como verdadeira (par), a resposta atribuída à variável `resposta` será "Gold". Se a expressão `classe % 2 == 0` for avaliada como falsa (ímpar), a resposta atribuída à variável `resposta` será "Iron". Caso o valor de `classe` não corresponda a nenhum dos casos anteriores, ou seja, não seja igual a 1, 2, 3, 4 ou 5, a resposta atribuída à variável `resposta` será "Dado inválido." Por fim, a resposta contida na variável `resposta` é impressa no console utilizando o comando `println`. Ao analisar o código Kotlin fornecido, podemos identificar que o valor a ser atribuído à variável `"classe"` é 5, conforme indicado na segunda linha do código. Em seguida, o código utiliza a estrutura `"when"` para avaliar o valor da variável `"classe"` e determinar o valor da variável `"resposta"`. Assim, quando o valor de `"classe"` é igual a 5, o código entra no bloco correspondente e avalia a expressão `"classe % 2 == 0"`. No entanto, o resto da divisão de 5 por 2 é igual a 1, o que não atende à condição de ser igual a 0. Portanto, o código segue para o bloco alternativo e atribui o valor "Iron" à variável `"resposta"`. Finalmente, o valor de `"resposta"` é exibido utilizando o comando `"println"`, resultando na impressão de "5/Iron" como resposta final.

Gabarito: Letra C

4. (Cesgranrio – Caixa / 2021) Na linguagem de programação Kotlin, é possível criar uma variável cujo valor nunca pode ser mudado, na prática, uma constante, com o nome `idademinima`, do tipo básico inteiro de 32 bits, com o valor 18.

Para que isso aconteça, qual das seguintes instruções deve ser usada?

- a) `val idademinima : Int = 18`

- b) val idademinima : Integer = 18
- c) val idademinima = 18 : Integer
- d) var idademinima : Int = 18
- e) var idademinima : Integer = 18

Comentários:

Na linguagem de programação Kotlin, para criar uma variável cujo valor nunca pode ser alterado, ou seja, uma constante, deve-se utilizar a instrução "val". Para definir a constante "idademinima", que possui um valor fixo e não pode ser modificado, deve-se utilizar a instrução "val". Essa instrução indica que a variável é imutável e não é possível atribuir um novo valor a ela após a sua inicialização. No caso específico da constante "idademinima", que possui o tipo básico inteiro de 32 bits, a opção correta é a alternativa A: "val idademinima: Int = 18". Essa declaração cria uma constante chamada "idademinima" do tipo "Int" e atribui o valor 18 a ela. É importante notar que as alternativas B, C, D e E são incorretas, pois utilizam a instrução "var" em vez de "val". A instrução "var" é utilizada para criar variáveis mutáveis, ou seja, variáveis cujo valor pode ser alterado ao longo do código. Portanto, a resposta correta para criar uma constante imutável em Kotlin é a alternativa A: "val idademinima: Int = 18".

Gabarito: Letra A

5. (CESPE – PGDF/ 2021) O Spring WebFlux é compatível com Java 8 lambdas e Kotlin e tem a vantagem de permitir a criação de microserviços com requisitos menos complexos.

Comentários:

O Spring WebFlux é compatível com Java 8 lambdas e Kotlin. Isso significa que é possível utilizar tanto lambdas em Java 8 como recursos da linguagem Kotlin ao definir os endpoints no Spring WebFlux. A vantagem de utilizar o Spring WebFlux é que ele permite a criação de microserviços com requisitos menos complexos. Isso é especialmente útil para aplicações menores ou microserviços que não possuem requisitos muito complicados. Em termos de criação de endpoints, uma abordagem mais recente é o uso de expressões lambda. Essas expressões são suportadas pelo Java a partir da versão 1.8 e, com a ajuda do Kotlin, é possível utilizá-las mesmo em versões anteriores do Java. É uma questão que trata majoritariamente sobre Spring WebFlux, porém é uma informação adicional ao estudo de Kotlin.

Gabarito: Correto

6. (CESPE – PGDF/ 2021) O JUnit 5 é formado por JUnit Platform, JUnit Jupiter e JUnit Vintage; o JUnit Jupiter pode ser utilizado em programas escritos em Kotlin.

Comentários:

O JUnit 5 é um framework de teste que consiste em três projetos principais: JUnit Platform, JUnit Jupiter e JUnit Vintage. Esses projetos trabalham juntos para fornecer uma arquitetura nova e abrangente para testes. Vou comentar as informações usando conjunções subordinativas:

A nova arquitetura do JUnit 5 é dividida em vários módulos, que são agrupados em três projetos que compõem o novo framework. Essa divisão é importante porque permite uma estrutura modularizada e organizada para o desenvolvimento e execução dos testes. O JUnit Platform é o primeiro projeto e contém os elementos estruturais necessários para a execução de testes. Ele define APIs que permitem que outros frameworks de teste sejam executados dentro da plataforma do JUnit. Essa capacidade de integração com outros frameworks é extremamente útil para a compatibilidade e reutilização de código de teste existente. Por sua vez, o JUnit Jupiter é responsável por definir o modelo de programação utilizado para escrever os testes no JUnit 5. Ele fornece APIs que permitem a extensão de comportamentos, ou seja, permite adicionar funcionalidades personalizadas aos testes. Nesse projeto, são definidas as anotações e classes fundamentais para a construção dos testes, como a anotação "@Test" e outras. Além disso, ele inclui uma implementação do TestEngine, que é responsável por executar o novo modelo de programação na plataforma do JUnit. Também uma questão que trata majoritariamente sobre JUnit, porém é uma informação adicional ao estudo de Kotlin.

Gabarito: Correto

7. (AOCP – Prefeitura de Novo Hamburgo - RS / 2020) Considerando o ambiente de desenvolvimento Android Studio, assinale a alternativa que apresenta apenas linguagens válidas ao adicionar uma nova activity ao projeto.

- a) Java e Swift.
- b) C e Java.
- c) JavaScript e Python.
- d) C# e PHP.
- e) Java e Kotlin.

Comentários:

No ambiente de desenvolvimento Android Studio, ao adicionar uma nova activity ao projeto, as linguagens válidas são Kotlin e Java. Essas duas linguagens são oficialmente suportadas pelo Google para o desenvolvimento de aplicativos Android.

Kotlin é uma linguagem de programação moderna e concisa que foi oficializada pelo Google como uma das linguagens preferenciais para o desenvolvimento Android. Ela oferece uma sintaxe mais simples e mais segura em comparação com o Java, além de trazer recursos avançados que aumentam a produtividade do desenvolvedor.

Java, por sua vez, é a linguagem de programação nativa do Android. Ela tem sido amplamente utilizada na plataforma Android há muitos anos e ainda é amplamente adotada. Muitos

aplicativos existentes foram desenvolvidos em Java e ainda há uma grande quantidade de código Java disponível para reutilização.

Quanto ao C++, embora seja uma linguagem válida para o desenvolvimento Android, ela não é uma escolha comum para criar atividades ou telas no Android Studio. Isso ocorre porque o C++ não é executado na JVM (Java Virtual Machine) como o Kotlin e o Java, o que torna seu uso menos recomendado para criar interfaces de usuário no contexto do desenvolvimento Android.

Gabarito: Letra E

8. (CESGRANRIO – Banco do Brasil / 2021) Foi solicitado a um programador de sistemas de informação que transformasse uma classe escrita em Java em uma classe equivalente, para ser executada em um programa Kotlin.

O código da classe Java é:

```
public class AlunoJava{  
    private String codigo;  
    private String nome;  
    private int numero=0;  
    private String texto= "EscolaX";  
    public AlunoJava (String codigo,String nome)  
        { this.codigo = codigo;  
          this.nome = nome; }  
}
```

A classe em Kotlin equivalente à classe Java acima é

- a) `public class AlunoKotlin (private String: nome , private String: codigo )`
`{ private:`
 `numero int = 0`
 `texto String = "EscolaX" }`
- b) `public class AlunoKotlin (private var nome; codigo: String)`
`{ private var numero = 0`
 `private var texto = "EscolaX" }`
- c) `class AlunoKotlin (val nome: String, val codigo: String)`
`{ private this.nome = nome`
 `private this.codigo=codigo`
 `private var int numero = 0`
 `private var String texto = "EscolaX" }`
- d) `class AlunoKotlin (var nome: String, var codigo: String)`

```
{ private var numero = 0  
  private var texto = "EscolaX"  
  private AlunoKotlin.nome, AlunoKotlin.codigo }
```

```
e)class AlunoKotlin (private val nome: String, private val codigo: String)  
  { private var numero = 0  
    private var texto = "EscolaX" }
```

Comentários:

A classe equivalente em Kotlin para a classe Java apresentada é a letra E. Vamos analisar cada elemento da classe Java original e como eles foram traduzidos para Kotlin:

Modificadores de acesso: A classe Java original não possui especificação de modificador de acesso, então a classe em Kotlin também não possui nenhum modificador. Por padrão, as classes em Kotlin são públicas.

Atributos: Os atributos privados codigo e nome da classe Java foram traduzidos diretamente para Kotlin como parâmetros de entrada do construtor primário. Em Kotlin, a declaração de atributos é feita diretamente no construtor primário quando são definidos como parâmetros.

Atributos adicionais: Os atributos numero e texto da classe Java foram declarados como atributos da classe em Kotlin e inicializados com os valores padrão de 0 e "EscolaX", respectivamente. Em Kotlin, a declaração de atributos é feita diretamente na classe.

Construtor: Em Kotlin, o construtor primário é definido diretamente na declaração da classe. Os parâmetros codigo e nome foram definidos como parâmetros do construtor primário em Kotlin. Assim, quando uma instância de AlunoKotlin é criada, os valores correspondentes devem ser passados como argumentos.

Gabarito: Letra E

9. (AOCP – SANESUL/ 2021) Qual é o nome de uma API de Fluxo do Kotlin para programação android que é considerada uma ótima opção para classes que precisam manter um estado mutável observável?

- a) ClassFlow
- b) MutableFlow
- c) QuickFlow
- d) StateFlow
- e) SharedFlow

Comentários:

Vamos entender cada item: A) ClassFlow: Não é uma opção válida. Não existe uma API chamada "ClassFlow" no contexto da programação Android. B) MutableFlow: Não é uma opção válida. Embora exista a classe MutableSharedFlow no Kotlin, ela não é a melhor opção para classes que precisam manter um estado mutável observável. C) QuickFlow: Não é uma opção válida. Não existe uma API chamada "QuickFlow" no contexto da programação Android. D) StateFlow: É a opção correta. O StateFlow é uma API de Fluxo do Kotlin específica para o Android. É um fluxo observável com estado, o qual emite as atualizações de estado novas e atuais para os coletores. Ele fornece acesso ao valor atual do estado por meio da propriedade "value" da classe MutableStateFlow. É uma ótima opção para classes que precisam manter um estado mutável observável. E) SharedFlow: Não é a opção correta. O SharedFlow é outro tipo de fluxo observável do Kotlin, mas não possui estado mutável como o StateFlow. O SharedFlow é usado para transmitir fluxos de eventos assíncronos semelhantes ao StateFlow, mas não possui a capacidade de atualizar o estado atual.

Gabarito: Letra D

10.(UNICENTRO – UNICENTRO / 2019) Assinale a alternativa que contenha o código na linguagem de programação Kotlin que NÃO será executado na plataforma Android por erro de compilação:

a) `text!!.let {
 val tamanho = text.length
}`

b) `var res = when(pontos) {
 9, 10 -> "Excelente"
 in 6..8 -> "Bom"
 4, 5 -> "Ok"
 in 1..3 -> "Ruim"
 else -> "Ruim"
}`

c) `for (i in 1..10) {
 println(i)
}`

d) `for (i in 10 upTo 0) {
 println(i)
}`

e) `for (i in 10 downTo 1 step 2) {
 println(i)
}`

Comentários:

Vamos analisar cada item e explicar por que está certo ou errado em relação ao código que não será executado na plataforma Android devido a um erro de compilação:

A) O código está correto e será executado sem erros na plataforma Android. A função `let` é um escopo de função do Kotlin que permite realizar operações em um objeto não nulo. Neste caso, o código está usando o operador de chamada segura `!!` para acessar a propriedade `length` de `text` e, em seguida, obtendo o tamanho da string.

B) O código está correto e será executado sem erros na plataforma Android. Ele usa uma expressão `when` para atribuir a variável `res` uma string dependendo do valor de `pontos`. O código usa as estruturas de controle `in` e `else` corretamente.

C) O código está correto e será executado sem erros na plataforma Android. É um laço `for` simples que imprime os números de 1 a 10.

D) O código está incorreto e resultará em um erro de compilação na plataforma Android. O operador `upTo` não é reconhecido no Kotlin. Para percorrer de 10 a 0, você pode usar o operador `downTo`, como mostrado na opção E. Lembrando: As progressões em Kotlin são usadas para iterar sobre intervalos de valores numéricos. Para iterar em ordem reversa, utiliza-se a função `downTo`.

E) O código está correto e será executado sem erros na plataforma Android. É um laço `for` que percorre de 10 a 1 com um passo de 2 e imprime os valores.

Gabarito: Letra B

QUESTÕES COMENTADAS

Kotlin

1. (CESGRANRIO – Banco do Brasil/ 2023) Kotlin é uma linguagem de programação usada no desenvolvimento Android.

Entre suas características, está um grau de compatibilidade com Java, que permite

- a) chamar funções feitas em Java, apenas, mas não permite que suas funções Kotlin sejam chamadas por Java.
- b) ler dados que foram salvos por apps Java, apenas.
- c) ler e escrever dados que podem ser lidos e escritos por apps Java, apenas.
- d) ter suas funções chamadas por Java, apenas, mas não consegue chamar funções feitas em Java.
- e) construir apps com código parcialmente em Java e parcialmente em Kotlin, sem restrições.

Comentários:

Muito interessante a questão! Vamos analisar cada item!

- a) Chamar funções feitas em Java, apenas, mas não permite que suas funções Kotlin sejam chamadas por Java.

Essa afirmação está incorreta. Kotlin e Java são interoperáveis, o que significa que é possível chamar funções escritas em Kotlin a partir de código Java e vice-versa. A compatibilidade entre as duas linguagens é um dos principais benefícios do Kotlin, permitindo que desenvolvedores migrem gradualmente de Java para Kotlin ou combinem ambas em um mesmo projeto.

- b) Ler dados que foram salvos por apps Java, apenas.

Essa afirmação também está incorreta. O Kotlin é capaz de ler dados salvos por aplicativos Java, bem como salvar e ler seus próprios dados. Não há restrições em relação à leitura de dados entre as duas linguagens.

- c) Ler e escrever dados que podem ser lidos e escritos por apps Java, apenas.

Essa afirmação também está incorreta. Assim como mencionado anteriormente, o Kotlin é capaz de ler e escrever dados que foram salvos por aplicativos Java, além de lidar com seus próprios dados. Não há restrições nesse aspecto.

- d) Ter suas funções chamadas por Java, apenas, mas não consegue chamar funções feitas em Java.

Essa afirmação está incorreta. O Kotlin permite que suas funções sejam chamadas por código Java, assim como também permite que funções feitas em Java sejam chamadas por código Kotlin. A interoperabilidade entre as duas linguagens permite que o desenvolvedor trabalhe com ambas sem restrições.

e) Construir apps com código parcialmente em Java e parcialmente em Kotlin, sem restrições.

Essa afirmação está correta. O Kotlin oferece a flexibilidade de construir aplicativos combinando código escrito em Java e Kotlin sem restrições. É possível utilizar qualquer quantidade de Kotlin no projeto, combinando-o com código Java de forma natural e aproveitando a interoperabilidade entre as duas linguagens.

Gabarito: Letra E

2. (FCM – CEFETMINAS / 2022) Com relação ao desenvolvimento de aplicativos móveis, relacione as linguagens de programação com suas características.

LINGUAGEM DE PROGRAMAÇÃO

- 1 - Java
- 2 - JavaScript
- 3 - Kotlin
- 4 - TypeScript
- 5 - Objective-C

CARACTERÍSTICAS

() É um superconjunto da linguagem de programação C, ou seja, agrega recursos ao C. Ele possibilita o uso do paradigma programação orientada a objetos, contendo sintaxe para a criação de métodos e classes.

() É uma linguagem mais recente e que tem ganhado força nos últimos tempos. É desenvolvida e mantida pela JetBrains. Utiliza o paradigma orientado a objetos e tem suporte ao paradigma funcional, com o uso de expressões lambda (anônimas).

() É uma linguagem de programação usada principalmente para controlar o Hypertext Markup Language (HTML) e o Cascading Style Sheets (CSS) e manipular comportamentos em uma página web. É mantido pela European Computer Manufacturer's Association (ECMA). Originalmente, foi criada para o desenvolvimento de aplicações no lado cliente, mas evoluiu para possibilitar o desenvolvimento de aplicações desktop e no lado servidor.

() Mantida pela Oracle, é composta por uma linguagem de programação e uma plataforma computacional utilizada como base por muitas aplicações. É orientada a objetos, o que significa ser baseada na modelagem e comunicação entre os objetos. Também é uma linguagem estaticamente tipada, ou seja, o usuário precisa declarar o tipo de dados que será armazenado em cada variável declarada.

() É uma linguagem de programação desenvolvida pela Microsoft e que possui tipagem. A tipagem possibilita que o desenvolvedor declare o tipo de uma variável, como numérico, textual ou data, por exemplo. Ele permite desenvolver aplicações tanto do lado do cliente como do lado do servidor.

A sequência correta dessa associação é

- a) 5, 3, 2, 1, 4.
- b) 4, 3, 2, 1, 5.
- c) 5, 1, 3, 2, 4.
- d) 1, 2, 3, 5, 4.
- e) 4, 5, 1, 2, 3.

Comentários:

Questão extensa, né? Mas muito didática! Vamos lembrar todos os conceitos nessa questão! E aprender o que você não sabe ainda! Vamos fazer uma revisão de cada item, mesmo que não esteja compreendido na aula, é bom lembrar até porque na prova as questões vem assim, né?

1 - Java: Mantida pela Oracle, Java é uma linguagem de programação amplamente utilizada no desenvolvimento de aplicativos móveis, especialmente para o sistema operacional Android. Java é orientada a objetos, o que significa que se baseia na modelagem e comunicação entre objetos. Além disso, é uma linguagem estaticamente tipada, o que requer que o desenvolvedor declare o tipo de dados que será armazenado em cada variável declarada. Com sua ampla adoção e suporte, Java é uma das principais linguagens para desenvolvimento móvel.

2 - JavaScript: JavaScript é uma linguagem de programação versátil e amplamente utilizada no desenvolvimento web, incluindo aplicativos móveis baseados em tecnologias como Cordova e React Native. Originalmente criada para controlar o HTML, CSS e manipular comportamentos em páginas web, o JavaScript evoluiu para possibilitar o desenvolvimento de aplicações tanto no lado cliente como no lado servidor. Mantida pela ECMA, é uma linguagem dinamicamente tipada, o que significa que não é necessário declarar explicitamente o tipo das variáveis.

3 - Kotlin: Kotlin é uma linguagem de programação moderna e mais recente que tem ganhado popularidade no desenvolvimento de aplicativos móveis, especialmente para Android. Desenvolvida e mantida pela JetBrains, ela combina elementos da programação orientada a objetos e funcional. Kotlin é uma linguagem concisa e segura, com recursos avançados como

suporte a expressões lambda (anônimas) e inferência de tipos. Ela visa melhorar a produtividade do desenvolvedor, oferecendo uma sintaxe mais limpa e reduzindo a possibilidade de erros comuns.

4 - TypeScript: TypeScript é uma linguagem de programação desenvolvida pela Microsoft que adiciona recursos de tipagem estática ao JavaScript. Ela permite que os desenvolvedores declarem explicitamente o tipo de uma variável, fornecendo assim maior segurança e ferramentas de verificação de tipo durante o desenvolvimento. O TypeScript é um superconjunto sintático estrito do JavaScript, o que significa que qualquer código JavaScript válido também é código TypeScript válido. Essa linguagem é amplamente utilizada no desenvolvimento de aplicativos móveis, fornecendo uma maneira mais robusta e escalável de escrever código JavaScript.

5 - Objective-C: Objective-C é uma linguagem de programação usada principalmente para o desenvolvimento de aplicativos iOS. Ela é um superconjunto da linguagem de programação C, o que significa que agrega recursos ao C e adiciona recursos orientados a objetos. A Apple adotou o Objective-C como a linguagem principal para o desenvolvimento de aplicativos iOS antes do lançamento do Swift. Embora o Swift tenha se tornado a linguagem de programação recomendada pela Apple, ainda existem aplicativos legados escritos em Objective-C.

Gabarito: Letra A

3. (FGV – TJ RO/ 2021) Analise o código Kotlin exibido a seguir.

```
fun main() {  
    val classe = ?  
    val resposta = when (classe) {  
        1 -> "Premium"  
        2 -> "Superior"  
        3 -> "Econômica"  
        4, 5 -> when (classe % 2 == 0)  
            {true -> "Gold" false -> "Iron"}  
        else -> "Dado inválido."  
    }  
    println(resposta)  
}
```

O par que indica corretamente o valor a ser atribuído à variável classe, na segunda linha, e o valor computado e exibido pelo comando println é:

- a) 3 / Iron
- b) 4 / Iron
- c) 5 / Iron
- d) 5 / Gold
- e) 9 / Gold

Comentários:

A combinação válida entre as alternativas é a seguinte: c) 5 / Iron. O algoritmo apresentado é um código escrito na linguagem de programação Kotlin. Vamos entender passo a passo o que esse código faz: O código começa com a função `main()`, que é a função principal do programa. É a partir dessa função que o programa será executado. Na segunda linha, declara-se uma variável chamada `classe` com um valor indefinido, indicado pelo ponto de interrogação `?`. O valor dessa variável será fornecido posteriormente durante a execução do programa. Na terceira linha, declara-se uma variável chamada `resposta` e utiliza-se a estrutura `when` para fazer uma seleção condicional com base no valor da variável `classe`. Dentro do bloco `when`, são especificados os possíveis valores de `classe` e as respostas correspondentes a cada um deles. Se o valor de `classe` for igual a 1, a resposta atribuída à variável `resposta` será "Premium". Se o valor de `classe` for igual a 2, a resposta atribuída à variável `resposta` será "Superior". Se o valor de `classe` for igual a 3, a resposta atribuída à variável `resposta` será "Econômica". Se o valor de `classe` for igual a 4 ou 5, o código entra em outro bloco `when` para fazer uma nova seleção condicional com base no resultado da expressão `classe % 2 == 0`. Essa expressão verifica se o valor de `classe` é divisível por 2 (ou seja, se é par). Se a expressão `classe % 2 == 0` for avaliada como verdadeira (par), a resposta atribuída à variável `resposta` será "Gold". Se a expressão `classe % 2 == 0` for avaliada como falsa (ímpar), a resposta atribuída à variável `resposta` será "Iron". Caso o valor de `classe` não corresponda a nenhum dos casos anteriores, ou seja, não seja igual a 1, 2, 3, 4 ou 5, a resposta atribuída à variável `resposta` será "Dado inválido." Por fim, a resposta contida na variável `resposta` é impressa no console utilizando o comando `println`. Ao analisar o código Kotlin fornecido, podemos identificar que o valor a ser atribuído à variável "classe" é 5, conforme indicado na segunda linha do código. Em seguida, o código utiliza a estrutura "when" para avaliar o valor da variável "classe" e determinar o valor da variável "resposta". Assim, quando o valor de "classe" é igual a 5, o código entra no bloco correspondente e avalia a expressão "classe % 2 == 0". No entanto, o resto da divisão de 5 por 2 é igual a 1, o que não atende à condição de ser igual a 0. Portanto, o código segue para o bloco alternativo e atribui o valor "Iron" à variável "resposta". Finalmente, o valor de "resposta" é exibido utilizando o comando "println", resultando na impressão de "5/Iron" como resposta final.

Gabarito: Letra C

4. (Cesgranrio – Caixa / 2021) Na linguagem de programação Kotlin, é possível criar uma variável cujo valor nunca pode ser mudado, na prática, uma constante, com o nome `idademinima`, do tipo básico inteiro de 32 bits, com o valor 18.

Para que isso aconteça, qual das seguintes instruções deve ser usada?

- a) `val idademinima : Int = 18`
- b) `val idademinima : Integer = 18`
- c) `val idademinima = 18 : Integer`
- d) `var idademinima : Int = 18`
- e) `var idademinima : Integer = 18`

Comentários:

Na linguagem de programação Kotlin, para criar uma variável cujo valor nunca pode ser alterado, ou seja, uma constante, deve-se utilizar a instrução "val". Para definir a constante "idademinima", que possui um valor fixo e não pode ser modificado, deve-se utilizar a instrução "val". Essa instrução indica que a variável é imutável e não é possível atribuir um novo valor a ela após a sua inicialização. No caso específico da constante "idademinima", que possui o tipo básico inteiro de 32 bits, a opção correta é a alternativa A: "val idademinima: Int = 18". Essa declaração cria uma constante chamada "idademinima" do tipo "Int" e atribui o valor 18 a ela. É importante notar que as alternativas B, C, D e E são incorretas, pois utilizam a instrução "var" em vez de "val". A instrução "var" é utilizada para criar variáveis mutáveis, ou seja, variáveis cujo valor pode ser alterado ao longo do código. Portanto, a resposta correta para criar uma constante imutável em Kotlin é a alternativa A: "val idademinima: Int = 18".

Gabarito: Letra A

5. (CESPE – PGDF/ 2021) O Spring WebFlux é compatível com Java 8 lambdas e Kotlin e tem a vantagem de permitir a criação de microsserviços com requisitos menos complexos.

Comentários:

O Spring WebFlux é compatível com Java 8 lambdas e Kotlin. Isso significa que é possível utilizar tanto lambdas em Java 8 como recursos da linguagem Kotlin ao definir os endpoints no Spring WebFlux. A vantagem de utilizar o Spring WebFlux é que ele permite a criação de microsserviços com requisitos menos complexos. Isso é especialmente útil para aplicações menores ou microsserviços que não possuem requisitos muito complicados. Em termos de criação de endpoints, uma abordagem mais recente é o uso de expressões lambda. Essas expressões são suportadas pelo Java a partir da versão 1.8 e, com a ajuda do Kotlin, é possível utilizá-las mesmo em versões anteriores do Java. É uma questão que trata majoritariamente sobre Spring WebFlux, porém é uma informação adicional ao estudo de Kotlin.

Gabarito: Correto

6. (CESPE – PGDF/ 2021) O JUnit 5 é formado por JUnit Platform, JUnit Jupiter e JUnit Vintage; o JUnit Jupiter pode ser utilizado em programas escritos em Kotlin.

Comentários:

O JUnit 5 é um framework de teste que consiste em três projetos principais: JUnit Platform, JUnit Jupiter e JUnit Vintage. Esses projetos trabalham juntos para fornecer uma arquitetura nova e abrangente para testes. Vou comentar as informações usando conjunções subordinativas:

A nova arquitetura do JUnit 5 é dividida em vários módulos, que são agrupados em três projetos que compõem o novo framework. Essa divisão é importante porque permite uma estrutura

modularizada e organizada para o desenvolvimento e execução dos testes. O JUnit Platform é o primeiro projeto e contém os elementos estruturais necessários para a execução de testes. Ele define APIs que permitem que outros frameworks de teste sejam executados dentro da plataforma do JUnit. Essa capacidade de integração com outros frameworks é extremamente útil para a compatibilidade e reutilização de código de teste existente. Por sua vez, o JUnit Jupiter é responsável por definir o modelo de programação utilizado para escrever os testes no JUnit 5. Ele fornece APIs que permitem a extensão de comportamentos, ou seja, permite adicionar funcionalidades personalizadas aos testes. Nesse projeto, são definidas as anotações e classes fundamentais para a construção dos testes, como a anotação "@Test" e outras. Além disso, ele inclui uma implementação do TestEngine, que é responsável por executar o novo modelo de programação na plataforma do JUnit. Também uma questão que trata majoritariamente sobre JUnit, porém é uma informação adicional ao estudo de Kotlin.

Gabarito: Correto

7. (AOCP – Prefeitura de Novo Hamburgo - RS / 2020) Considerando o ambiente de desenvolvimento Android Studio, assinale a alternativa que apresenta apenas linguagens válidas ao adicionar uma nova activity ao projeto.

- a) Java e Swift.
- b) C e Java.
- c) JavaScript e Python.
- d) C# e PHP.
- e) Java e Kotlin.

Comentários:

No ambiente de desenvolvimento Android Studio, ao adicionar uma nova activity ao projeto, as linguagens válidas são Kotlin e Java. Essas duas linguagens são oficialmente suportadas pelo Google para o desenvolvimento de aplicativos Android.

Kotlin é uma linguagem de programação moderna e concisa que foi oficializada pelo Google como uma das linguagens preferenciais para o desenvolvimento Android. Ela oferece uma sintaxe mais simples e mais segura em comparação com o Java, além de trazer recursos avançados que aumentam a produtividade do desenvolvedor.

Java, por sua vez, é a linguagem de programação nativa do Android. Ela tem sido amplamente utilizada na plataforma Android há muitos anos e ainda é amplamente adotada. Muitos aplicativos existentes foram desenvolvidos em Java e ainda há uma grande quantidade de código Java disponível para reutilização.

Quanto ao C++, embora seja uma linguagem válida para o desenvolvimento Android, ela não é uma escolha comum para criar atividades ou telas no Android Studio. Isso ocorre porque o C++ não é

executado na JVM (Java Virtual Machine) como o Kotlin e o Java, o que torna seu uso menos recomendado para criar interfaces de usuário no contexto do desenvolvimento Android.

Gabarito: Letra E

8. (CESGRANRIO – Banco do Brasil / 2021) Foi solicitado a um programador de sistemas de informação que transformasse uma classe escrita em Java em uma classe equivalente, para ser executada em um programa Kotlin.

O código da classe Java é:

```
public class AlunoJava{  
    private String codigo;  
    private String nome;  
    private int numero=0;  
    private String texto= "EscolaX";  
    public AlunoJava (String codigo,String nome)  
        { this.codigo = codigo;  
          this.nome = nome; }  
}
```

A classe em Kotlin equivalente à classe Java acima é

- a) public class AlunoKotlin (private String: nome , private String: codigo)
 { private:
 numero int = 0
 texto String = "EscolaX" }
- b) public class AlunoKotlin (private var nome; codigo: String)
 { private var numero = 0
 private var texto = "EscolaX" }
- c) class AlunoKotlin (val nome: String, val codigo: String)
 { private this.nome = nome
 private this.codigo=codigo
 private var int numero = 0
 private var String texto = "EscolaX" }
- d) class AlunoKotlin (var nome: String, var codigo: String)
 { private var numero = 0
 private var texto = "EscolaX"
 private AlunoKotlin.nome, AlunoKotlin.codigo }
- e)class AlunoKotlin (private val nome: String, private val codigo: String)

```
{ private var numero = 0  
  private var texto = "EscolaX" }
```

Comentários:

A classe equivalente em Kotlin para a classe Java apresentada é a letra E. Vamos analisar cada elemento da classe Java original e como eles foram traduzidos para Kotlin:

Modificadores de acesso: A classe Java original não possui especificação de modificador de acesso, então a classe em Kotlin também não possui nenhum modificador. Por padrão, as classes em Kotlin são públicas.

Atributos: Os atributos privados código e nome da classe Java foram traduzidos diretamente para Kotlin como parâmetros de entrada do construtor primário. Em Kotlin, a declaração de atributos é feita diretamente no construtor primário quando são definidos como parâmetros.

Atributos adicionais: Os atributos número e texto da classe Java foram declarados como atributos da classe em Kotlin e inicializados com os valores padrão de 0 e "EscolaX", respectivamente. Em Kotlin, a declaração de atributos é feita diretamente na classe.

Construtor: Em Kotlin, o construtor primário é definido diretamente na declaração da classe. Os parâmetros código e nome foram definidos como parâmetros do construtor primário em Kotlin. Assim, quando uma instância de AlunoKotlin é criada, os valores correspondentes devem ser passados como argumentos.

Gabarito: Letra E

9. (AOCP – SANESUL/ 2021) Qual é o nome de uma API de Fluxo do Kotlin para programação android que é considerada uma ótima opção para classes que precisam manter um estado mutável observável?

- a) ClassFlow
- b) MutableFlow
- c) QuickFlow
- d) StateFlow
- e) SharedFlow

Comentários:

Vamos entender cada item: A) ClassFlow: Não é uma opção válida. Não existe uma API chamada "ClassFlow" no contexto da programação Android. B) MutableFlow: Não é uma opção válida. Embora exista a classe MutableSharedFlow no Kotlin, ela não é a melhor opção para classes que precisam manter um estado mutável observável. C) QuickFlow: Não é uma opção válida. Não existe uma API chamada "QuickFlow" no contexto da programação Android. D) StateFlow: É a opção

correta. O StateFlow é uma API de Fluxo do Kotlin específica para o Android. É um fluxo observável com estado, o qual emite as atualizações de estado novas e atuais para os coletores. Ele fornece acesso ao valor atual do estado por meio da propriedade "value" da classe MutableStateFlow. É uma ótima opção para classes que precisam manter um estado mutável observável. E) SharedFlow: Não é a opção correta. O SharedFlow é outro tipo de fluxo observável do Kotlin, mas não possui estado mutável como o StateFlow. O SharedFlow é usado para transmitir fluxos de eventos assíncronos semelhantes ao StateFlow, mas não possui a capacidade de atualizar o estado atual.

Gabarito: Letra D

10.(UNICENTRO – UNICENTRO / 2019) Assinale a alternativa que contenha o código na linguagem de programação Kotlin que NÃO será executado na plataforma Android por erro de compilação:

a) `text!!.let {
 val tamanho = text.length
}`

b) `var res = when(pontos) {
 9, 10 -> "Excelente"
 in 6..8 -> "Bom"
 4, 5 -> "Ok"
 in 1..3 -> "Ruim"
 else -> "Ruim"
}`

c) `for (i in 1..10) {
 println(i)
}`

d) `for (i in 10 upTo 0) {
 println(i)
}`

e) `for (i in 10 downTo 1 step 2) {
 println(i)
}`

Comentários:

Vamos analisar cada item e explicar por que está certo ou errado em relação ao código que não será executado na plataforma Android devido a um erro de compilação:

- A) O código está correto e será executado sem erros na plataforma Android. A função `let` é um escopo de função do Kotlin que permite realizar operações em um objeto não nulo. Neste caso, o código está usando o operador de chamada segura `!!` para acessar a propriedade `length` de `text` e, em seguida, obtendo o tamanho da string.
- B) O código está correto e será executado sem erros na plataforma Android. Ele usa uma expressão `when` para atribuir a variável `res` uma string dependendo do valor de pontos. O código usa as estruturas de controle `in` e `else` corretamente.
- C) O código está correto e será executado sem erros na plataforma Android. É um laço `for` simples que imprime os números de 1 a 10.
- D) O código está incorreto e resultará em um erro de compilação na plataforma Android. O operador `upTo` não é reconhecido no Kotlin. Para percorrer de 10 a 0, você pode usar o operador `downTo`, como mostrado na opção E. Lembrando: As progressões em Kotlin são usadas para iterar sobre intervalos de valores numéricos. Para iterar em ordem reversa, utiliza-se a função `downTo`.
- E) O código está correto e será executado sem erros na plataforma Android. É um laço `for` que percorre de 10 a 1 com um passo de 2 e imprime os valores.

Gabarito: Letra B

LISTA DE QUESTÕES

Kotlin

1. (CESGRANRIO – Banco do Brasil/ 2023) Kotlin é uma linguagem de programação usada no desenvolvimento Android.

Entre suas características, está um grau de compatibilidade com Java, que permite

- a) chamar funções feitas em Java, apenas, mas não permite que suas funções Kotlin sejam chamadas por Java.
 - b) ler dados que foram salvos por apps Java, apenas.
 - c) ler e escrever dados que podem ser lidos e escritos por apps Java, apenas.
 - d) ter suas funções chamadas por Java, apenas, mas não consegue chamar funções feitas em Java.
 - e) construir apps com código parcialmente em Java e parcialmente em Kotlin, sem restrições.
2. (FCM – CEFETMINAS / 2022) Com relação ao desenvolvimento de aplicativos móveis, relacione as linguagens de programação com suas características.

LINGUAGEM DE PROGRAMAÇÃO

- 1 - Java
- 2 - JavaScript
- 3 - Kotlin
- 4 - TypeScript
- 5 - Objective-C

CARACTERÍSTICAS

() É um superconjunto da linguagem de programação C, ou seja, agrega recursos ao C. Ele possibilita o uso do paradigma programação orientada a objetos, contendo sintaxe para a criação de métodos e classes.

() É uma linguagem mais recente e que tem ganhado força nos últimos tempos. É desenvolvida e mantida pela JetBrains. Utiliza o paradigma orientado a objetos e tem suporte ao paradigma funcional, com o uso de expressões lambda (anônimas).

() É uma linguagem de programação usada principalmente para controlar o Hypertext Markup Language (HTML) e o Cascading Style Sheets (CSS) e manipular comportamentos em uma página web. É mantido pela European Computer Manufacturer's Association (ECMA). Originalmente, foi criada para o desenvolvimento de aplicações no lado cliente, mas evoluiu para possibilitar o desenvolvimento de aplicações desktop e no lado servidor.

() Mantida pela Oracle, é composta por uma linguagem de programação e uma plataforma computacional utilizada como base por muitas aplicações. É orientada a objetos, o que significa ser baseada na modelagem e comunicação entre os objetos. Também é uma linguagem estaticamente tipada, ou seja, o usuário precisa declarar o tipo de dados que será armazenado em cada variável declarada.

() É uma linguagem de programação desenvolvida pela Microsoft e que possui tipagem. A tipagem possibilita que o desenvolvedor declare o tipo de uma variável, como numérico, textual ou data, por exemplo. Ele permite desenvolver aplicações tanto do lado do cliente como do lado do servidor.

A sequência correta dessa associação é

- a) 5, 3, 2, 1, 4.
- b) 4, 3, 2, 1, 5.
- c) 5, 1, 3, 2, 4.
- d) 1, 2, 3, 5, 4.
- e) 4, 5, 1, 2, 3.

3. (FGV – TJ RO/ 2021) Analise o código Kotlin exibido a seguir.

```
fun main() {  
    val classe = ?  
    val resposta = when (classe) {  
        1 -> "Premium"  
        2 -> "Superior"  
        3 -> "Econômica"  
        4, 5 -> when (classe % 2 == 0)  
            {true -> "Gold" false -> "Iron"}  
        else -> "Dado inválido."  
    }  
    println(resposta)  
}
```

O par que indica corretamente o valor a ser atribuído à variável classe, na segunda linha, e o valor computado e exibido pelo comando println é:

- a) 3 / Iron
- b) 4 / Iron
- c) 5 / Iron
- d) 5 / Gold
- e) 9 / Gold

4. **(Cesgranrio – Caixa / 2021)** Na linguagem de programação Kotlin, é possível criar uma variável cujo valor nunca pode ser mudado, na prática, uma constante, com o nome `idademinima`, do tipo básico inteiro de 32 bits, com o valor 18.

Para que isso aconteça, qual das seguintes instruções deve ser usada?

- a) `val idademinima : Int = 18`
- b) `val idademinima : Integer = 18`
- c) `val idademinima = 18 : Integer`
- d) `var idademinima : Int = 18`
- e) `var idademinima : Integer = 18`

5. **(CESPE – PGDF/ 2021)** O Spring WebFlux é compatível com Java 8 lambdas e Kotlin e tem a vantagem de permitir a criação de microsserviços com requisitos menos complexos.

6. **(CESPE – PGDF/ 2021)** O JUnit 5 é formado por JUnit Platform, JUnit Jupiter e JUnit Vintage; o JUnit Jupiter pode ser utilizado em programas escritos em Kotlin.

7. **(AOCP – Prefeitura de Novo Hamburgo - RS / 2020)** Considerando o ambiente de desenvolvimento Android Studio, assinale a alternativa que apresenta apenas linguagens válidas ao adicionar uma nova activity ao projeto.

- a) Java e Swift.
- b) C e Java.
- c) JavaScript e Python.
- d) C# e PHP.
- e) Java e Kotlin.

8. **(CESGRANRIO – Banco do Brasil / 2021)** Foi solicitado a um programador de sistemas de informação que transformasse uma classe escrita em Java em uma classe equivalente, para ser executada em um programa Kotlin.

O código da classe Java é:

```
public class AlunoJava{
    private String codigo;
    private String nome;
    private int numero=0;
    private String texto= "EscolaX";
    public AlunoJava (String codigo,String nome)
        { this.codigo = codigo;
          this.nome = nome; }
}
```

A classe em Kotlin equivalente à classe Java acima é

a) public class AlunoKotlin (private String: nome , private String: codigo)
 { private:
 numero int = 0
 texto String = "EscolaX" }

b) public class AlunoKotlin (private var nome; codigo: String)
 { private var numero = 0
 private var texto = "EscolaX" }

c) class AlunoKotlin (val nome: String, val codigo: String)
 { private this.nome = nome
 private this.codigo=codigo
 private var int numero = 0
 private var String texto = "EscolaX" }

d) class AlunoKotlin (var nome: String, var codigo: String)
 { private var numero = 0
 private var texto = "EscolaX"
 private AlunoKotlin.nome, AlunoKotlin.codigo }

e)class AlunoKotlin (private val nome: String, private val codigo: String)
 { private var numero = 0
 private var texto = "EscolaX" }

9. (AOCP – SANESUL/ 2021) Qual é o nome de uma API de Fluxo do Kotlin para programação android que é considerada uma ótima opção para classes que precisam manter um estado mutável observável?

- a) ClassFlow
- b) MutableFlow
- c) QuickFlow
- d) StateFlow
- e) SharedFlow

10.(UNICENTRO – UNICENTRO / 2019) Assinale a alternativa que contenha o código na linguagem de programação Kotlin que NÃO será executado na plataforma Android por erro de compilação:

a) `text!!.let {
 val tamanho = text.length
}`

b) `var res = when(pontos) {
 9, 10 -> "Excelente"
 in 6..8 -> "Bom"
 4, 5 -> "Ok"
 in 1..3 -> "Ruim"
 else -> "Ruim"
}`

c) `for (i in 1..10) {
 println(i)
}`

d) `for (i in 10 upTo 0) {
 println(i)
}`

e) `for (i in 10 downTo 1 step 2) {
 println(i)
}`

GABARITO

Kotlin

- | | | |
|------------|------------|-------------|
| 1. Letra E | 5. Correto | 8. Letra E |
| 2. Letra A | 6. Correto | 9. Letra D |
| 3. Letra C | 7. Letra E | 10. Letra B |
| 4. Letra A | | |

Sumário

Apresentação da Aula	2
Swift	3
Conceitos Básicos	3
Constantes e variáveis	5
Comentários	8
Ponto e vírgula	9
Inteiros	9
Floating-Point Numbers	10
Segurança de Tipo e Inferência de Tipo	10
Literais Numéricos	11
Conversão de Tipo Numérico	12
Booleans	14
Tuplas	15
Operadores Básicos	16
Strings e Caracteres	21
Interpolação de Strings	23
View controllers	27
Apresentando e gerenciando visualizações com um view controller	28
Exibindo e ocultando controladores de exibição	31
UIViewController	33
UITableViewController	34
UICollectionViewController	34
UIContentContainer	35
UISplitViewController	35
UINavigationController	36
UINavigationController	36
UINavigationController	36
UITabBarController	37
UITabBar	37
UIPageViewController	37
Protocolos	38
Referências	40

APRESENTAÇÃO DA AULA



Swift

Vamos falar sobre o desenvolvimento mobile com Swift, uma linguagem de programação moderna criada pela Apple, e as opções disponíveis para criar aplicativos para dispositivos iOS.

O Swift é a linguagem de programação oficial da Apple para o desenvolvimento de aplicativos iOS, macOS, watchOS e tvOS. Introduzida em 2014, o Swift trouxe melhorias significativas em relação à sua antecessora, a Objective-C, tornando o desenvolvimento de aplicativos mais seguro, rápido e fácil de aprender.

Uma das principais vantagens do desenvolvimento com Swift é a integração profunda com o ecossistema da Apple. Ao criar um aplicativo com Swift, os desenvolvedores têm acesso total às tecnologias e recursos exclusivos dos dispositivos iOS, como câmera, GPS, Touch ID, Face ID, Apple Pay e muitos outros. Isso permite a criação de experiências de usuário ricas e personalizadas, aproveitando ao máximo o hardware e o software da Apple.

Além disso, o Swift possui uma sintaxe mais limpa e concisa em comparação com o Objective-C, facilitando a leitura e a manutenção do código. Sua natureza segura também ajuda a prevenir erros comuns de programação e torna o aplicativo mais robusto e confiável.

No desenvolvimento mobile com Swift, temos duas opções principais: desenvolvimento nativo e desenvolvimento híbrido.

Para o desenvolvimento nativo, utilizamos o Xcode, a IDE oficial da Apple, que fornece todas as ferramentas necessárias para criar aplicativos iOS. Com o Xcode, podemos escrever código Swift e projetar interfaces de usuário usando o Interface Builder, tornando o processo de desenvolvimento mais eficiente e visual.

Quanto ao desenvolvimento híbrido, uma opção popular é o React Native, que mencionamos anteriormente. O React Native permite que os desenvolvedores escrevam parte do aplicativo em Swift (ou Objective-C) para acessar recursos específicos da plataforma, enquanto o restante do aplicativo é escrito em JavaScript, proporcionando uma experiência de usuário similar à de um aplicativo nativo.

Quando se trata de arquiteturas de aplicativos Swift, as mesmas opções que mencionamos antes também são aplicáveis: MVC (Model-View-Controller), MVP (Model-View-Presenter) e MVVM (Model-View-ViewModel). Cada uma delas tem suas características e benefícios, ajudando os desenvolvedores a criar aplicativos bem organizados, escaláveis e testáveis. Agora é hora de debruçar nos estudos e aprender o máximo! 😊

SWIFT

Conceitos Básicos

Swift é uma linguagem de programação desenvolvida pela Apple, com o objetivo de proporcionar uma experiência consistente e intuitiva no desenvolvimento de aplicativos para diversos dispositivos da marca, como iOS, Mac, Apple TV e Apple Watch. Com Swift, a Apple busca oferecer aos desenvolvedores uma ferramenta poderosa e acessível, permitindo que eles explorem todo o potencial criativo de suas ideias. Se você tem experiência em programação em C ou Objective-C, muitas partes do Swift serão familiares para você.

O Swift fornece suas próprias versões de todos os tipos fundamentais em C e Objective-C, incluindo Int para números inteiros, Double e Float para valores de ponto flutuante, Bool para valores booleanos e String para dados textuais. O Swift também oferece versões poderosas dos três principais tipos de coleção, Array, Set e Dictionary, conforme descrito em Tipos de Coleção.

A linguagem Swift foi projetada para ser fácil de usar, com uma sintaxe clara e concisa que facilita o desenvolvimento de aplicativos. Ela foi desenvolvida levando em consideração a experiência de programadores, incorporando conceitos modernos de programação e eliminando muitas das complexidades presentes em outras linguagens.

Uma das grandes vantagens do Swift é sua natureza de código aberto, o que significa que qualquer pessoa pode acessar seu código-fonte, contribuir com melhorias e utilizar a linguagem em seus próprios projetos. Essa abertura permite que a comunidade de desenvolvedores se envolva ativamente no aprimoramento e expansão do Swift, resultando em uma linguagem cada vez mais poderosa e versátil.

Com Swift, a Apple visa oferecer uma plataforma de desenvolvimento robusta e flexível, permitindo que os desenvolvedores criem aplicativos incríveis com maior facilidade. Seja para desenvolver apps inovadores para dispositivos móveis, explorar novas possibilidades na área de realidade aumentada ou criar soluções para o ecossistema Apple, Swift se destaca como uma linguagem aberta e poderosa, disponível para todos que desejam transformar suas ideias em realidade.

A linguagem de programação Swift é disponibilizada gratuitamente. A Apple desenvolveu a linguagem Swift para a criação de aplicativos para iOS, macOS, watchOS e tvOS. Além disso, a linguagem Swift é licenciada sob a licença Apache 2.0, que é uma licença de software livre e de código aberto. Essa licença permite que os desenvolvedores utilizem, modifiquem e distribuam a linguagem Swift de acordo com os termos da licença.

(CEBRASPE – TRT - 7ª Região (CE) / 2017) Assinale a opção que apresenta a linguagem de programação disponível, grátis e em código aberto, para desenvolvedores sob a licença Apache 2.0 e desenvolvida pela Apple para a criação de aplicativos para IOS.

- a) Xcode
- b) Swift

- c) Objective-C
- d) Python

Comentários:

A opção correta é "Swift". A linguagem de programação Swift é disponibilizada gratuitamente e é de código aberto, o que significa que seu código-fonte é aberto e acessível para desenvolvedores. A Apple desenvolveu a linguagem Swift para a criação de aplicativos para iOS, macOS, watchOS e tvOS. Além disso, a linguagem Swift é licenciada sob a licença Apache 2.0, que é uma licença de software livre e de código aberto. Essa licença permite que os desenvolvedores utilizem, modifiquem e distribuam a linguagem Swift de acordo com os termos da licença. Portanto, a resposta correta é "Swift". (Gabarito: Letra B)

(IBADE – Prefeitura de Itapemirim - ES / 2019) A Apple desenvolveu uma linguagem de programação própria para desenvolvimento de aplicações sob IOS. Ela se chama:

- a) C++.
- b) PHP.
- c) Python.
- d) Swift.
- e) Java Script.

Comentários:

De fato, a Apple desenvolveu sua própria linguagem de programação para o desenvolvimento de aplicativos iOS, e essa linguagem é chamada Swift. O Swift foi lançado pela Apple em 2014 e rapidamente ganhou popularidade devido à sua sintaxe concisa, facilidade de uso e segurança. Ele foi projetado para substituir a Objective-C como a principal linguagem de programação para o desenvolvimento de aplicativos Apple. Uma das principais motivações por trás do desenvolvimento do Swift foi fornecer aos desenvolvedores uma linguagem mais moderna, com recursos avançados e uma curva de aprendizado mais suave. Com o Swift, os desenvolvedores podem criar aplicativos iOS eficientes e poderosos, aproveitando as amplas bibliotecas e recursos disponíveis na plataforma da Apple. Portanto, o Swift é a linguagem de programação desenvolvida pela Apple especificamente para o desenvolvimento de aplicativos iOS. (Gabarito: Letra D)

Assim como em C, o Swift usa variáveis para armazenar e se referir a valores por um nome identificador. O Swift também faz amplo uso de variáveis cujos valores não podem ser alterados. Essas são conhecidas como constantes e são muito mais poderosas do que as constantes em C. As constantes são usadas em todo o Swift para tornar o código mais seguro e mais claro na intenção quando você trabalha com valores que não precisam ser alterados.

Além dos tipos familiares, o Swift introduz tipos avançados não encontrados no Objective-C, como tuples. Tuples permitem que você crie e passe grupos de valores. Você pode usar uma tuple para retornar vários valores de uma função como um único valor composto.

O Swift também introduz tipos opcionais, que lidam com a ausência de um valor. Optionals indicam se "existe um valor e ele é igual a x" ou "não existe um valor". O uso de optionals é

semelhante ao uso de nil com ponteiros no Objective-C, mas eles funcionam para qualquer tipo, não apenas classes. Os optionals não são apenas mais seguros e expressivos do que os ponteiros nil no Objective-C, eles estão no cerne de muitos dos recursos mais poderosos do Swift.

O Swift é uma linguagem com segurança de tipos, o que significa que a linguagem ajuda você a ser claro sobre os tipos de valores com os quais seu código pode trabalhar. Se uma parte do seu código requer uma String, a segurança de tipos impede que você passe um Int por engano. Da mesma forma, a segurança de tipos impede que você passe acidentalmente uma String opcional para um trecho de código que requer uma String não opcional. A segurança de tipos ajuda você a detectar e corrigir erros o mais cedo possível no processo de desenvolvimento.

Constantes e variáveis

Constantes e variáveis associam um nome (como `maximumNumberOfLoginAttempts` ou `welcomeMessage`) a um valor de um tipo específico (como o número 10 ou a string "Olá"). O valor de uma constante não pode ser alterado depois de ser definido, enquanto uma variável pode ter um valor diferente no futuro.

Declaração de Constantes e Variáveis

As constantes e variáveis devem ser declaradas antes de serem utilizadas. Você declara constantes com a palavra-chave `let` e variáveis com a palavra-chave `var`. Aqui está um exemplo de como constantes e variáveis podem ser usadas para acompanhar o número de tentativas de login que um usuário fez:

```
let maximumNumberOfLoginAttempts = 10  
var currentLoginAttempt = 0
```

Esse código pode ser lido da seguinte forma: "Declare uma nova constante chamada `maximumNumberOfLoginAttempts` e atribua a ela o valor 10. Em seguida, declare uma nova variável chamada `currentLoginAttempt` e atribua um valor inicial de 0."

Nesse exemplo, o número máximo de tentativas de login permitidas é declarado como uma constante, porque o valor máximo nunca muda. O contador de tentativas de login atual é declarado como uma variável, porque esse valor deve ser incrementado após cada tentativa de login malsucedida. Você pode declarar várias constantes ou várias variáveis em uma única linha, separadas por vírgulas:

```
var x = 0.0, y = 0.0, z = 0.0
```

Observação: Se um valor armazenado em seu código não mudar, declare-o sempre como uma constante usando a palavra-chave `let`. Use variáveis apenas para armazenar valores que precisam ser capazes de mudar.

Anotações de Tipo

Você pode fornecer uma anotação de tipo ao declarar uma constante ou variável, para deixar claro o tipo de valores que a constante ou variável pode armazenar. Escreva uma anotação de tipo colocando dois pontos após o nome da constante ou variável, seguido de um espaço e o

nome do tipo a ser usado. Este exemplo fornece uma anotação de tipo para uma variável chamada `welcomeMessage`, para indicar que a variável pode armazenar valores do tipo `String`:

```
var welcomeMessage: String
```

Os dois pontos na declaração significam "...do tipo...", então o código acima pode ser lido como: "Declare uma variável chamada `welcomeMessage` que é do tipo `String`". A frase "do tipo `String`" significa "pode armazenar qualquer valor do tipo `String`". Pense nisso como significando "o tipo de coisa" (ou "o tipo de objeto") que pode ser armazenado. A variável `welcomeMessage` agora pode ser atribuída a qualquer valor de string sem erro:

```
welcomeMessage = "Olá"
```

Você pode definir várias variáveis relacionadas do mesmo tipo em uma única linha, separadas por vírgulas, com uma única anotação de tipo após o nome da variável final:

```
var red, green, blue: Double
```

Observação:

É raro precisar escrever anotações de tipo na prática. Se você fornecer um valor inicial para uma constante ou variável no momento em que ela for definida, o Swift quase sempre pode inferir o tipo a ser usado para aquela constante ou variável, conforme descrito em Segurança de Tipo e Inferência de Tipo. No exemplo `welcomeMessage` acima, nenhum valor inicial é fornecido, e portanto, o tipo da variável `welcomeMessage` é especificado com uma anotação de tipo em vez de ser inferido a partir de um valor inicial.

Nomenclatura de Constantes e Variáveis

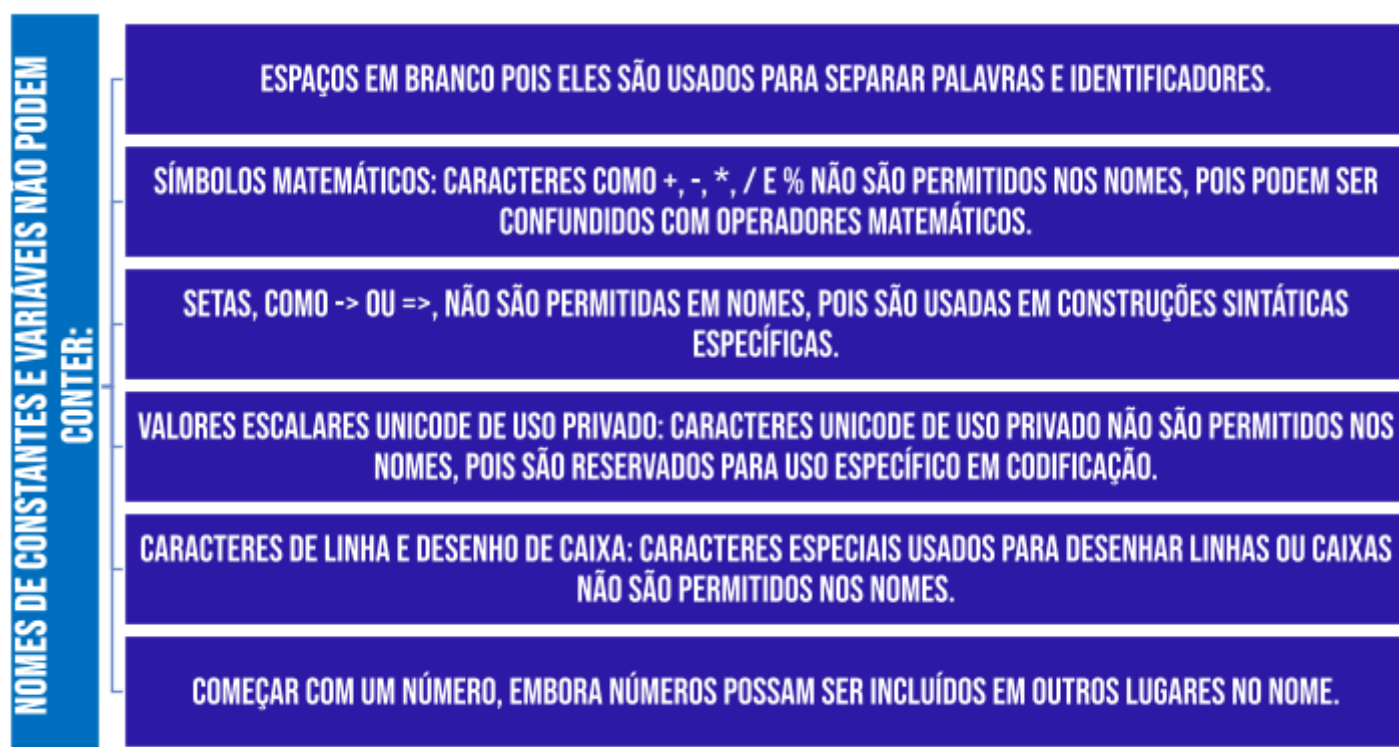
Nomes de constantes e variáveis podem conter quase todos os caracteres, oferecendo flexibilidade aos programadores. Primeiramente, é permitido utilizar letras maiúsculas e minúsculas do alfabeto inglês (A-Z, a-z), permitindo que os nomes sejam descritivos e representativos. Além disso, a linguagem Swift também suporta o uso de caracteres Unicode, permitindo a inclusão de letras em outros idiomas, símbolos matemáticos e até mesmo emojis.

Além das letras, é possível incluir números (0-9) nos nomes de constantes e variáveis. No entanto, é importante lembrar que o nome não pode iniciar com um número. Para melhorar a legibilidade, é comum utilizar o caractere de sublinhado (`_`), que pode ser usado para separar palavras em nomes compostos, seguindo a convenção conhecida como `snake_case`.

Os programadores têm a liberdade de combinar letras, números e sublinhados para formar nomes significativos. **No entanto, é necessário ter em mente que o primeiro caractere do nome não pode ser um número.** Ao escolher nomes para constantes e variáveis, é fundamental optar por aqueles que sejam descritivos e facilmente compreensíveis. É recomendado seguir as convenções de nomenclatura da linguagem Swift, que geralmente utiliza o `camelCase` para nomes compostos por mais de uma palavra. Nessa convenção, a primeira letra de cada palavra, exceto a primeira, é escrita em maiúscula, o que contribui para uma melhor organização e legibilidade do código.

Nomes de constantes e variáveis **não podem conter espaços em branco, símbolos matemáticos, setas, valores escalares Unicode de uso privado ou caracteres de linha e desenho de caixa.** Também **não podem começar com um número, embora números possam ser incluídos em outros lugares no nome.**

Depois de declarar uma constante ou variável de um determinado tipo, **você não pode declará-la novamente com o mesmo nome ou alterá-la para armazenar valores de um tipo diferente.** Também **não é possível alterar uma constante para uma variável ou uma variável para uma constante.**



Se você precisa dar o mesmo nome de uma palavra-chave reservada do Swift a uma constante ou variável, coloque a palavra-chave entre acentos graves (``) ao usá-la como nome. No entanto, evite usar palavras-chave como nomes, a menos que você não tenha absolutamente nenhuma outra escolha.

Você pode alterar o valor de uma variável existente para outro valor de um tipo compatível. Neste exemplo, o valor de `friendlyWelcome` é alterado de "Olá!" para "Bonjour!":

```
var friendlyWelcome = "Olá!"  
friendlyWelcome = "Bonjour!"  
// friendlyWelcome agora é "Bonjour!"
```

Ao contrário de uma variável, o valor de uma constante não pode ser alterado após ser definido. Tentar fazer isso é relatado como um erro quando o código é compilado:

```
let languageName = "Swift"
```

```
languageName = "Swift++"  
// Isso é um erro de compilação: languageName não pode ser alterado.
```

Imprimindo Constantes e Variáveis

Você pode imprimir o valor atual de uma constante ou variável usando a função `print(_:separator:terminator:)`:

```
print(friendlyWelcome)  
// Imprime "Bonjour!"
```

A função `print(separator:terminator:)` é uma função global que imprime um ou mais valores em uma saída apropriada. No Xcode, por exemplo, a função `print(separator:terminator:)` imprime sua saída no painel "console" do Xcode. O separador e o terminador têm valores padrão, portanto, você pode omiti-los ao chamar essa função. Por padrão, a função termina a linha que imprime adicionando uma quebra de linha. Para imprimir um valor sem uma quebra de linha depois, passe uma string vazia como terminador - por exemplo, `print(someValue, terminator: "")`.

O Swift usa interpolação de string para incluir o nome de uma constante ou variável como um espaço reservado em uma string mais longa e para solicitar ao Swift que o substitua pelo valor atual dessa constante ou variável. Coloque o nome entre parênteses e escape-o com uma barra invertida antes do parêntese de abertura:

```
print("O valor atual de friendlyWelcome é (friendlyWelcome)")  
// Imprime "O valor atual de friendlyWelcome é Bonjour!"
```

A interpolação de strings em Swift permite que valores de variáveis sejam inseridos diretamente em uma string usando a sintaxe `\(valor)`.

Comentários

Use comentários para incluir texto não executável em seu código, como uma nota ou lembrete para si mesmo. Os comentários são ignorados pelo compilador Swift quando o código é compilado. Comentários em Swift são muito semelhantes aos comentários em C. Comentários de uma linha começam com duas barras (`//`):

```
// Este é um comentário.
```

Comentários de várias linhas começam com uma barra seguida de um asterisco (`/`) e terminam com um asterisco seguido de uma barra (`/`):

```
/* Este também é um comentário  
mas é escrito em várias linhas. */
```

Ao contrário dos comentários de várias linhas em C, os comentários de várias linhas em Swift podem ser aninhados dentro de outros comentários de várias linhas. Você pode escrever comentários aninhados iniciando um bloco de comentário de várias linhas e, em seguida,

iniciando um segundo bloco de comentário dentro do primeiro bloco. O segundo bloco é então fechado, seguido pelo primeiro bloco:

```
/* Este é o início do primeiro comentário de várias linhas.  
/* Este é o segundo comentário de várias linhas aninhado. */  
Este é o final do primeiro comentário de várias linhas. */
```

Comentários de várias linhas aninhados permitem que você comente rapidamente blocos grandes de código, mesmo se o código já contiver comentários de várias linhas.

Ponto e vírgula

Ao contrário de muitas outras linguagens, o Swift não exige que você escreva um ponto e vírgula (;) após cada instrução do seu código, embora você possa fazê-lo se quiser. No entanto, pontos e vírgulas são necessários se você quiser escrever várias instruções separadas em uma única linha:

```
let cat = "🐱"; print(cat)  
// Imprime "🐱"
```

Inteiros

Inteiros são números inteiros sem componente fracionário, como 42 e -23. Inteiros podem ser assinados (positivos, zero ou negativos) ou não assinados (positivos ou zero).

O Swift fornece inteiros assinados e não assinados em formas de 8, 16, 32 e 64 bits. Esses inteiros seguem uma convenção de nomenclatura semelhante ao C, em que um inteiro não assinado de 8 bits é do tipo UInt8 e um inteiro assinado de 32 bits é do tipo Int32. Como todos os tipos em Swift, esses tipos de inteiros têm nomes em maiúsculas.

Limites dos Inteiros

Você pode acessar os valores mínimos e máximos de cada tipo de inteiro por meio de suas propriedades min e max:

```
let minValue = UInt8.min // minValue é igual a 0 e é do tipo UInt8  
let maxValue = UInt8.max // maxValue é igual a 255 e é do tipo UInt8
```

Os valores dessas propriedades são do tipo de número do tamanho apropriado (como UInt8 no exemplo acima) e, portanto, podem ser usados em expressões junto com outros valores do mesmo tipo.

Int

Na maioria dos casos, você não precisa escolher um tamanho específico de inteiro para usar em seu código. O Swift fornece um tipo de inteiro adicional, Int, que tem o mesmo tamanho que o tamanho da palavra nativa da plataforma atual:

- Em uma plataforma de 32 bits, Int tem o mesmo tamanho que Int32.
- Em uma plataforma de 64 bits, Int tem o mesmo tamanho que Int64.

A menos que você precise trabalhar com um tamanho específico de inteiro, sempre use `Int` para valores inteiros em seu código. Isso ajuda na consistência e interoperabilidade do código. Mesmo em plataformas de 32 bits, `Int` pode armazenar qualquer valor entre -2.147.483.648 e 2.147.483.647 e é grande o suficiente para muitos intervalos de inteiros.

UInt

O Swift também fornece um tipo de inteiro não assinado, `UInt`, que tem o mesmo tamanho que o tamanho da palavra nativa da plataforma atual:

- Em uma plataforma de 32 bits, `UInt` tem o mesmo tamanho que `UInt32`.
- Em uma plataforma de 64 bits, `UInt` tem o mesmo tamanho que `UInt64`.

Use `UInt` apenas quando você precisa especificamente de um tipo de inteiro não assinado com o mesmo tamanho que a palavra nativa da plataforma. Se esse não for o caso, `Int` é preferido, mesmo quando os valores a serem armazenados são conhecidos por serem não negativos. O uso consistente de `Int` para valores inteiros ajuda na interoperabilidade do código, evita a necessidade de converter entre diferentes tipos de números e corresponde à inferência de tipo de inteiro, conforme descrito em *Segurança de Tipo e Inferência de Tipo*.

Floating-Point Numbers

Números de ponto flutuante são números com uma parte fracionária, como 3.14159, 0.1 e -273.15. Tipos de ponto flutuante podem representar uma faixa muito maior de valores do que os tipos inteiros e podem armazenar números que são muito maiores ou menores do que podem ser armazenados em um `Int`. O Swift fornece dois tipos de números de ponto flutuante com sinal:

- `Double` representa um número de ponto flutuante de 64 bits.
- `Float` representa um número de ponto flutuante de 32 bits.

`Double` tem uma precisão de pelo menos 15 dígitos decimais, enquanto a precisão do `Float` pode ser de apenas 6 dígitos decimais. O tipo de ponto flutuante apropriado a ser usado depende da natureza e da faixa de valores com os quais você precisa trabalhar em seu código. Em situações em que qualquer um dos tipos seria adequado, o `Double` é preferido.

Segurança de Tipo e Inferência de Tipo

O Swift é uma linguagem com segurança de tipo. Uma linguagem com segurança de tipo encoraja você a ser claro sobre os tipos de valores com os quais seu código pode trabalhar. Se uma parte do seu código requer uma `String`, você não pode passar um `Int` por engano.

Como o Swift é seguro em relação a tipos, ele realiza verificações de tipo ao compilar o código e sinaliza quaisquer tipos incompatíveis como erros. Isso permite que você detecte e corrija erros o mais cedo possível no processo de desenvolvimento.

A verificação de tipo ajuda a evitar erros ao trabalhar com diferentes tipos de valores. No entanto, isso não significa que você precise especificar o tipo de todas as constantes e variáveis.

que declara. Se você não especificar o tipo de valor necessário, o Swift usará a inferência de tipo para determinar o tipo apropriado. A inferência de tipo permite que o compilador deduza o tipo de uma expressão específica automaticamente ao compilar seu código, simplesmente examinando os valores fornecidos.

Devido à inferência de tipo, o Swift requer muito menos declarações de tipo do que linguagens como C ou Objective-C. As constantes e variáveis ainda têm um tipo explicitamente declarado, mas grande parte do trabalho de especificar o tipo é feita automaticamente para você.

A inferência de tipo é particularmente útil ao declarar uma constante ou variável com um valor inicial. Isso é frequentemente feito atribuindo um valor literal (ou literal) à constante ou variável no ponto em que você a declara. (Um valor literal é um valor que aparece diretamente em seu código-fonte, como 42 e 3.14159 nos exemplos abaixo.)

Por exemplo, se você atribuir um valor literal de 42 a uma nova constante sem especificar o tipo, o Swift inferirá que você deseja que a constante seja um Int, porque a inicializou com um número que se parece com um número inteiro:

```
let significadoDaVida = 42
// significadoDaVida é inferido como sendo do tipo Int
```

Da mesma forma, se você não especificar um tipo para um literal de ponto flutuante, o Swift inferirá que você deseja criar um Double:

```
let pi = 3.14159
// pi é inferido como sendo do tipo Double
```

O Swift sempre escolhe Double (em vez de Float) ao inferir o tipo de números de ponto flutuante. Se você combinar literais inteiros e de ponto flutuante em uma expressão, um tipo Double será inferido a partir do contexto:

```
let anotherPi = 3 + 0.14159
// anotherPi também é inferido como sendo do tipo Double
```

O valor literal de 3 não tem um tipo explícito em si mesmo, e assim um tipo de saída apropriado de Double é inferido a partir da presença de um literal de ponto flutuante como parte da adição.

Literais Numéricos

Literais inteiros podem ser escritos como:

- Um número decimal, sem prefixo
- Um número binário, com o prefixo 0b
- Um número octal, com o prefixo 0o
- Um número hexadecimal, com o prefixo 0x

Todos esses literais inteiros têm um valor decimal de 17:

```
let decimalInteger = 17
let binaryInteger = 0b10001 // 17 na notação binária
let octalInteger = 0o21 // 17 na notação octal
let hexadecimalInteger = 0x11 // 17 na notação hexadecimal
```

Literais de ponto flutuante podem ser decimais (sem prefixo) ou hexadecimais (com o prefixo 0x). Eles sempre devem ter um número (ou número hexadecimal) em ambos os lados do ponto decimal. Floats decimais também podem ter um expoente opcional, indicado por um 'e' maiúsculo ou minúsculo; floats hexadecimais devem ter um expoente, indicado por um 'p' maiúsculo ou minúsculo.

Para números decimais com um expoente de x, o número base é multiplicado por 10^x:

- 1.25e2 significa 1.25 x 10², ou 125.0.
- 1.25e-2 significa 1.25 x 10⁻², ou 0.0125.

Para números hexadecimais com um expoente de x, o número base é multiplicado por 2^x:

- 0xFp2 significa 15 x 2², ou 60.0.
- 0xFp-2 significa 15 x 2⁻², ou 3.75.

Todos esses literais de ponto flutuante têm um valor decimal de 12.1875:

```
let decimalDouble = 12.1875
let exponentDouble = 1.21875e1
let hexadecimalDouble = 0xC.3p0
```

Literais numéricos podem conter formatação adicional para facilitar a leitura. Tanto os inteiros quanto os floats podem ser preenchidos com zeros extras e podem conter sublinhados para ajudar na legibilidade. Nenhum dos tipos de formatação afeta o valor subjacente do literal:

```
let paddedDouble = 000123.456
let oneMillion = 1_000_000
let justOverOneMillion = 1_000_000.000_000_1
```

Conversão de Tipo Numérico

Ao lidar com conversões de tipos numéricos, utilize o tipo `Int` para todas as constantes e variáveis de inteiros de propósito geral em seu código, mesmo que sejam conhecidas como não negativas. Usar o tipo inteiro padrão em situações cotidianas significa que as constantes e variáveis inteiras são imediatamente interoperáveis em seu código e corresponderão ao tipo inferido para valores literais inteiros.

Utilize outros tipos inteiros somente quando forem especificamente necessários para a tarefa em questão, seja por causa de dados explicitamente dimensionados provenientes de uma fonte externa, desempenho, uso de memória ou outras otimizações necessárias. Usar tipos

explicitamente dimensionados nessas situações ajuda a detectar quaisquer transbordamentos de valor acidentais e documenta implicitamente a natureza dos dados sendo usados.

Conversão de Inteiros

A conversão de inteiros é restrita pela faixa de números que pode ser armazenada em uma constante ou variável inteira de cada tipo numérico. Uma constante ou variável do tipo `Int8` pode armazenar números entre -128 e 127, enquanto uma constante ou variável do tipo `UInt8` pode armazenar números entre 0 e 255. Um número que não se encaixa em uma constante ou variável de um tipo inteiro dimensionado é relatado como um erro durante a compilação do código:

```
let cannotBeNegative: UInt8 = -1
// UInt8 não pode armazenar números negativos, portanto isso gerará um erro
let tooBig: Int8 = Int8.max + 1
// Int8 não pode armazenar um número maior que seu valor máximo,
// portanto isso também gerará um erro
```

Devido a cada tipo numérico poder armazenar uma faixa diferente de valores, é necessário optar pela conversão de tipos numéricos caso a caso. Essa abordagem de optar pela conversão impede erros de conversão ocultos e ajuda a tornar as intenções de conversão de tipos explícitas em seu código.

Para converter um tipo de número específico para outro, você inicializa um novo número do tipo desejado com o valor existente. No exemplo abaixo, a constante `twoThousand` é do tipo `UInt16`, enquanto a constante `one` é do tipo `UInt8`. Elas não podem ser adicionadas diretamente porque não são do mesmo tipo. Em vez disso, este exemplo chama `UInt16(one)` para criar um novo `UInt16` inicializado com o valor de `one` e usa esse valor no lugar do original:

```
let twoThousand: UInt16 = 2_000
let one: UInt8 = 1
let twoThousandAndOne = twoThousand + UInt16(one)
```

Agora que ambos os lados da adição são do tipo `UInt16`, a adição é permitida. A constante de saída (`twoThousandAndOne`) é inferida como sendo do tipo `UInt16`, pois é a soma de dois valores `UInt16`.

`SomeType(ofInitialValue)` é a forma padrão de chamar o inicializador de um tipo Swift e passar um valor inicial. Nos bastidores, `UInt16` possui um inicializador que aceita um valor `UInt8`, então esse inicializador é usado para criar um novo `UInt16` a partir de um `UInt8` existente. No entanto, você não pode passar qualquer tipo aqui, deve ser um tipo para o qual `UInt16` forneça um inicializador.

Conversão entre Inteiro e Ponto Flutuante

As conversões entre tipos numéricos inteiro e ponto flutuante devem ser feitas de forma explícita.

```
let three = 3
let pointOneFourOneFiveNine = 0.14159
```

```
let pi = Double(três) + pointOneFourOneFiveNine
// pi é igual a 3.14159 e é inferido como sendo do tipo Double
```

Aqui, o valor da constante "three" é usado para criar um valor do tipo "Double", de modo que ambos os lados da adição sejam do mesmo tipo. Sem essa conversão, a adição não seria permitida. A conversão de ponto flutuante para inteiro também deve ser feita de forma explícita. Um tipo inteiro pode ser inicializado com um valor "Double" ou "Float". Valores de ponto flutuante são sempre truncados ao serem usados para inicializar um novo valor inteiro dessa forma. Isso significa que 4.75 se torna 4 e -3.9 se torna -3.

Além disso, Swift possui os "type aliases", que permitem definir um nome alternativo para um tipo existente. Esses aliases são úteis quando se deseja se referir a um tipo existente por um nome mais adequado ao contexto. Por exemplo, ao trabalhar com dados de um tamanho específico de uma fonte externa.

```
 typealias AudioSample = UInt16
var maxAmplitudeFound = AudioSample.min
// maxAmplitudeFound is now 0
```

No exemplo acima, AudioSample é definido como um alias para UInt16. Por ser um alias, a chamada para AudioSample.min na verdade chama UInt16.min, que fornece um valor inicial de 0 para a variável maxAmplitudeFound.

Swift também possui um tipo básico chamado "Bool" para representar valores booleanos. Os valores booleanos podem ser "true" (verdadeiro) ou "false" (falso). O tipo "Bool" é particularmente útil ao lidar com declarações condicionais, como a declaração "if". A segurança de tipo do Swift impede que valores não booleanos sejam usados em substituição a um valor "Bool", garantindo a clareza e evitando erros acidentais.

Booleans

O Swift possui um tipo básico chamado Bool para representar valores booleanos. Os valores booleanos são chamados de lógicos, pois só podem ser verdadeiros ou falsos. O Swift fornece duas constantes booleanas, true e false:

```
let orangesAreOrange = true
let turnipsAreDelicious = false
```

Os tipos de orangesAreOrange e turnipsAreDelicious são inferidos como Bool a partir do fato de terem sido inicializados com valores literais booleanos. Assim como acontece com Int e Double mencionados anteriormente, não é necessário declarar constantes ou variáveis como Bool se você as definir como true ou false assim que as criar. A inferência de tipo ajuda a tornar o código Swift mais conciso e legível ao inicializar constantes ou variáveis com outros valores cujo tipo já é conhecido.

Valores booleanos são especialmente úteis quando se trabalha com declarações condicionais, como a declaração if:

```
if turnipsAreDelicious {  
    print("Mmm, tasty turnips!")  
} else {  
    print("Eww, turnips are horrible.")  
}  
// Imprime "Eww, turnips are horrible."
```

A segurança de tipo do Swift impede que valores não booleanos sejam substituídos por Bool. O exemplo a seguir gera um erro de compilação:

```
let i = 1  
if i {  
    // este exemplo não irá compilar e reportará um erro  
}
```

No entanto, o exemplo alternativo abaixo é válido:

```
let i = 1  
if i == 1 {  
    // este exemplo será compilado com sucesso  
}
```

O resultado da comparação `i == 1` é do tipo Bool, e, portanto, o segundo exemplo passa na verificação de tipo. Comparações como `i == 1` são discutidas em Operadores Básicos. Assim como em outros exemplos de segurança de tipo no Swift, essa abordagem evita erros acidentais e garante que a intenção de uma determinada seção de código esteja sempre clara.

Tuplas

Tuplas agrupam múltiplos valores em um único valor composto. Os valores dentro de uma tupla podem ser de qualquer tipo e não precisam ser do mesmo tipo entre si.

```
let http404Error = (404, "Not Found")  
// http404Error is of type (Int, String), and equals (404, "Not Found")
```

Neste exemplo, `(404, "Not Found")` é uma tupla que descreve um código de status HTTP. Um código de status HTTP é um valor especial retornado por um servidor web sempre que você solicita uma página da web. Um código de status 404 Not Found é retornado se você solicitar uma página da web que não existe.

```
let http404Error = (404, "Not Found")  
// http404Error é do tipo (Int, String) e é igual a (404, "Not Found")
```

A tupla `(404, "Not Found")` agrupa um Int e uma String para dar ao código de status HTTP dois valores separados: um número e uma descrição legível para humanos. Pode ser descrita como "uma tupla do tipo `(Int, String)`". Você pode criar tuplas a partir de qualquer permutação de tipos, e elas podem conter quantos tipos diferentes você quiser. Nada impede que você tenha

uma tupla do tipo (Int, Int, Int), ou (String, Bool), ou qualquer outra permutação que você precisar. Você pode decompor o conteúdo de uma tupla em constantes ou variáveis separadas, que você então acessa como de costume:

```
let (statusCode, statusMessage) = http404Error
print("O código de status é (statusCode)")
// Imprime "O código de status é 404"
print("A mensagem de status é (statusMessage)")
// Imprime "A mensagem de status é Not Found"
```

Se você só precisar de alguns valores da tupla, ignore partes da tupla com um sublinhado (_) ao decompor a tupla:

```
let (apenasOStatusCode, _) = http404Error
print("O código de status é (apenasOStatusCode)")
// Imprime "O código de status é 404"
```

Alternativamente, acesse os valores dos elementos individuais em uma tupla usando números de índice a partir de zero:

```
print("O código de status é (http404Error.0)")
// Imprime "O código de status é 404"
print("A mensagem de status é (http404Error.1)")
// Imprime "A mensagem de status é Not Found"
```

Você pode nomear os elementos individuais de uma tupla quando a tupla é definida:

```
let http200Status = (statusCode: 200, description: "OK")
```

Se você nomear os elementos em uma tupla, poderá usar os nomes dos elementos para acessar os valores desses elementos:

```
print("O código de status é (http200Status.statusCode)")
// Imprime "O código de status é 200"
print("A mensagem de status é (http200Status.description)")
// Imprime "A mensagem de status é OK"
```

Tuplas são particularmente úteis como valores de retorno de funções. Uma função que tenta recuperar uma página da web pode retornar o tipo de tupla (Int, String) para descrever o sucesso ou a falha da recuperação da página. Ao retornar uma tupla com dois valores distintos, cada um de um tipo diferente, a função fornece informações mais úteis sobre o resultado do que se pudesse retornar apenas um valor de um único tipo.

Operadores Básicos

Um operador é um símbolo especial ou frase que você usa para verificar, alterar ou combinar valores. Por exemplo, o operador de adição (+) adiciona dois números, como em `let i = 1 + 2`, e o

operador lógico AND (&&) combina dois valores booleanos, como em `if enteredDoorCode && passedRetinaScan`.

Swift suporta os operadores que você pode conhecer de outras linguagens como C e melhora várias capacidades para eliminar erros comuns de codificação. O operador de atribuição (`=`) não retorna um valor, para evitar que seja usado erroneamente quando o operador de igualdade (`==`) é pretendido. Operadores aritméticos (`+`, `-`, `*`, `/`, `%` e assim por diante) detectam e impedem o estouro de valor, para evitar resultados inesperados ao trabalhar com números que se tornam maiores ou menores do que o intervalo de valor permitido pelo tipo que os armazena. Swift também fornece operadores de intervalo que não são encontrados em C, como `a..<b` e `a...b`, como uma forma abreviada de expressar um intervalo de valores.

Operadores de atribuição:

O operador de atribuição (`a = b`) inicializa ou atualiza o valor de `a` com o valor de `b`:

```
let b = 10
var a = 5
a = b
// a agora é igual a 10
```

Se o lado direito da atribuição for uma tupla com vários valores, seus elementos podem ser decompostos em várias constantes ou variáveis de uma vez:

```
let (x, y) = (1, 2)
// x é igual a 1 e y é igual a 2
```

Ao contrário do operador de atribuição em C e Objective-C, o operador de atribuição em Swift não retorna um valor em si. A seguinte declaração não é válida:

```
if x = y {
// Isso não é válido, porque x = y não retorna um valor.
}
```

Essa característica impede que o operador de atribuição (`=`) seja usado por engano quando o operador de igualdade (`==`) é o pretendido. Ao tornar `if x = y` inválido, o Swift ajuda a evitar esse tipo de erro no seu código.

Operadores aritméticos:

Swift suporta os quatro operadores aritméticos padrão para todos os tipos numéricos:

- Adição (`+`)
- Subtração (`-`)
- Multiplicação (`*`)
- Divisão (`/`)

```
1 + 2 // resulta em 3
```

```
5 - 3 // resulta em 2  
2 * 3 // resulta em 6  
10.0 / 2.5 // resulta em 4.0
```

Ao contrário dos operadores aritméticos em C e Objective-C, os operadores aritméticos do Swift não permitem que os valores transbordem por padrão. Você pode optar por ter comportamento de transbordamento de valor usando os operadores de transbordamento do Swift (como a &+ b).

O operador de adição também é suportado para concatenação de Strings:

```
"olá, " + "mundo" // resulta em "olá, mundo"
```

Operadores de atribuição compostos:

Assim como em C, o Swift fornece operadores de atribuição compostos que combinam a atribuição (=) com outra operação. Um exemplo é o operador de atribuição de adição (+=):

```
var a = 1  
a += 2  
// a agora é igual a 3
```

A expressão `a += 2` é uma forma abreviada de `a = a + 2`. Efetivamente, a adição e a atribuição são combinadas em um único operador que realiza ambas as tarefas ao mesmo tempo.

Operadores de comparação:

O Swift suporta os seguintes operadores de comparação:

- Igual a (`a == b`)
- Diferente de (`a != b`)
- Maior que (`a > b`)
- Menor que (`a < b`)
- Maior ou igual a (`a >= b`)
- Menor ou igual a (`a <= b`)

Cada um dos operadores de comparação retorna um valor Bool para indicar se a afirmação é verdadeira ou não:

```
1 == 1 // verdadeiro porque 1 é igual a 1  
2 != 1 // verdadeiro porque 2 é diferente de 1  
2 > 1 // verdadeiro porque 2 é maior que 1  
1 < 2 // verdadeiro porque 1 é menor que 2  
1 >= 1 // verdadeiro porque 1 é maior ou igual a 1  
2 <= 1 // falso porque 2 não é menor ou igual a 1
```

Os operadores de comparação são frequentemente usados em declarações condicionais, como a declaração if:

```
let nome = "mundo"
if nome == "mundo" {
  print("olá, mundo")
} else {
  print("Desculpe, (nome), mas não reconheço você")
}
// Imprime "olá, mundo", porque nome é de fato igual a "mundo".
```

Operador condicional ternário:

O operador condicional ternário é um operador especial com três partes, que assume a forma pergunta ? resposta1 : resposta2. É um atalho para avaliar uma das duas expressões com base em se a pergunta é verdadeira ou falsa. Se a pergunta for verdadeira, ele avalia resposta1 e retorna seu valor; caso contrário, ele avalia resposta2 e retorna seu valor.

O operador condicional ternário é uma forma abreviada para o código abaixo:

```
if pergunta {
  resposta1
} else {
  resposta2
}
```

Aqui está um exemplo que calcula a altura de uma linha de tabela. A altura da linha deve ser 50 pontos maior que a altura do conteúdo se a linha tiver um cabeçalho e 20 pontos maior se a linha não tiver um cabeçalho:

```
let alturaConteudo = 40
let temCabecalho = true
let alturaLinha = alturaConteudo + (temCabecalho ? 50 : 20)
// alturaLinha é igual a 90
```

O exemplo acima é uma forma abreviada para o código abaixo:

```
let alturaConteudo = 40
let temCabecalho = true
let alturaLinha: Int
if temCabecalho {
  alturaLinha = alturaConteudo + 50
} else {
  alturaLinha = alturaConteudo + 20
}
// alturaLinha é igual a 90
```

O uso do operador condicional ternário no primeiro exemplo permite definir o valor de `alturaLinha` em uma única linha de código, o que é mais conciso do que o código usado no segundo exemplo.

O operador condicional ternário fornece uma forma eficiente de decidir qual das duas expressões considerar. No entanto, use-o com cuidado. Sua concisão pode levar a código de difícil leitura se for usado em excesso. Evite combinar várias instâncias do operador condicional ternário em uma única declaração composta.

Operadores lógicos:

Os operadores lógicos modificam ou combinam os valores lógicos booleanos `true` e `false`. Swift suporta os três operadores lógicos padrão encontrados em linguagens baseadas em C:

- Operador lógico NOT (`!`a)
- Operador lógico AND (`a && b`)
- Operador lógico OR (`a || b`)
- Operador lógico NOT

O operador lógico NOT (`!`a) inverte um valor booleano, fazendo com que `true` se torne `false` e `false` se torne `true`. O operador lógico NOT é um operador prefixo e aparece imediatamente antes do valor em que opera, sem nenhum espaço em branco. Ele pode ser lido como "não a", como visto no exemplo a seguir:

```
let entradaPermitida = false
if !entradaPermitida {
    print("ACESSO NEGADO")
}
// Imprime "ACESSO NEGADO"
```

A frase `if !entradaPermitida` pode ser lida como "se não for permitida a entrada". A linha subsequente é executada apenas se "não for permitida a entrada" for verdadeiro, ou seja, se `entradaPermitida` for `false`. Como neste exemplo, a escolha cuidadosa de nomes de constantes e variáveis booleanas pode ajudar a manter o código legível e conciso, evitando duplas negativas ou declarações lógicas confusas.

Operador lógico AND

O operador lógico AND (`a && b`) cria expressões lógicas em que ambos os valores devem ser verdadeiros para que a expressão geral também seja verdadeira. Se qualquer um dos valores for `false`, a expressão geral também será `false`. Na verdade, se o primeiro valor for `false`, o segundo valor nem será avaliado, porque não pode tornar a expressão geral verdadeira. Isso é conhecido como avaliação de circuito curto.

Este exemplo considera dois valores booleanos e permite o acesso apenas se ambos os valores forem `true`:

```
let codigoPortaDigitado = true
```

```
let retinaEscanerAprovado = false
if codigoPortaDigitado && retinaEscanerAprovado {
print("Bem-vindo!")
} else {
print("ACESSO NEGADO")
}
// Imprime "ACESSO NEGADO"
```

Operador lógico OR

O operador lógico OR ($a \parallel b$) é um operador infixo composto por dois caracteres de pipe adjacentes. Ele é usado para criar expressões lógicas em que apenas um dos dois valores precisa ser true para que a expressão geral seja verdadeira.

Assim como o operador lógico AND acima, o operador lógico OR usa a avaliação de circuito curto para considerar suas expressões. Se o lado esquerdo de uma expressão OR for true, o lado direito não será avaliado, porque não pode alterar o resultado da expressão geral.

No exemplo abaixo, o primeiro valor booleano (`possuiChavePorta`) é false, mas o segundo valor (`conheceSenhaAnulacao`) é true. Como um valor é true, a expressão geral também é avaliada como true e o acesso é permitido:

```
let possuiChavePorta = false
let conheceSenhaAnulacao = true
if possuiChavePorta || conheceSenhaAnulacao {
print("Bem-vindo!")
} else {
print("ACESSO NEGADO")
}
// Imprime "Bem-vindo!"
```

Combinação de Operadores Lógicos

Você pode combinar vários operadores lógicos para criar expressões compostas mais longas:

```
if codigoPortaDigitado && retinaEscanerAprovado || possuiChavePorta ||
conheceSenhaAnulacao {
print("Bem-vindo!")
} else {
print("ACESSO NEGADO")
}
// Imprime "Bem-vindo!"
```

Este exemplo usa vários operadores `&&` e `||` para criar uma expressão composta mais longa. No entanto, os operadores `&&` e `||` ainda operam apenas em dois valores, portanto, isso são, na verdade, três expressões menores encadeadas. O exemplo pode ser lido como:

Se tivermos digitado o código da porta correto e passado pela varredura da retina, ou se tivermos uma chave de porta válida, ou se conhecermos a senha de anulação de emergência, permita o acesso.

Com base nos valores de `codigoPortaDigitado`, `retinaEscanerAprovado` e `possuiChavePorta`, as duas primeiras subexpressões são `false`. No entanto, a senha de anulação de emergência é conhecida, portanto, a expressão composta geral ainda é avaliada como `true`.

Strings e Caracteres

Strings e caracteres são usados para armazenar e manipular texto em Swift. Uma string é uma série de caracteres, como "olá, mundo" ou "albatroz". Em Swift, as strings são representadas pelo tipo `String`. O conteúdo de uma string pode ser acessado de várias maneiras, incluindo como uma coleção de valores de `Character`.

Os tipos `String` e `Character` em Swift fornecem uma maneira rápida e compatível com Unicode de trabalhar com texto no seu código. A sintaxe para criação e manipulação de strings é leve e legível, com uma sintaxe literal de string semelhante à linguagem C. A concatenação de strings é tão simples quanto combinar duas strings com o operador `+`, e a mutabilidade da string é gerenciada escolhendo entre uma constante ou uma variável, assim como qualquer outro valor em Swift. Você também pode usar strings para inserir constantes, variáveis, literais e expressões em strings maiores, em um processo conhecido como interpolação de strings. Isso facilita a criação de valores de string personalizados para exibição, armazenamento e impressão.

Apesar da simplicidade da sintaxe, o tipo `String` em Swift é uma implementação moderna e rápida de strings. Cada string é composta por caracteres Unicode independentes de codificação e oferece suporte para acessar esses caracteres em várias representações Unicode.

Para criar uma string vazia como ponto de partida para construir uma string mais longa, você pode atribuir um literal de string vazio a uma variável ou inicializar uma nova instância de `String` usando a sintaxe do inicializador.

Por exemplo, você pode usar a atribuição de literal de string vazia:

```
var emptyString = "" // literal de string vazio
```

Ou você pode usar a sintaxe do inicializador:

```
var anotherEmptyString = String() // sintaxe do inicializador
```

Essas duas strings estão ambas vazias e são equivalentes entre si. Você pode verificar se uma string está vazia verificando sua propriedade Booleana `isEmpty`. Por exemplo, você pode usar um bloco condicional para imprimir uma mensagem se a string estiver vazia:

```
if emptyString.isEmpty {  
    print("Nada para ver aqui")  
}
```

```
// Imprime "Nada para ver aqui"
```

Você pode indicar se uma determinada String pode ser modificada (ou mutada) atribuindo-a a uma variável (na qual pode ser modificada) ou a uma constante (na qual não pode ser modificada). Por exemplo, se você atribuir uma String a uma variável, você pode modificá-la posteriormente:

```
var variableString = "Cavalo"  
variableString += " e carruagem"  
// variableString agora é "Cavalo e carruagem"
```

Por outro lado, se você atribuir uma String a uma constante, não poderá modificá-la posteriormente:

```
let constantString = "Highlander"  
constantString += " e outro Highlander"  
// isso gera um erro de compilação - uma string constante não pode ser modificada
```

A String em Swift é um tipo de valor. Isso significa que quando você cria um novo valor de String, esse valor é copiado quando é passado para uma função ou método, ou quando é atribuído a uma constante ou variável. Em cada caso, é criada uma nova cópia do valor de String existente, e a nova cópia é passada ou atribuída, não a versão original. Os tipos de valor são descritos em Estruturas e Enumerações São Tipos de Valor.

O comportamento de cópia-por-padrão da String em Swift garante que, quando uma função ou método passa um valor de String, fica claro que você possui exatamente aquele valor de String, independentemente de sua origem. Você pode ter a confiança de que a string que lhe foi passada não será modificada, a menos que você a modifique.

Nos bastidores, o compilador do Swift otimiza o uso de strings para que a cópia real ocorra apenas quando for absolutamente necessário. Isso significa que você sempre obtém um ótimo desempenho ao trabalhar com strings como tipos de valor.

Valores de String podem ser adicionados (ou concatenados) usando o operador de adição (+) para criar um novo valor de String:

```
let string1 = "olá"  
let string2 = " mundo"  
var saudacao = string1 + string2  
// saudacao agora é "olá mundo"
```

Você também pode adicionar um valor de String a uma variável de String existente usando o operador de atribuição com adição (+=):

```
var instrucao = "dê uma olhada"  
instrucao += string2  
// instrucao agora é "dê uma olhada mundo"
```

Você pode anexar um valor de Character a uma variável de String usando o método `append()` do tipo String:

```
let pontoExclamacao: Character = "!"  
saudacao.append(pontoExclamacao)  
// saudacao agora é "olá mundo!"
```

Interpolação de Strings

A interpolação de strings em Swift permite construir uma nova String a partir de uma mistura de constantes, variáveis, literais e expressões, incluindo seus valores dentro de uma string literal. É possível usar a interpolação de strings tanto em literais de string de uma linha como em literais de string de várias linhas. Cada item inserido na string literal é envolto por um par de parênteses, precedido por uma barra invertida (`\()`):

Exemplo 1: Interpolação de variáveis e expressões

```
let nome = "João"  
let idade = 25  
let saudacao = "Olá, meu nome é \("\(nome)\) e tenho \("\(idade)\) anos."  
// saudacao é "Olá, meu nome é João e tenho 25 anos."
```

Neste exemplo, as variáveis `nome` e `idade` são interpoladas na string literal usando a sintaxe `\()`. Seus valores são inseridos na string resultante.

Exemplo 2: Interpolação de expressões e chamadas de função

```
let a = 10  
let b = 5  
let soma = "A soma de \("\(a)\) e \("\(b)\) é \("\(a + b)\)"  
// soma é "A soma de 10 e 5 é 15"  
  
func multiplicar(_ x: Int, _ y: Int) -> Int {  
    return x * y  
}  
  
let resultado = "O resultado da multiplicação de \("\(a)\) por \("\(b)\) é  
\("\(multiplicar(a, b))\)"  
// resultado é "O resultado da multiplicação de 10 por 5 é 50"
```

Neste exemplo, as expressões `a + b` e `multiplicar(a, b)` são avaliadas e seus resultados são interpolados nas literais de string.

Exemplo 3: Interpolação de valores booleanos

```
let estaChovendo = true  
let statusTempo = "Está \("\(estaChovendo ? "chovendo" : "sem chuva")\) hoje."
```

```
// statusTempo é "Está chovendo hoje."
```

Neste exemplo, o valor booleano `estaChovendo` é usado em um operador condicional ternário dentro da interpolação de string. A string resultante depende do valor de `estaChovendo`.

Exemplo 4: Interpolação de strings de várias linhas

```
let nomeUsuario = "johndoe"  
let mensagem = ""  
    Bem-vindo, \(nomeUsuario)!  
    Obrigado por se juntar à nossa comunidade.  
    Esperamos que você aproveite sua estadia.  
    ""  
print(mensagem)
```

Saída:

```
Bem-vindo, johndoe!  
Obrigado por se juntar à nossa comunidade.  
Esperamos que você aproveite sua estadia.
```

Neste exemplo, é usada uma literal de string de várias linhas. A variável `nomeUsuario` é interpolada na string, e a string resultante abrange várias linhas. Vejamos agora uma questão didática sobre interpolação de strings.

(CESGRANRIO – Banco do Brasil / 2023) Em um programa em Swift, o programador deseja incluir o resultado de uma operação dentro de uma string. Nesse contexto, considere o seguinte código:

```
let quantidade = 4 let valor = 10
```

Dado o código acima, o programador deseja uma string saída cujo valor seja

```
"valor total = 40"
```

Para isso, o programador deve utilizar o seguinte fragmento de código Swift:

- a) `let saida := "valor total = \(quantidade*valor)"`
- b) `let saida := "valor total = \{quantidade*valor\}"`
- c) `let saida = "valor total = %[quantidade*valor]"`
- d) `let saida = "valor total = \[quantidade*valor]"`
- e) `let saida = "valor total = \[quantidade*valor]"`

Comentários:

Para incluir o resultado de uma operação dentro de uma string em Swift, o programador deve utilizar interpolação de strings. Dada a declaração das variáveis `quantidade` com valor 4 e `valor` com valor 10, o programador deseja criar uma string chamada `saida` com o valor "valor total =

40". A interpolação de strings em Swift permite que valores de variáveis sejam inseridos diretamente em uma string usando a sintaxe `\(valor)`. Dessa forma, a expressão dentro dos parênteses será avaliada e seu resultado será incluído na string.

Analisando as opções apresentadas:

a) `let saida := "valor total = \(quantidade*valor)"`

O resultado da operação `quantidade*valor` será calculado e inserido na string. No entanto, há um erro de sintaxe, pois usa `:=` em vez de `=` para atribuição de valor a uma variável.

b) `let saida := "valor total = \{quantidade*valor\}"`

Essa opção utiliza uma sintaxe incorreta. A expressão dentro das chaves não será avaliada e não produzirá o resultado esperado.

c) `let saida = "valor total = %[quantidade*valor]"`

Essa opção utiliza uma sintaxe incorreta para interpolação de strings em Swift. O uso de `%` não é adequado nesse contexto.

d) `let saida = "valor total = \(quantidade*valor)"`

Essa opção utiliza a sintaxe correta de interpolação de strings em Swift. O resultado da operação `quantidade*valor` será calculado e inserido na string. Portanto, essa é a opção correta para obter o resultado desejado.

e) `let saida = "valor total = \[quantidade*valor]"`

Essa opção utiliza uma sintaxe incorreta. O uso de colchetes não é adequado para interpolação de strings em Swift.

Portanto, a opção correta é a letra d): `let saida = "valor total = \(quantidade*valor)"`. Essa opção cria a string `saida` com o valor desejado `"valor total = 40"`, incluindo o resultado da operação de multiplicação `quantidade*valor` dentro da string. (Gabarito: Letra D)

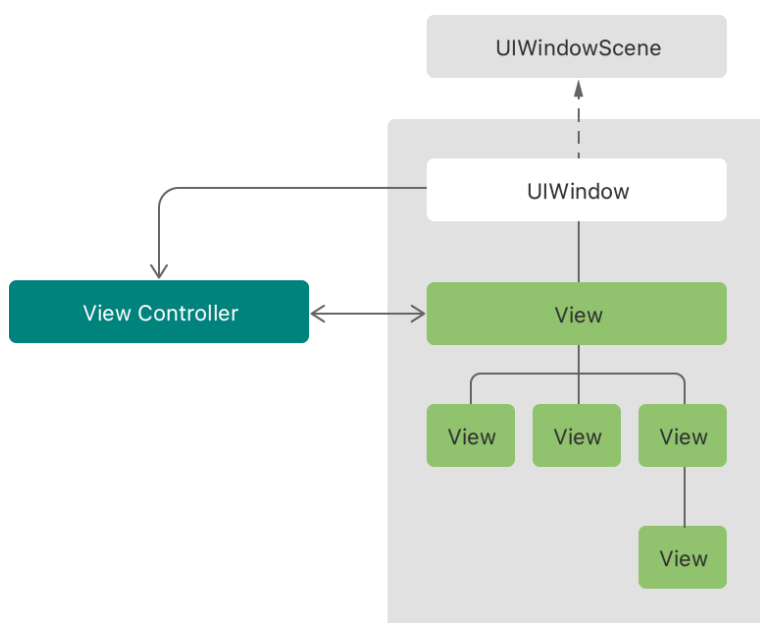
View controllers

Os view controllers são componentes essenciais para gerenciar a interface do seu aplicativo UIKit, facilitando a navegação pelo conteúdo. Vamos explorar os detalhes dos view controllers e seus aspectos técnicos, utilizando uma linguagem clara e concisa, com o auxílio de conjunções subordinativas para tornar o assunto mais didático.

Um view controller é responsável por gerenciar uma única visualização raiz dentro da interface do seu aplicativo. Essa visualização raiz pode conter qualquer número de subvisualizações, permitindo uma organização hierárquica. Consequentemente, o view controller se torna a entidade central para lidar com as interações do usuário que ocorrem dentro dessa hierarquia de visualização. Ele coordena com outros objetos do seu aplicativo conforme necessário, a fim de fornecer uma experiência do usuário perfeita. É importante ressaltar que todo aplicativo deve ter pelo menos um view controller cujo conteúdo preenche a janela principal. Se o seu aplicativo possuir mais conteúdo do que pode ser exibido de uma só vez, utilize vários view controllers para gerenciar diferentes partes desse conteúdo.

Um container view controller incorpora o conteúdo de outros view controllers em sua própria visualização raiz. Um container view controller pode misturar visualizações personalizadas com o conteúdo de seus view controllers filhos para facilitar a navegação ou criar interfaces únicas. Por exemplo, um objeto UINavigationController gerencia uma barra de navegação e uma pilha de view controllers filhos (dos quais apenas um é visível por vez) e fornece uma API para adicionar e remover view controllers filhos da pilha.

O UIKit fornece vários view controllers padrão para navegação e gerenciamento de tipos específicos de conteúdo. Você define os view controllers que contêm o conteúdo personalizado do seu aplicativo. Você também pode definir container view controllers personalizados para implementar novos esquemas de navegação.



Você pode construir a interface do usuário do seu aplicativo utilizando view controllers e alterar o view controller atualmente visível sempre que desejar exibir novo conteúdo. Cada cena na interface do seu aplicativo é composta por uma janela e uma ou mais visualizações. A janela atua como um contêiner invisível para as visualizações, roteando eventos e servindo como o nível superior da hierarquia das visualizações. As visualizações são responsáveis por exibir o conteúdo real na tela, como texto, imagens e outros elementos personalizados.

Para gerenciar as visualizações de forma eficaz e facilitar a adição e remoção delas da janela, o UIKit oferece os view controllers. Um view controller gerencia um conjunto específico

de visualizações para o seu aplicativo e mantém essas visualizações atualizadas. Cada janela possui um view controller raiz, que é responsável por definir o conjunto inicial de visualizações. Quando você deseja alterar esse conjunto, basta informar ao UIKit para apresentar ou descartar outros view controllers. O UIKit cuida da transição entre os conjuntos de visualizações e gerencia toda a interface do seu aplicativo por meio dos view controllers.

Ao projetar a interface do usuário do seu aplicativo, é importante dividir a interface em páginas distintas de conteúdo. Cada página representa uma maneira única de exibir informações personalizadas do seu aplicativo. Por exemplo, uma página pode exibir dados em forma de tabelas, outra pode exibir imagens em um layout de grid e outra pode fornecer campos de texto para a captura de dados. O foco principal é na estrutura e aparência de cada página, e não nos dados específicos exibidos. É comum usar o mesmo tipo de página em diferentes partes do aplicativo, preenchendo cada uma delas com diferentes conjuntos de dados.

Para cada página distinta, defina um view controller para representar essa página e gerenciar suas visualizações correspondentes. Definir um view controller sempre envolve a criação de uma subclasse e a adição de comportamento personalizado. Todo view controller precisa realizar o seguinte:



Tabular interface

Grid interface

Data-entry interface

- Preencher suas visualizações com dados e atualizar essas visualizações quando os dados mudarem.
- Informar as alterações nos dados de volta aos objetos do seu modelo de dados.
- Ajustar o tamanho, posição e visibilidade das visualizações para corresponder ao ambiente atual.

- Facilitar transições para outras páginas de conteúdo.

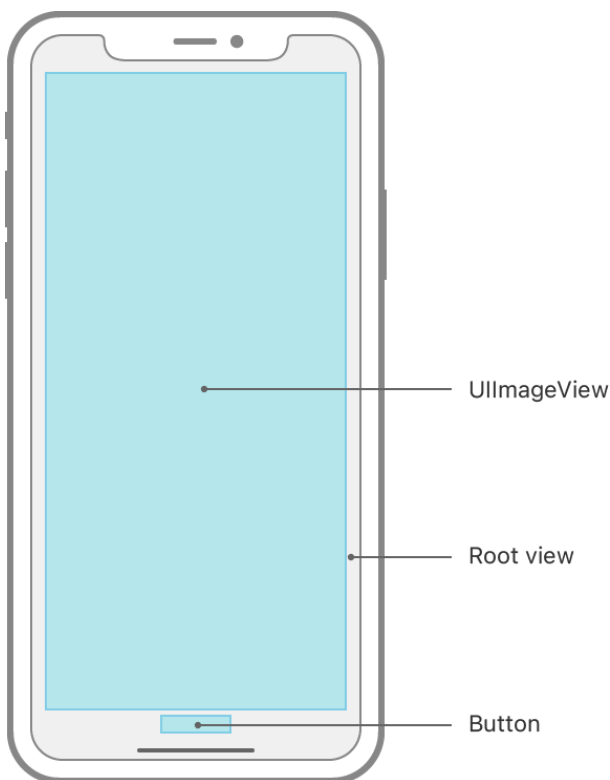
Apresentando e gerenciando visualizações com um view controller

No paradigma de design model-view-controller, um view controller se encaixa entre os objetos de visualização que apresentam informações na tela e os objetos de dados que armazenam o conteúdo do seu aplicativo. Especificamente, um view controller gerencia uma hierarquia de visualizações e as informações de estado necessárias para manter essas visualizações atualizadas. Todo aplicativo UIKit depende muito dos view controllers para apresentar conteúdo, e você frequentemente define view controllers personalizados para gerenciar suas visualizações e lógica relacionada à interface do usuário.

A maioria dos view controllers personalizados que você cria são view controllers de conteúdo - ou seja, o view controller possui todas as suas visualizações e gerencia as interações com essas visualizações. Use view controllers de conteúdo para apresentar o conteúdo personalizado do seu aplicativo na tela e use o objeto do seu view controller para gerenciar a transferência de dados de e para suas visualizações personalizadas.

Para definir um view controller de conteúdo, comece criando uma subclasse de UIViewController. Se a sua interface incluir uma tabela ou uma coleção de visualizações, crie uma subclasse de UITableViewController ou UICollectionViewController, respectivamente. Novos projetos no Xcode já incluem uma ou mais classes de view controllers de conteúdo para você modificar, e você pode adicionar mais.

Adicione visualizações ao seu view controller



UIViewController contém uma visualização de conteúdo, acessível pela propriedade `view`, que serve como a visualização raiz de sua hierarquia de visualizações. Nessa visualização raiz, você adiciona as visualizações personalizadas necessárias para apresentar sua interface. Em storyboards, você adiciona visualizações arrastando-as para a cena do view controller. Por exemplo, a figura a seguir mostra um view controller com uma visualização de imagem e um botão em um iPhone.

Depois de adicionar visualizações a um view controller, sempre adicione restrições de Auto Layout para definir o tamanho e a posição dessas visualizações. Restrições são regras que especificam como dimensionar e posicionar cada visualização em relação à sua visualização pai ou irmã, e elas garantem que suas visualizações se adaptem automaticamente a diferentes ambientes e dispositivos.

Armazene referências para visualizações importantes

Durante a execução, você pode precisar acessar visualizações através do código do seu view controller. Por exemplo, você pode querer obter o texto em uma text view ou alterar a imagem em uma image view. Para isso, você precisa de uma referência à visualização em sua hierarquia de visualizações. Você cria essas referências usando outlets.

Um outlet é uma propriedade em seu view controller que inclui a palavra-chave IBOutlet. A presença dessa palavra-chave informa ao Xcode para expor essa propriedade no seu storyboard. O código de exemplo a seguir mostra as definições para dois outlets. Em Swift, inclua a palavra-chave weak para evitar que seu view controller mantenha uma segunda referência forte à visualização - a primeira origina-se da própria hierarquia de visualizações.

```
@IBOutlet weak var imageView : UIImageView?  
@IBOutlet weak var button : UIButton?
```

No seu storyboard, conecte cada outlet à visualização correspondente, conforme descrito em Adicionar uma conexão de outlet para enviar uma mensagem a um objeto de interface do usuário. Você não precisa armazenar referências para todas as visualizações em sua hierarquia de visualizações; armazene referências apenas para as visualizações que você modificar posteriormente.

Quando você instanciar um view controller, o UIKit reconecta quaisquer outlets que você configurou em seu storyboard. O UIKit reestabelece essas conexões antes de chamar o método viewDidLoad() do seu view controller, para que você possa acessar os objetos nessas propriedades a partir desse método. Se você criar visualizações programaticamente, deverá atribuir explicitamente essas visualizações às propriedades apropriadas do seu view controller.

Lidar com eventos ocorrendo em visualizações e controles

Os controles usam o padrão de design alvo-ação para relatar interações do usuário, e algumas visualizações enviam notificações ou chamam métodos de delegado em resposta a alterações. Um view controller precisa saber sobre muitas dessas interações para poder atualizar suas visualizações, e você tem várias maneiras de fazer isso:

- Implemente métodos de delegado e ação em seu view controller. Essa opção é simples e fácil de implementar, mas oferece menos flexibilidade e torna mais difícil testar e validar seu código.
- Implemente métodos de delegado e ação em uma extensão de classe em seu view controller. Essa opção separa seu código de tratamento de eventos do restante do seu view controller, facilitando o teste e a validação desse código.
- Implemente métodos de delegado e ação em objetos dedicados que, em seguida, encaminham informações relevantes para seu view controller. Essa opção oferece mais

flexibilidade e reutilização. A separação de responsabilidades também torna mais fácil escrever testes unitários.

Para responder a interações do usuário com controles, defina um método de ação com uma das assinaturas mostradas no código a seguir. Em sua definição de método, você pode substituir a referência genérica a UIControl por uma classe de controle mais específica.

```
@IBAction func fazerAlgo()  
@IBAction func fazerAlgo(sender: UIControl)  
@IBAction func fazerAlgo(sender: UIControl, forEvent event: UIEvent)
```

Prepare suas visualizações para aparecerem na tela

O UIKit oferece várias oportunidades para configurar seu view controller e visualizações antes de exibi-los na tela. Quando você instancia seu view controller a partir de um storyboard, o UIKit cria esse objeto usando seu método `init(coder:)`.

Ao apresentar um view controller na tela, o UIKit precisa primeiro carregar e configurar as visualizações correspondentes, o que ele faz seguindo a seguinte sequência de etapas:

1. Cria cada visualização usando o método `init(coder:)` da visualização
2. Conecta visualizações às ações e outlets correspondentes no view controller
3. Chama o método `awakeFromNib()` de cada visualização e do view controller
4. Atribui a hierarquia de visualizações à propriedade `view` do view controller
5. Chama o método `viewDidLoad()` do view controller

No momento do carregamento, realize apenas as etapas de configuração única necessárias para preparar o view controller para uso. Use o tempo de carregamento para criar e configurar quaisquer visualizações adicionais que não façam parte do seu storyboard. Não execute tarefas que precisam ocorrer cada vez que seu view controller aparece na tela. Por exemplo, não inicie animações ou atualize os valores das visualizações.

Execute quaisquer tarefas finais relacionadas às visualizações quando elas aparecerem pela primeira vez na tela. O UIKit notifica o view controller proprietário quando suas visualizações estão aparecendo na tela e atualiza o layout dessas visualizações para se adequar ao ambiente atual da seguinte maneira:

1. Chama o `viewWillAppear(_)` no início da transição
2. Adiciona a visualização à hierarquia
3. Atualiza as coleções de traits do view controller e de sua visualização
4. Atualiza a geometria da visualização, incluindo seu tamanho e posição em sua visualização pai. Atualiza as margens de layout e a área segura, e chama `viewLayoutMarginsDidChange()` e `viewSafeAreaInsetsDidChange()`, se necessário
5. Chama o método `viewWillAppear(_)` para informar que a visualização do view controller está aparecendo na tela
6. Chama o método `viewWillLayoutSubviews()`
7. Atualiza o layout da hierarquia de visualizações
8. Chama o método `viewDidLayoutSubviews()`

9. Exibe as visualizações na tela
10. Chama o método `viewDidAppear(_:)` do view controller após a conclusão de quaisquer animações

Atualize o conteúdo de suas visualizações no método `viewWillAppear(_:)` do seu view controller. Quando o sistema chama esse método, ele já adicionou a visualização à hierarquia de visualizações e definiu a estrutura, limites, margens e recortes. O conteúdo que o sistema adiciona em `viewWillAppear(_:)` é exibido quando a visualização é visível pela primeira vez na tela.

Exibindo e ocultando controladores de exibição

Ao exibir e ocultar controladores de exibição, você pode utilizar diferentes técnicas e passar dados entre eles durante as transições. Para alterar a interface do seu aplicativo, você pode apresentar e fechar controladores de exibição. Cada janela possui um controlador de exibição raiz, que fornece o conteúdo inicial para a janela. Ao apresentar um novo controlador de exibição, o conteúdo é alterado ao instalar um novo conjunto de visualizações na janela. Quando não há mais necessidade do controlador de exibição, ao fechá-lo, suas visualizações são removidas da janela. Existem três maneiras de apresentar controladores de exibição:

1. Configurar apresentações visualmente em seu storyboard:

Essa técnica envolve o uso de "segues"¹ em seu storyboard, sendo a abordagem recomendada para apresentar e fechar controladores de exibição. Um "segue" representa uma transição visual de um controlador de exibição para outro. Um "segue" é iniciado por uma ação, como um toque em um botão ou a seleção de uma linha em uma tabela, no controlador de exibição inicial. Quando essa ação ocorre, o UIKit cria automaticamente o controlador de exibição no final do "segue" e o apresenta. Como você cria e configura os "segues" em seu storyboard, é possível alterá-los rapidamente.

2. Incorporá-los em um controlador de exibição de contêiner:

Essa técnica envolve colocar um controlador de exibição dentro de outro controlador de exibição, conhecido como controlador de exibição de contêiner. O controlador de exibição de contêiner gerencia a apresentação e ocultação dos controladores de exibição incorporados. Essa abordagem oferece um alto grau de controle sobre o processo de apresentação e ocultação.

3. Chamar métodos diretamente da classe `UIViewController`:

Essa técnica envolve chamar métodos diretamente da classe `UIViewController` para apresentar e ocultar controladores de exibição.

Por exemplo, você pode chamar o método `present(_:animated:completion:)` para apresentar um controlador de exibição modally.

Essa abordagem oferece o maior nível de controle, permitindo que você personalize completamente o processo de apresentação e ocultação.

¹ Um "segue" é uma representação visual de uma transição de um controlador de exibição para outro. Ele define a forma como os controladores de exibição estão conectados e como a transição entre eles ocorre. No contexto do storyboard, um "segue" é criado entre dois controladores de exibição para definir um fluxo de navegação ou uma transição específica. Por exemplo, ao tocar em um botão em um controlador de exibição, você pode definir um "segue" para apresentar outro controlador de exibição relacionado.

Cada uma dessas técnicas oferece diferentes níveis de controle sobre o processo de apresentação e ocultação de controladores de exibição.

Especificar apresentações visualmente em seu arquivo de storyboard

Ao especificar apresentações visualmente em seu arquivo de storyboard, você utiliza "segues" para apresentar e ocultar controladores de exibição. Um "segue" é uma representação visual de uma transição de um controlador de exibição para outro. Um "segue" é iniciado por uma ação, como um toque em um botão ou a seleção de uma linha em uma tabela, no controlador de exibição inicial. Quando essa ação ocorre, o UIKit cria automaticamente o controlador de exibição no final do "segue" e o apresenta. Por criar e configurar os "segues" no storyboard, você pode alterá-los facilmente.

Para iniciar um "segue", você pode partir de qualquer objeto que implemente um método de ação, como um controle ou um reconhecedor de gestos. Além disso, é possível iniciar "segues" a partir de linhas de tabela e células de visualização em coleção.



1. Para começar, clique com o botão direito do mouse no controle ou objeto no controlador de exibição atual.
2. Arraste o cursor até o controlador de exibição que você deseja apresentar.
3. Selecione o tipo de "segue" desejado na lista fornecida pelo Xcode.

No storyboard, as "segues" são mostradas como uma seta entre dois controladores de exibição. Ao selecionar a "segue", serão exibidas informações sobre ela, incluindo o tipo de apresentação que você deseja que o UIKit realize. É possível modificar o tipo de apresentação ou configurar detalhes adicionais, como um identificador para a "segue". Essas informações são utilizadas em tempo de execução para personalizar ainda mais a "segue", conforme descrito em "Personalizando o comportamento de apresentações baseadas em segues".

UIViewController

A classe UIViewController define o comportamento compartilhado que é comum a todos os controladores de exibição. Raramente se cria instâncias da classe UIViewController diretamente. Em vez disso, você cria uma subclasse de UIViewController e adiciona os métodos e propriedades necessários para gerenciar a hierarquia de visualização do controlador de exibição.

As principais responsabilidades de um controlador de exibição incluem:

- Atualizar o conteúdo das visualizações, geralmente em resposta a mudanças nos dados subjacentes.
- Responder às interações do usuário com as visualizações.

- Redimensionar as visualizações e gerenciar o layout da interface geral.
- Coordenar com outros objetos, incluindo outros controladores de exibição, em seu aplicativo.

Um controlador de exibição está intimamente ligado às visualizações que ele gerencia e participa do tratamento de eventos em sua hierarquia de visualização. Especificamente, os controladores de exibição são objetos UIResponder e são inseridos na cadeia de resposta entre a visualização raiz do controlador de exibição e a visualização pai dessa visualização, que normalmente pertence a um controlador de exibição diferente. Se nenhuma das visualizações do controlador de exibição tratar um evento, o controlador de exibição tem a opção de tratar o evento ou repassá-lo para a visualização pai.

Os controladores de exibição raramente são usados isoladamente. Em vez disso, frequentemente são usados vários controladores de exibição, cada um deles possuindo uma parte da interface do usuário do seu aplicativo. Por exemplo, um controlador de exibição pode exibir uma tabela de itens enquanto um controlador de exibição diferente exibe o item selecionado dessa tabela. Geralmente, apenas as visualizações de um controlador de exibição são visíveis por vez. Um controlador de exibição pode apresentar um controlador de exibição diferente para exibir um novo conjunto de visualizações, ou pode atuar como um contêiner para o conteúdo de outros controladores de exibição e animar as visualizações conforme necessário.

UITableViewController

Ao criar uma interface que consiste principalmente em uma tabela e possui pouco ou nenhum outro conteúdo, você deve criar uma subclasse de UITableViewController. Os controladores de exibição de tabela (UITableViewController) já adotam os protocolos necessários para gerenciar o conteúdo da tabela e responder a mudanças. Além disso, UITableViewController implementa os seguintes comportamentos:

Carrega automaticamente a tabela arquivada em um storyboard ou arquivo NIB, que pode ser acessada usando a propriedade tableView.

Define o data source e o delegate da tabela como o próprio controlador.

Implementa o método viewWillAppear(_:), recarregando automaticamente os dados da tabela em sua primeira aparição e limpando a seleção (com ou sem animação, dependendo da solicitação) toda vez que a tabela é exibida (esse comportamento pode ser desabilitado alterando o valor de clearsSelectionOnViewWillAppear).

Implementa o método viewDidAppear(_:), e piscará automaticamente os indicadores de rolagem da tabela quando ela aparecer pela primeira vez.

Implementa o método setEditing(_:animated:), alternando automaticamente o modo de edição da tabela quando o usuário toca em um botão Editar/Concluir na barra de navegação.

Redimensiona automaticamente a tabela para acomodar a aparição ou desaparecimento do teclado na tela.

Crie uma subclasse personalizada de `UITableViewController` para cada tabela que você gerencia. Ao inicializar o controlador de exibição da tabela, você deve especificar o estilo da tabela (plain ou grouped). Você também deve substituir os métodos do data source e do delegate necessários para preencher sua tabela com dados. Você pode substituir o método `loadView()` ou qualquer outro método da superclasse, mas se fizer isso, certifique-se de invocar a implementação da superclasse do método, geralmente como a primeira chamada de método.

UICollectionViewController

O controlador de exibição implementa o seguinte comportamento:

- Se o controlador de exibição de coleção tiver um arquivo NIB atribuído ou tiver sido carregado de um storyboard, ele carrega sua exibição a partir do arquivo NIB ou storyboard correspondente. Se você criar o controlador de exibição de coleção programaticamente, ele cria automaticamente um novo objeto de coleção não configurado, que pode ser acessado usando a propriedade `collectionView`.
- Ao carregar uma coleção de exibição de um storyboard ou arquivo NIB, os objetos de origem de dados (data source) e delegado (delegate) para a coleção de exibição são obtidos do arquivo NIB. Se um data source ou delegate não for especificado, o controlador de exibição de coleção atribui a si mesmo para o papel não especificado.
- Quando a coleção de exibição está prestes a aparecer pela primeira vez, o controlador de exibição de coleção recarrega os dados da coleção de exibição. Ele também limpa a seleção atual toda vez que a visualização é exibida. Você pode alterar esse comportamento definindo o valor da propriedade `clearsSelectionOnViewWillAppear` como `false`.

Você cria uma subclasse personalizada de `UICollectionViewController` para cada coleção de exibição que deseja gerenciar. Ao inicializar o controlador, usando o método `init(collectionViewLayout:)`, você especifica o layout que a coleção de exibição deve ter. Como a coleção de exibição inicialmente criada está sem dimensões ou conteúdo, o data source e delegate da coleção de exibição - geralmente o próprio controlador de exibição de coleção - devem fornecer essas informações.

Você pode substituir o método `loadView()` ou qualquer outro método da superclasse, mas se fizer isso, certifique-se de chamar `super` na implementação do seu método. Se não fizer isso, o controlador de exibição de coleção pode não ser capaz de executar todas as tarefas necessárias para manter a integridade da coleção de exibição.

UICollectionContainer

Os métodos deste protocolo lidam com transições relacionadas ao tamanho que estão relacionadas a alterações no ambiente de características atual ou na hierarquia do controlador de exibição. Quando o controlador de exibição pai muda, ou quando ocorrem alterações de características que afetam o tamanho de um controlador de exibição, o UIKit chama esses métodos para dar às objetos afetados a oportunidade de responder adequadamente.

Todos os objetos `UIViewController` e `UIPresentationController` fornecem implementações padrão para os métodos deste protocolo. Ao criar seu próprio controlador de exibição personalizado ou controlador de apresentação, você pode substituir as implementações padrão para fazer ajustes em seu conteúdo. Por exemplo, você pode usar esses métodos para ajustar o tamanho ou a posição de quaisquer controladores de exibição secundários.

Ao substituir os métodos deste protocolo, chame `super` para permitir que o `UIKit` execute quaisquer comportamentos padrão. Controladores de exibição e controladores de apresentação realizam seus próprios ajustes quando esses métodos são chamados. Chamar `super` garante que o `UIKit` possa continuar ajustando outras partes da interface do usuário.

UISplitViewController

Um controlador de divisão (`split view controller`) é um controlador de visualização (`view controller`) de contêiner que gerencia controladores de visualização secundários em uma interface hierárquica. Nesse tipo de interface, as alterações em um controlador de visualização influenciam o conteúdo de outro.

As interfaces de divisão são mais adequadas para conteúdos filtráveis ou para navegar em hierarquias de conteúdo, como percorrer pastas e notas dentro do aplicativo Notas para visualizar cada nota. No aplicativo Notas, ao selecionar uma pasta na barra lateral principal, são exibidas a lista de notas dentro dessa pasta e, ao selecionar uma nota na lista, são exibidos os conteúdos específicos dessa nota na visualização secundária.

UINavigationController

O `UINavigationController` é um `view controller` de contêiner que gerencia um ou mais `view controllers` filhos em uma interface de navegação. Nesse tipo de interface, apenas um `view controller` filho é visível de cada vez. Ao selecionar um item no `view controller`, um novo `view controller` é exibido na tela utilizando uma animação, ocultando o `view controller` anterior. Ao tocar no botão de voltar na barra de navegação no topo da interface, o `view controller` superior é removido, revelando assim o `view controller` subjacente.

Utilize uma interface de navegação para simular a organização de dados hierárquicos gerenciados pelo seu aplicativo. Em cada nível da hierarquia, você fornece uma tela apropriada (gerenciada por um `view controller` personalizado) para exibir o conteúdo daquele nível. A imagem a seguir mostra um exemplo da interface de navegação apresentada pelo aplicativo de Configurações no iOS Simulator. A primeira tela apresenta ao usuário a lista de aplicativos que contêm preferências. Ao selecionar um aplicativo, são reveladas as configurações individuais e grupos de configurações para aquele aplicativo. Ao selecionar um grupo, mais configurações são exibidas, e assim por diante. Para todos os `views`, exceto o `view` raiz, o `UINavigationController` fornece um botão de voltar para permitir que o usuário retorne na hierarquia.

UINavigationController

Um objeto `UINavigationController` é uma barra, geralmente exibida no topo da janela, contendo botões para navegação dentro de uma hierarquia de telas. Os componentes principais são um botão esquerdo (voltar), um título central e um botão opcional direito. Você pode usar uma barra de

navegação como um objeto independente ou em conjunto com um objeto de controlador de navegação (navigation controller).

UINavigationController

Ao construir uma interface de navegação, cada view controller que você adiciona à pilha de navegação deve ter um objeto UINavigationController que contém os botões e as visualizações que você deseja exibir na barra de navegação. O objeto UINavigationController gerenciador usa os itens de navegação dos dois view controllers mais acima na pilha para preencher a barra de navegação com conteúdo.

Um item de navegação sempre reflete informações sobre o view controller associado a ele. O item de navegação deve fornecer um título para exibir quando o view controller estiver no topo da pilha de navegação. O item também pode conter botões adicionais para exibir no lado direito da barra de navegação. Você pode especificar botões e visualizações para exibir no lado esquerdo da barra de navegação usando a propriedade leftBarButtonItems, mas o navigation controller exibe esses botões apenas quando houver espaço disponível.

A propriedade backButtonItem de um item de navegação reflete o botão "Voltar" que você deseja exibir quando o view controller atual estiver logo abaixo do view controller mais acima. O botão "Voltar" não aparece quando o view controller atual está no topo.

UITabBarController

A interface de barra de guias exibe guias na parte inferior da janela para selecionar entre os diferentes modos e para exibir as visualizações para esse modo. Essa classe é geralmente usada como está, mas também pode ser subclassificada.

Cada guia de uma interface de barra de guias está associada a um view controller personalizado. Quando o usuário seleciona uma guia específica, o barra de guias exibe a visualização raiz do view controller correspondente, substituindo quaisquer visualizações anteriores. (Os toques do usuário sempre exibem a visualização raiz da guia, independentemente de qual guia foi selecionada anteriormente. Isso é verdade mesmo se a guia já tiver sido selecionada.) Como selecionar uma guia substitui o conteúdo da interface, o tipo de interface gerenciada em cada guia não precisa ser semelhante de forma alguma. Na verdade, as interfaces de barra de guias são comumente usadas para apresentar diferentes tipos de informações ou para apresentar as mesmas informações usando um estilo de interface completamente diferente.

UITabBar

Normalmente, você usa barras de guias em conjunto com um objeto UITabBarController, mas também pode usá-las como controles independentes em seu aplicativo. As barras de guias sempre aparecem na parte inferior da tela e exibem o conteúdo de um ou mais objetos UITabBarItem. A aparência de uma barra de guias pode ser personalizada com uma imagem de plano de fundo ou cor de destaque para atender às necessidades da sua interface. Toque em um item para selecioná-lo e realçá-lo, e você usa a seleção do item para habilitar o modo correspondente do seu aplicativo.

Você pode configurar barras de guias programaticamente ou no Interface Builder. Um objeto `UITabBarController` fornece seu próprio objeto de barra de guias e você deve configurar o objeto fornecido. Ao criar uma barra de guias programaticamente, use o método `init(frame:)` ou outro método de inicialização de visualização para definir sua configuração inicial. Use os métodos desta classe para configurar a aparência da barra de guias. Para barras de guias criadas por você mesmo, você também usa os métodos desta classe para especificar os itens exibidos pela barra de guias.

UIPageViewController

O controlador de visualização de página (`UIPageViewController`) permite a navegação por páginas, que pode ser controlada programaticamente pelo seu aplicativo ou diretamente pelo usuário usando gestos. Ao navegar de página em página, o controlador de visualização de página usa a transição que você especificar para animar a mudança. Ao definir uma interface de controlador de visualização de página, você pode fornecer os controladores de visualização de conteúdo um de cada vez (ou dois de cada vez, dependendo da posição da lombada e do estado de duas faces) ou conforme necessário usando um data source. Ao fornecer os controladores de visualização de conteúdo um de cada vez, você usa o método `setViewControllers(_:direction:animated:completion:)` para definir os controladores de visualização de conteúdo atuais. Para oferecer suporte à navegação baseada em gestos, você deve fornecer seus controladores de visualização usando um objeto data source.

O data source para um controlador de visualização de página é responsável por fornecer os controladores de visualização de conteúdo sob demanda e deve seguir o protocolo `UIPageViewControllerDataSource`. O objeto de delegado - um objeto que segue o protocolo `UIPageViewControllerDelegate` - fornece algumas informações relacionadas à aparência e recebe notificações sobre transições iniciadas por gestos.

Ufa! Umuuuuito conteúdo não é mesmo? Fiz uma tabela para ajudá-los a memorizar.

VIEW CONTROLLER	DESCRIÇÃO
UIVIEWCONTROLLER	Um objeto que gerencia uma hierarquia de visualização para o seu aplicativo UIKit.
UITABLEVIEWCONTROLLER	Um view controller que se especializa em gerenciar uma tabela.
UICOLLECTIONVIEWCONTROLLER	Um view controller que se especializa em gerenciar uma collection view.
UICONTENTCONTAINER	Um conjunto de métodos para adaptar o conteúdo dos seus view controllers a alterações de tamanho e traits.

UISPLITVIEWCONTROLLER	Um view controller de contêiner que gerencia a navegação entre páginas de conteúdo, onde cada página é gerenciada por um child view controller.
UINavigationController	Um view controller de navegação que gerencia a pilha de view controllers, permitindo a transição entre eles.
UINavigationController	Uma barra que normalmente é exibida na parte superior da janela, contendo botões para navegação em uma hierarquia de telas.
UINavigationControllerItem	Um objeto que contém botões e views a serem exibidos na barra de navegação, associado a um view controller.
UITABBarController	Um view controller de abas que exibe abas na parte inferior da janela e gerencia a exibição de views para cada aba selecionada.
UITABBAR	Uma barra exibida na parte inferior da janela que contém os itens de cada aba do UITabBarController.
UIPAGEVIEWCONTROLLER	Um view controller de contêiner que gerencia a navegação entre páginas de conteúdo, onde cada página é gerenciada por um child view controller.

Protocolos

Imagine que você está criando uma receita para fazer um bolo. Essa receita é um protocolo. Ele define um conjunto de etapas e ingredientes necessários para fazer o bolo, sem se importar com quem vai realmente fazer o bolo. Agora, você dá essa receita para diferentes pessoas, como um chef de confeitaria profissional, um amador entusiasta da cozinha e uma criança aprendendo a cozinhar. Cada uma dessas pessoas é um tipo diferente, com habilidades e conhecimentos diferentes, mas todas podem seguir a receita.

O protocolo em Swift funciona de maneira semelhante. Ele define um conjunto de requisitos, como métodos e propriedades, que devem ser implementados por qualquer tipo que adote o protocolo. É como uma lista de coisas a serem feitas. O tipo que adota o protocolo é como a pessoa que segue a receita. Ao adotar um protocolo, um tipo concorda em fornecer uma implementação para todos os requisitos do protocolo. Essa implementação pode variar de acordo com o tipo. Por exemplo, um protocolo pode ter um requisito de um método chamado

"preparar" que retorna uma string. Um tipo que adote o protocolo pode implementar esse método de forma diferente, dependendo do seu propósito.

Além de especificar requisitos que os tipos devem implementar, você também pode estender um protocolo para fornecer implementações padrão para alguns requisitos ou para adicionar funcionalidades extras que os tipos adotantes podem aproveitar. O protocolo define um contrato que especifica quais propriedades e métodos devem ser implementados pelas classes ou estruturas que estão em conformidade com o protocolo.

Um dos protocolos mais comumente utilizados na linguagem de programação Swift é o protocolo **Equatable**. Esse protocolo permite determinar se dois objetos são iguais em termos de valor. Ao adotar esse protocolo, uma classe ou estrutura deve implementar a função `static func ==(lhs: Self, rhs: Self) -> Bool`, que compara dois objetos do mesmo tipo e retorna um valor booleano indicando se eles são iguais.

Além disso, com as extensões condicionais a esse protocolo, é possível adicionar funcionalidades específicas para tipos de objetos específicos em conformidade com o protocolo **Equatable**. Isso significa que é possível estender as capacidades de comparação de igualdade para tipos personalizados, além dos tipos básicos fornecidos pela biblioteca padrão do Swift.

Essa flexibilidade oferecida pelos protocolos e pelas extensões condicionais permite que os desenvolvedores personalizem o comportamento dos seus tipos de acordo com as necessidades específicas de seus projetos.

Tipos que estão em conformidade com o protocolo **Equatable** podem ser comparados para igualdade usando o operador de igualdade (`==`) ou para desigualdade usando o operador de não igualdade (`!=`). A maioria dos tipos básicos na biblioteca padrão do Swift está em conformidade com o protocolo **Equatable**.

Algumas operações de sequência e coleção podem ser utilizadas de forma mais simples quando os elementos estão em conformidade com o protocolo **Equatable**. Por exemplo, para verificar se um array contém um valor específico, você pode passar o próprio valor para o método `contains(:)` quando o elemento do array estiver em conformidade com **Equatable**, em vez de fornecer um closure que determine a equivalência. O seguinte exemplo mostra como o método `contains(:)` pode ser usado com um array de strings.

REFERÊNCIAS

Kotlin for Android Developers

Antonio Leiva. Kotlin for Android Developers: Learn Kotlin the Easy Way While Developing an Android App. 2ª Edição, 2018.

Kotlin Language Documentation on Wikipedia

Wikipedia. Kotlin (programming language). Disponível em: <https://pt.wikipedia.org/wiki/Kotlin>

Kotlin by Example

JetBrains. Kotlin by Example. Disponível em: <https://kotlinlang.org/docs/examples.html>

Kotlin in Action

Dmitry Jemerov, Svetlana Isakova. Kotlin in Action. 2017.

Android Manifest Documentation

Google. *Manifesto do Android: Introdução ao Manifesto*. Disponível em: <https://developer.android.com/guide/topics/manifest/manifest-intro?hl=pt-br>

Temok Blog

Temok. *Blog de Hospedagem e Marketing Online*. Disponível em: <https://www.temok.com/blog/>

Android Enterprise Documentation

Google. *Documentação Android Enterprise*. Disponível em: <https://www.androidenterprise.dev/s/>

Android Studio Publishing

Google. *Publicação no Android Studio*. Disponível em: <https://developer.android.com/studio/publish?hl=pt-br>

Apple Developer Documentation

Apple. *Documentação do Apple Developer*. Disponível em: <https://developer.apple.com/documentation/>

Flutter Official Website

Flutter. *Flutter: Beautiful native apps in record time*. Disponível em: <https://flutter.dev>

Flutter Wikipedia

Wikipedia. *Flutter*. Disponível em: <https://pt.wikipedia.org/wiki/Flutter>

Flutter API Documentation

Flutter. *Documentação da API Flutter*. Disponível em: <https://api.flutter.dev>

Swift Documentation

Apple. *The Swift Programming Language Documentation*. Disponível em: <https://docs.swift.org/swift-book/documentation/the-swift-programming-language/>

Flutter Docs

Flutter. *Flutter Documentation*. Disponível em: <https://docs.flutter.dev>

QUESTÕES COMENTADAS

Swift

1. (CESGRANRIO – Banco do Brasil / 2023) Em um programa em Swift, o programador deseja incluir o resultado de uma operação dentro de uma string. Nesse contexto, considere o seguinte código:

```
let quantidade = 4 let valor = 10
```

Dado o código acima, o programador deseja uma string saída cujo valor seja

```
"valor total = 40"
```

Para isso, o programador deve utilizar o seguinte fragmento de código Swift:

- a) `let saida := "valor total = \((quantidade*valor))"`
- b) `let saida := "valor total = \{quantidade*valor\}"`
- c) `let saida = "valor total = %[quantidade*valor]"`
- d) `let saida = "valor total = \((quantidade*valor))"`
- e) `let saida = "valor total = \[quantidade*valor]"`

Comentários:

Para incluir o resultado de uma operação dentro de uma string em Swift, o programador deve utilizar interpolação de strings. Dada a declaração das variáveis `quantidade` com valor 4 e `valor` com valor 10, o programador deseja criar uma string chamada `saida` com o valor "valor total = 40". A interpolação de strings em Swift permite que valores de variáveis sejam inseridos diretamente em uma string usando a sintaxe `\(valor)`. Dessa forma, a expressão dentro dos parênteses será avaliada e seu resultado será incluído na string.

Analisando as opções apresentadas:

- a) `let saida := "valor total = \((quantidade*valor))"`

Essa opção utiliza a sintaxe correta de interpolação de strings em Swift. O resultado da operação `quantidade*valor` será calculado e inserido na string. No entanto, há um erro de sintaxe, pois usa `:=` em vez de `=` para atribuição de valor a uma variável.

- b) `let saida := "valor total = \{quantidade*valor\}"`

Essa opção utiliza uma sintaxe incorreta. A expressão dentro das chaves não será avaliada e não produzirá o resultado esperado.

c) `let saida = "valor total = %[quantidade*valor]"`

Essa opção utiliza uma sintaxe incorreta para interpolação de strings em Swift. O uso de % não é adequado nesse contexto.

d) `let saida = "valor total = \(quantidade*valor)"`

Essa opção utiliza a sintaxe correta de interpolação de strings em Swift. O resultado da operação `quantidade*valor` será calculado e inserido na string. Portanto, essa é a opção correta para obter o resultado desejado.

e) `let saida = "valor total = \[quantidade*valor]"`

Essa opção utiliza uma sintaxe incorreta. O uso de colchetes não é adequado para interpolação de strings em Swift.

Portanto, a opção correta é a letra d): `let saida = "valor total = \(quantidade*valor)"`. Essa opção cria a string `saida` com o valor desejado `"valor total = 40"`, incluindo o resultado da operação de multiplicação `quantidade*valor` dentro da string.

Gabarito: Letra D

2. (AOCP – PRODEB / 2018) Considerando as linguagens de programação mobile, qual das dispostas a seguir foi criada pela Apple e pode ser utilizada para o desenvolvimento das suas aplicações?

- a) Kotlin
- b) Swift
- c) C#
- d) Python
- e) IOS

Comentários:

A linguagem de programação Swift foi criada pela Apple e é utilizada para o desenvolvimento de aplicativos em diversas plataformas da empresa, como iOS, macOS, tvOS e watchOS. Swift é uma linguagem moderna, poderosa e de fácil utilização, projetada para oferecer uma experiência de desenvolvimento eficiente e segura. Com Swift, os desenvolvedores podem criar aplicativos para dispositivos Apple com facilidade e expressividade.

Gabarito: Letra B

3. (CEBRASPE – TRE-BA / 2017) Na linguagem Swift do IOS, ao se declarar o código `var fruta=["maça", "banana", "abacaxi"]`, a linguagem automaticamente entenderá que `fruta` é array de

- a) integer.
- b) floatings.
- c) strings.
- d) imutáveis.
- e) double.

Comentários:

Em linguagem Swift do iOS, ao se declarar o código `var fruta=["maça", "banana", "abacaxi"]`, a linguagem automaticamente entenderá que `fruta` é um array de strings. Isso ocorre porque os elementos estão entre aspas duplas, o que indica que são do tipo string. Na linguagem Swift, para definir uma string, basta colocar o valor desejado entre aspas, não sendo mais necessário o uso do "@" como era feito anteriormente na Objective C.

Uma string é uma sequência especial de caracteres. Em C, elas são colocadas entre aspas duplas. Por exemplo, "oba" e "teste\n" são exemplos de strings, sendo o caractere de escape "\n" utilizado para representar uma quebra de linha, comumente usado no printf. É importante não confundir o uso de aspas simples ('x') e aspas duplas ("x"). Em C, 'x' representa o caractere 'x', enquanto "x" representa a string contendo o caractere 'x'. A diferença é que uma string é representada em C como um vetor de caracteres terminado pelo caractere nulo '\0'. Portanto, a única diferença entre um vetor de caracteres e uma string é a presença obrigatória do caractere nulo no final da string.

Ao escrever uma string na tela usando o printf, utiliza-se o formato %s para indicar que se trata de uma string. Por exemplo, `printf("Minha string é: %s", "string");` imprimirá "Minha string é: string". O valor entre as aspas duplas é considerado uma string pela linguagem Swift e, ao utilizá-lo como argumento para o printf, é tratado como uma string e exibido corretamente.

Gabarito: Letra C

4. (IF-SE – IF-SE / 2016) Em relação aos ambientes de desenvolvimento de software, analise as afirmativas abaixo.

- I. O XCode é o ambiente de desenvolvimento da Apple e permite trabalhar com as linguagens Swift e C#.
- II. O ambiente Eclipse suporta diversas linguagens de programação, tais como, Java, C/C++, AspectJ e PHP.
- III. O ambiente NetBeans é gratuito, porém com código fechado.
- IV. O Visual Studio suporta diversas linguagens de programação, tais como, C#, C++, F#, Python e Visual Basic.

De acordo com as afirmativas, marque a alternativa CORRETA:

- a) As afirmativas II e III estão corretas.
- b) Apenas as afirmativas II e IV estão corretas.
- c) Apenas a afirmativa I está incorreta
- d) Apenas a afirmativa III está incorreta.

Comentários:

I. A afirmativa é incorreta. O XCode é o ambiente de desenvolvimento integrado (IDE) da Apple, porém, suporta principalmente a linguagem Swift para desenvolvimento de aplicativos iOS, macOS, watchOS e tvOS. Não possui suporte nativo para a linguagem C#, mas é possível utilizar o framework Xamarin para desenvolver aplicativos em C# usando o XCode.

II. A afirmativa é correta. O ambiente Eclipse é uma IDE de código aberto e extensível, que oferece suporte a várias linguagens de programação, incluindo Java, C/C++, AspectJ e PHP. É amplamente utilizado no desenvolvimento de software em diferentes áreas.

III. A afirmativa é incorreta. O ambiente NetBeans é uma IDE de código aberto e também gratuito. Diferente do que foi mencionado, o código do NetBeans é aberto e pode ser acessado, modificado e distribuído livremente.

IV. A afirmativa é correta. O Visual Studio é uma IDE desenvolvida pela Microsoft e suporta diversas linguagens de programação, como C#, C++, F#, Python e Visual Basic. É amplamente utilizado no desenvolvimento de aplicativos e software em geral.

Portanto, a alternativa correta é a letra b) "Apenas as afirmativas II e IV estão corretas", pois as afirmativas II e IV estão corretas, enquanto as afirmativas I e III estão incorretas.

Gabarito: Letra B

5. (AOCP – IBGE / 2019) Compreender o ciclo de vida das views das aplicações é extremamente importante, sobretudo quando falamos de aplicações para dispositivos móveis. Sobre o ciclo de vida das aplicações iOS com Swift, assinale a alternativa que apresenta um método que é chamado toda vez que uma visão vai aparecer na tela, podendo ser chamado mais de uma vez, e é muito usado para acionar quaisquer operações que precisem ocorrer antes que a ViewController seja apresentada na tela, como atualizar os dados do usuário.

- a) viewDidLoad()
- b) viewWillAppear()
- c) viewWillAppear()
- d) viewWillLoad()
- e) viewRestart()

Comentários:

O ciclo de vida das views refere-se à sequência de eventos que ocorrem desde a criação até a destruição de uma view em um aplicativo. Dentre as opções fornecidas, o método que é chamado toda vez que uma visão vai aparecer na tela, podendo ser chamado mais de uma vez, e é amplamente utilizado para acionar operações antes que a ViewController seja apresentada na tela, como a atualização dos dados do usuário, é o `viewWillAppear()`. O `viewWillAppear()` é um método pertencente à ViewController, e é invocado quando a exibição está prestes a ser apresentada na hierarquia da tela. Ele é um ponto de entrada onde é possível realizar tarefas de configuração e atualização dos elementos visuais da interface antes que a view seja exibida ao usuário. Dessa forma, é comum utilizar esse método para atualizar dados do usuário ou qualquer outra operação necessária antes da exibição da ViewController.

Gabarito: Letra C

6. (AOCP – IBGE / 2019) Você está desenvolvendo um aplicativo iOS usando Swift, que é uma agenda de controle de tarefas do funcionário do departamento de Tecnologia da Informação que presta o serviço de manutenção e suporte para os usuários da corporação. Nesse momento do desenvolvimento, é preciso fornecer uma maneira para nosso usuário sair da listagem de tarefas e ir para a tela de nova tarefa, ou seja, é necessário trabalhar com a navegação entre telas, pois o usuário precisa navegar entre a tela de listagem e a de nova tarefa, tanto a ida quanto a volta. Para isso, é necessário ter uma barra de navegação. Assinale a alternativa que apresenta o que você deve utilizar para implementar essa ação.

- a) NavController
- b) Navigation Jetpacks
- c) Navigation Component
- d) Navigation Controller
- e) Routing Navigation

Comentários:

Ao desenvolver um aplicativo iOS usando Swift, que se trata de uma agenda de controle de tarefas para funcionários do departamento de Tecnologia da Informação, é necessário implementar a navegação entre telas para permitir que o usuário saia da listagem de tarefas e vá para a tela de criação de uma nova tarefa, assim como retornar da tela de nova tarefa para a listagem. Para realizar essa navegação, é necessário utilizar uma barra de navegação. Dentre as opções fornecidas, a alternativa correta é a D - Navigation Controller.

O Navigation Controller é um controlador de navegação que gerencia um ou mais controladores de visualização filhos em uma interface de navegação. Nesse tipo de interface, apenas um controlador de visualização filho é visível por vez. Ao selecionar um item no controlador de visualização, um novo controlador de visualização é apresentado na tela usando uma animação, ocultando o controlador de visualização anterior. Ao tocar no botão de retorno na barra de navegação no topo da interface, o controlador de visualização superior é removido, revelando o controlador de visualização abaixo. Utiliza-se uma interface de navegação para reproduzir a organização de dados

hierárquicos gerenciados pelo aplicativo. Em cada nível da hierarquia, você fornece uma tela apropriada (gerenciada por um controlador de visualização personalizado) para exibir o conteúdo daquele nível. O Navigation Controller fornece um botão de retorno para permitir que o usuário retorne à hierarquia anterior, exceto para a visualização raiz.

Gabarito: Letra D

7. (AOCP – IBGE / 2019) Com o swift no desenvolvimento para iOS, a Apple adotou novas características e capacidades para a linguagem de programação, como o uso de protocolos. Estes trabalham de uma maneira que visa estender a funcionalidade de uma classe ou estrutura existente. Um protocolo pode ser pensado como um escopo ou interface que define um conjunto de propriedades e métodos. Um dos protocolos mais utilizados nessa linguagem de programação é o que tem a capacidade de determinar quando dois objetos são iguais e, com extensões condicionais a esse protocolo, é possível fornecer funcionalidade específica para tipos específicos de objetos em conformidade com um protocolo. Assinale a alternativa que apresenta corretamente o nome desse protocolo.

- a) Equatable.
- b) CollectionType.
- c) ViewController.
- d) NSCoder.
- e) MyDelegate.

Comentários:

O protocolo correto para determinar quando dois objetos são iguais é o protocolo Equatable. Esse protocolo permite comparar dois valores para igualdade. Quando um tipo está em conformidade com o protocolo Equatable, é possível usar o operador igual a (==) para verificar se dois objetos são iguais em termos de valor, ou o operador diferente (!=) para verificar se eles são diferentes. A maioria dos tipos básicos fornecidos pela biblioteca padrão do Swift já estão em conformidade com o protocolo Equatable.

Gabarito: Letra A

8. (CESGRANRIO – Banco do Brasil / 2021) Um programador de aplicativos para dispositivos Apple com iOS recebeu a seguinte parte de um código, escrito na linguagem swift:

```
var i: Int
var texto: String
var num: Int = 0
var frase: String = ""

for i in 1...3 {
```

```
num = num + 1 + i * 2
switch num {
case 2...6:
    texto = "a "
case 7...9:
    texto = "casa "
case 10...13:
    texto = "carro "
case 14...16:
    texto = "eh "
case 17...20:
    texto = "o "
case 21...23:
    texto = "forte "
default:
    texto = "não eh "
}
frase = frase + texto
}

print(frase)
```

A execução dessa parte do código produz como resposta

- a) o carro eh
- b) a casa eh
- c) o carro não eh
- d) a casa eh forte
- e) o carro eh forte

Comentários:

O algoritmo apresentado é um exemplo de código escrito em Swift, uma linguagem de programação utilizada para desenvolvimento de aplicativos para dispositivos Apple com iOS. Vamos analisar o código e entender sua execução:

1. São declaradas quatro variáveis: i do tipo Int, texto do tipo String, num inicializada com o valor 0 e frase inicializada como uma string vazia.
2. É iniciado um loop for que itera no intervalo de 1 a 3 (inclusive). A cada iteração, o valor de i é atualizado para o próximo número do intervalo.

3. Dentro do loop, a variável num é atualizada com a expressão $\text{num} = \text{num} + 1 + i * 2$. Essa expressão realiza um cálculo baseado no valor atual de i, incrementando num em 1 mais o dobro de i.
4. Em seguida, é feito um switch na variável num. Dependendo do valor de num, a variável texto é atualizada com diferentes strings. Os intervalos no switch definem faixas de valores possíveis para num e associam uma string específica para cada faixa.
5. A linha `frase = frase + texto` concatena a variável texto à variável frase, formando uma frase gradualmente.
6. Após o fim do loop, a frase final é impressa na tela através do comando `print(frase)`.

Analisando o código, podemos identificar que a sequência de valores de num ao longo do loop é a seguinte: 4, 9, 16. Com base nisso, podemos ver qual string será atribuída à variável texto em cada iteração:

- 1ª iteração: num = 4, que se enquadra no intervalo 2...6, então texto é atualizado para "a".
2ª iteração: num = 9, que se enquadra no intervalo 7...9, então texto é atualizado para "casa".
3ª iteração: num = 16, que se enquadra no intervalo 14...16, então texto é atualizado para "eh".

Portanto, a frase final será "a casa eh". Agora, vamos analisar as opções de resposta:

- a) "o carro eh": Essa opção está errada porque a sequência de valores de num não atinge o intervalo correspondente a essa opção.
- b) "a casa eh": Essa opção está correta, como explicado anteriormente.
- c) "o carro não eh": Essa opção está errada porque a string associada a num = 16 é "eh", não "não eh".
- d) "a casa eh forte": Essa opção está errada porque a sequência de valores de num não atinge o intervalo correspondente a essa opção.
- e) "o carro eh forte": Essa opção está errada porque a sequência de valores de num não atinge o intervalo correspondente a essa opção.

Em linguagem natural e conjunções subordinativas, podemos dizer que o código apresentado é um loop que itera três vezes. A cada iteração, o valor de num é calculado com base no valor atual de i. Em seguida, dependendo do valor de num, uma determinada string é atribuída à variável texto. Essa string é concatenada à variável frase a cada iteração. Ao final do loop, a frase completa é exibida na tela. No caso específico dos valores de num apresentados, a resposta correta seria "a casa eh".

Gabarito: Letra B

9. (FEPESE – CELESC / 2022) Caso seja necessário o desenvolvimento de aplicativos para dispositivos móveis, mais especificamente para o sistema operacional IOS, assinale a alternativa

que apresenta corretamente uma linguagem de programação e um ambiente de desenvolvimento que podem ser utilizados para este fim.

- a) Swift e XCODE
- b) Swift e Objective-D
- c) Objective-D e iOSCODE
- d) Objective-D e VSCODE
- e) XCODE e VSCODE

Comentários:

Se a necessidade for desenvolver aplicativos para dispositivos móveis, especialmente para o sistema operacional iOS, a combinação adequada de linguagem de programação e ambiente de desenvolvimento seria Swift e Xcode.

A linguagem de programação Swift é uma linguagem moderna e poderosa, desenvolvida pela Apple, especificamente para o desenvolvimento de aplicativos iOS, macOS, watchOS e tvOS. Ela possui uma sintaxe concisa e segura, oferecendo recursos avançados para construir aplicativos robustos.

O ambiente de desenvolvimento Xcode é a ferramenta oficial da Apple para criar aplicativos para seus dispositivos. Ele fornece uma interface integrada para escrever, depurar e testar código Swift, além de oferecer recursos adicionais, como emuladores de dispositivos, gerenciamento de recursos e suporte a bibliotecas e frameworks.

Portanto, a alternativa correta é "A) Swift e Xcode" como a combinação adequada para o desenvolvimento de aplicativos para dispositivos iOS.

Gabarito: Letra A

10.(CEBRASPE – TRT - 7ª Região (CE) / 2017) Assinale a opção que apresenta a linguagem de programação disponível, grátis e em código aberto, para desenvolvedores sob a licença Apache 2.0 e desenvolvida pela Apple para a criação de aplicativos para IOS.

- a) Xcode
- b) Swift
- c) Objective-C
- d) Python

Comentários:

A opção correta é "Swift". A linguagem de programação Swift é disponibilizada gratuitamente e é de código aberto, o que significa que seu código-fonte é aberto e acessível para desenvolvedores. A Apple desenvolveu a linguagem Swift para a criação de aplicativos para iOS, macOS, watchOS e

tvOS. Além disso, a linguagem Swift é licenciada sob a licença Apache 2.0, que é uma licença de software livre e de código aberto. Essa licença permite que os desenvolvedores utilizem, modifiquem e distribuam a linguagem Swift de acordo com os termos da licença. Portanto, a resposta correta é "Swift".

Gabarito: Letra B

11. (AOCP – PRODEB / 2018) Considerando as linguagens de programação mobile, qual das dispostas a seguir foi criada pela Apple e pode ser utilizada para o desenvolvimento das suas aplicações?

- a) Kotlin
- b) Swift
- c) C#
- d) Python
- e) IOS

Comentários:

A linguagem de programação mobile criada pela Apple e que pode ser utilizada para o desenvolvimento de suas aplicações é o Swift. Desenvolvida pela Apple, o Swift é uma linguagem moderna, poderosa e de alto nível, projetada especificamente para o desenvolvimento de aplicativos para os sistemas operacionais iOS, macOS, watchOS e tvOS. Com o Swift, os desenvolvedores podem escrever código de maneira eficiente, segura e expressiva, aproveitando os recursos e frameworks oferecidos pela plataforma da Apple.

Gabarito: Letra B

12. (IBADE – Prefeitura de Itapemirim - ES / 2019) A Apple desenvolveu uma linguagem de programação própria para desenvolvimento de aplicações sob IOS. Ela se chama:

- a) C++.
- b) PHP.
- c) Python.
- d) Swift.
- e) Java Script.

Comentários:

De fato, a Apple desenvolveu sua própria linguagem de programação para o desenvolvimento de aplicativos iOS, e essa linguagem é chamada Swift. O Swift foi lançado pela Apple em 2014 e rapidamente ganhou popularidade devido à sua sintaxe concisa, facilidade de uso e segurança. Ele foi projetado para substituir a Objective-C como a principal linguagem de programação para o

desenvolvimento de aplicativos Apple. Uma das principais motivações por trás do desenvolvimento do Swift foi fornecer aos desenvolvedores uma linguagem mais moderna, com recursos avançados e uma curva de aprendizado mais suave. Com o Swift, os desenvolvedores podem criar aplicativos iOS eficientes e poderosos, aproveitando as amplas bibliotecas e recursos disponíveis na plataforma da Apple. Portanto, o Swift é a linguagem de programação desenvolvida pela Apple especificamente para o desenvolvimento de aplicativos iOS.

Gabarito: Letra D

LISTA DE QUESTÕES

Swift

1. **(CESGRANRIO – Banco do Brasil / 2023)** Em um programa em Swift, o programador deseja incluir o resultado de uma operação dentro de uma string. Nesse contexto, considere o seguinte código:

```
let quantidade = 4 let valor = 10
```

Dado o código acima, o programador deseja uma string saída cujo valor seja

```
"valor total = 40"
```

Para isso, o programador deve utilizar o seguinte fragmento de código Swift:

- a) `let saida := "valor total = \((quantidade*valor))"`
- b) `let saida := "valor total = \{quantidade*valor\}"`
- c) `let saida = "valor total = %[quantidade*valor]"`
- d) `let saida = "valor total = \((quantidade*valor))"`
- e) `let saida = "valor total = \[quantidade*valor]"`

2. **(AOCP – PRODEB / 2018)** Considerando as linguagens de programação mobile, qual das dispostas a seguir foi criada pela Apple e pode ser utilizada para o desenvolvimento das suas aplicações?

- a) Kotlin
- b) Swift
- c) C#
- d) Python
- e) IOS

3. **(CEBRASPE – TRE-BA / 2017)** Na linguagem Swift do IOS, ao se declarar o código `var fruta=["maça", "banana", "abacaxi"]`, a linguagem automaticamente entenderá que fruta é array de

- a) integer.
- b) floatings.
- c) strings.
- d) imutáveis.
- e) double.

4. (IF-SE – IF-SE / 2016) Em relação aos ambientes de desenvolvimento de software, analise as afirmativas abaixo.

I. O XCode é o ambiente de desenvolvimento da Apple e permite trabalhar com as linguagens Swift e C#.

II. O ambiente Eclipse suporta diversas linguagens de programação, tais como, Java, C/C++, AspectJ e PHP.

III. O ambiente NetBeans é gratuito, porém com código fechado.

IV. O Visual Studio suporta diversas linguagens de programação, tais como, C#, C++, F#, Python e Visual Basic.

De acordo com as afirmativas, marque a alternativa CORRETA:

- a) As afirmativas II e III estão corretas.
- b) Apenas as afirmativas II e IV estão corretas.
- c) Apenas a afirmativa I está incorreta
- d) Apenas a afirmativa III está incorreta.

5. (AOCP – IBGE / 2019) Compreender o ciclo de vida das views das aplicações é extremamente importante, sobretudo quando falamos de aplicações para dispositivos móveis. Sobre o ciclo de vida das aplicações iOS com Swift, assinale a alternativa que apresenta um método que é chamado toda vez que uma visão vai aparecer na tela, podendo ser chamado mais de uma vez, e é muito usado para acionar quaisquer operações que precisem ocorrer antes que a ViewController seja apresentada na tela, como atualizar os dados do usuário.

- a) viewDidLoad()
- b) viewDidAppear()
- c) viewWillAppear()
- d) viewWillLoad()
- e) viewRestart()

6. (AOCP – IBGE / 2019) Você está desenvolvendo um aplicativo iOS usando Swift, que é uma agenda de controle de tarefas do funcionário do departamento de Tecnologia da Informação que presta o serviço de manutenção e suporte para os usuários da corporação. Nesse momento do desenvolvimento, é preciso fornecer uma maneira para nosso usuário sair da listagem de tarefas e ir para a tela de nova tarefa, ou seja, é necessário trabalhar com a navegação entre telas, pois o usuário precisa navegar entre a tela de listagem e a de nova tarefa, tanto a ida

quanto a volta. Para isso, é necessário ter uma barra de navegação. Assinale a alternativa que apresenta o que você deve utilizar para implementar essa ação.

- a) NavController
- b) Navigation Jetpacks
- c) Navigation Component
- d) Navigation Controller
- e) Routing Navigation

7. (AOCP – IBGE / 2019) Com o swift no desenvolvimento para iOS, a Apple adotou novas características e capacidades para a linguagem de programação, como o uso de protocolos. Estes trabalham de uma maneira que visa estender a funcionalidade de uma classe ou estrutura existente. Um protocolo pode ser pensado como um escopo ou interface que define um conjunto de propriedades e métodos. Um dos protocolos mais utilizados nessa linguagem de programação é o que tem a capacidade de determinar quando dois objetos são iguais e, com extensões condicionais a esse protocolo, é possível fornecer funcionalidade específica para tipos específicos de objetos em conformidade com um protocolo. Assinale a alternativa que apresenta corretamente o nome desse protocolo.

- a) Equatable.
- b) CollectionType.
- c) ViewController.
- d) NSCodering.
- e) MyDelegate.

8. (CESGRANRIO – Banco do Brasil / 2021) Um programador de aplicativos para dispositivos Apple com iOS recebeu a seguinte parte de um código, escrito na linguagem swift:

```
var i: Int
var texto: String
var num: Int = 0
var frase: String = ""

for i in 1...3 {
    num = num + 1 + i * 2
    switch num {
    case 2...6:
        texto = "a "
    case 7...9:
        texto = "casa "
    case 10...13:
        texto = "carro "
```

```
case 14...16:  
    texto = "eh "  
case 17...20:  
    texto = "o "  
case 21...23:  
    texto = "forte "  
default:  
    texto = "não eh "  
}  
frase = frase + texto  
}
```

print(frase)

A execução dessa parte do código produz como resposta

- a) o carro eh
- b) a casa eh
- c) o carro não eh
- d) a casa eh forte
- e) o carro eh forte

9. (FEPese – CELESC / 2022) Caso seja necessário o desenvolvimento de aplicativos para dispositivos móveis, mais especificamente para o sistema operacional IOS, assinale a alternativa que apresenta corretamente uma linguagem de programação e um ambiente de desenvolvimento que podem ser utilizados para este fim.

- a) Swift e XCODE
- b) Swift e Objective-D
- c) Objective-D e iOSCODE
- d) Objective-D e VSCODE
- e) XCODE e VSCODE

10. (CEBRASPE – TRT - 7ª Região (CE) / 2017) Assinale a opção que apresenta a linguagem de programação disponível, grátis e em código aberto, para desenvolvedores sob a licença Apache 2.0 e desenvolvida pela Apple para a criação de aplicativos para IOS.

- a) Xcode
- b) Swift
- c) Objective-C
- d) Python

11. (AOCP – PRODEB / 2018) Considerando as linguagens de programação mobile, qual das dispostas a seguir foi criada pela Apple e pode ser utilizada para o desenvolvimento das suas aplicações?

- a) Kotlin
- b) Swift
- c) C#
- d) Python
- e) IOS

12. (IBADE – Prefeitura de Itapemirim - ES / 2019) A Apple desenvolveu uma linguagem de programação própria para desenvolvimento de aplicações sob IOS. Ela se chama:

- a) C++.
- b) PHP.
- c) Python.
- d) Swift.
- e) Java Script.

Flutter

1. **(CEBRASPE – TRT - 8ª Região (PA e AP) / 2022)** O widget básico do Flutter que permite criar layouts flexíveis nas direções horizontal e vertical, com design de objetos baseado no modelo de layout flexbox da Web é o
- a) Text.
 - b) Row, Column.
 - c) Stack.
 - d) Expanded.
 - e) Container.
2. **(IBFC – EBSEH / 2020)** Quanto aos frameworks mais destacados para o desenvolvimento de aplicativos mobile, analise as afirmativas abaixo quanto a existência dos mesmos e dê valores Verdadeiro (V) ou Falso (F).

() Flutter () Corona SDK () JQuery Mobile

Assinale a alternativa que apresenta a sequência correta de cima para baixo:

- a) V, V, V
 - b) V, V, F
 - c) V, F, V
 - d) F, F, V
 - e) F, F, F
3. **(UFC – UFC / 2019 – Adaptada)** Sobre o desenvolvimento de aplicações móveis, assinale a alternativa correta.

Flutter é um SDK de código aberto criado pelo Google para o desenvolvimento de aplicativos para dispositivos móveis utilizado para desenvolver aplicativos para Android e iOS.

GABARITO

Swift

- | | | |
|------------|-------------|-------------|
| 1. Letra D | 6. Letra D | 11. Letra B |
| 2. Letra B | 7. Letra A | 12. Letra D |
| 3. Letra C | 8. Letra B | |
| 4. Letra B | 9. Letra A | |
| 5. Letra C | 10. Letra B | |

ESSA LEI TODO MUNDO CONHECE: PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.



1 Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.



2 Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).



3 Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).



4 Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).



5 Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).



6 Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).



7 Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.



8 O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.